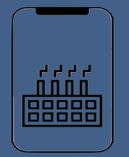
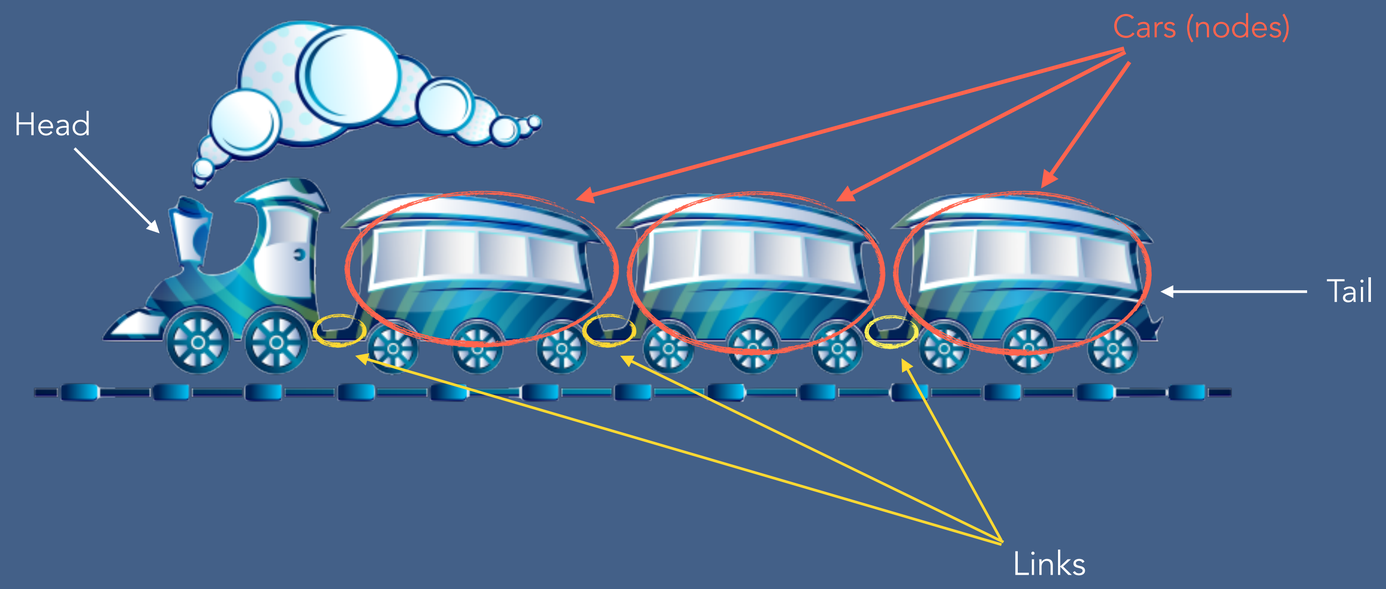
Linked List



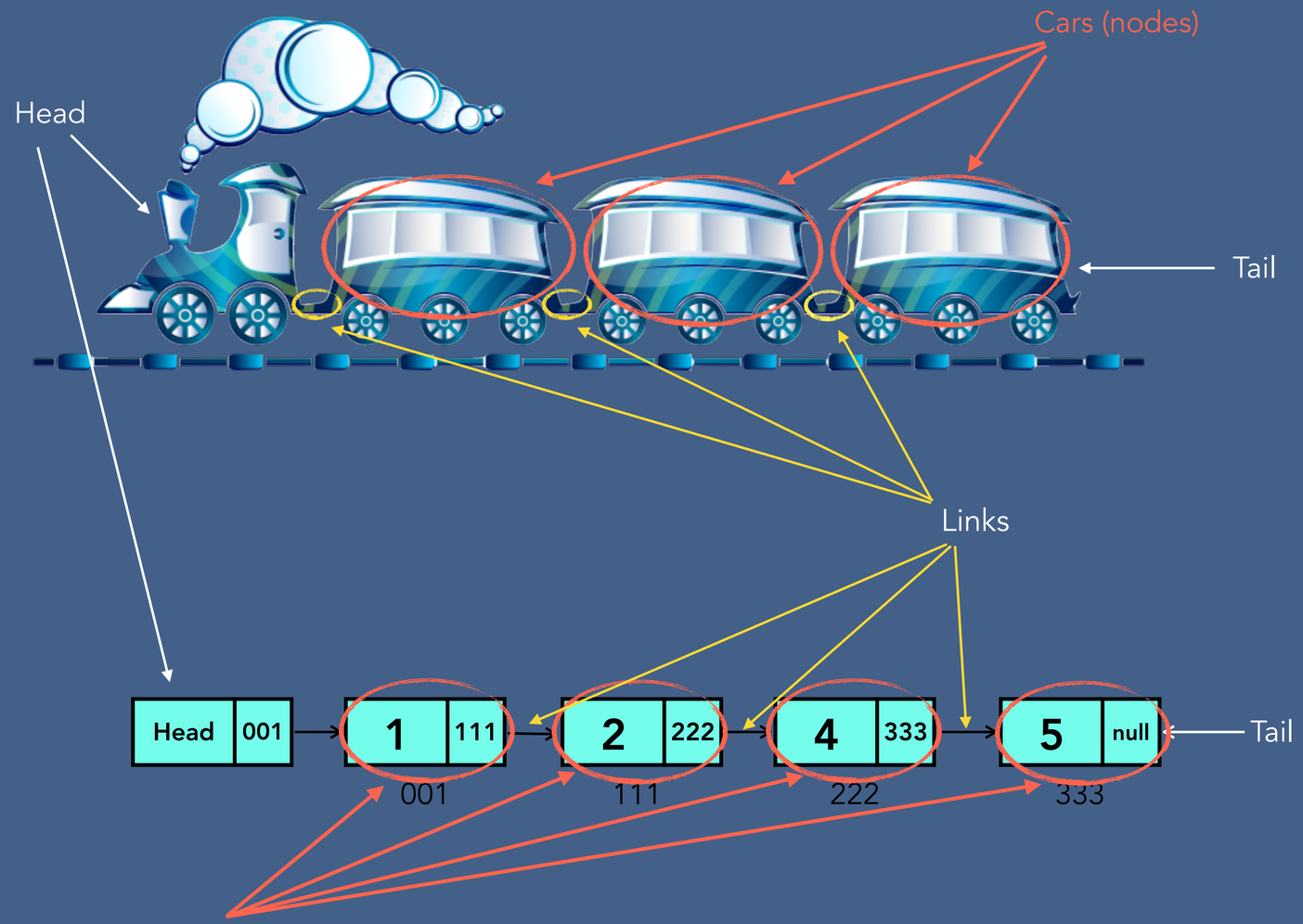
What is a Linked List?

Linked List is a form of a sequential collection and it does not have to be in order. A Linked list is made up of independent nodes that may contain any type of data and each node has a reference to the next node in the link.

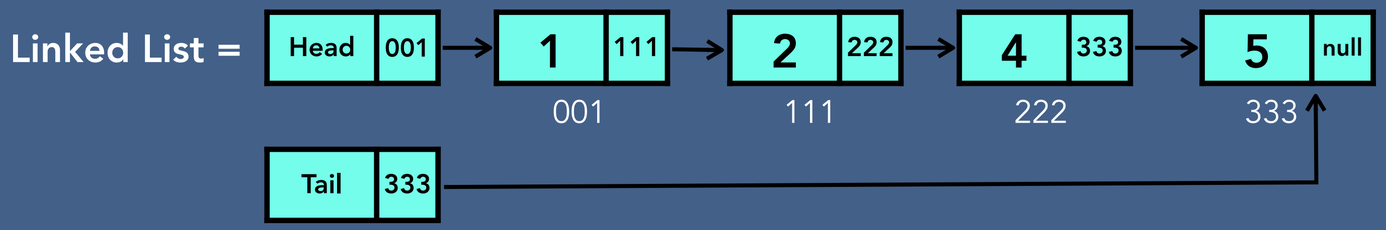


- Each car is independent
- Cars : passengers and links (nodes: data and links)

What is a Linked List?



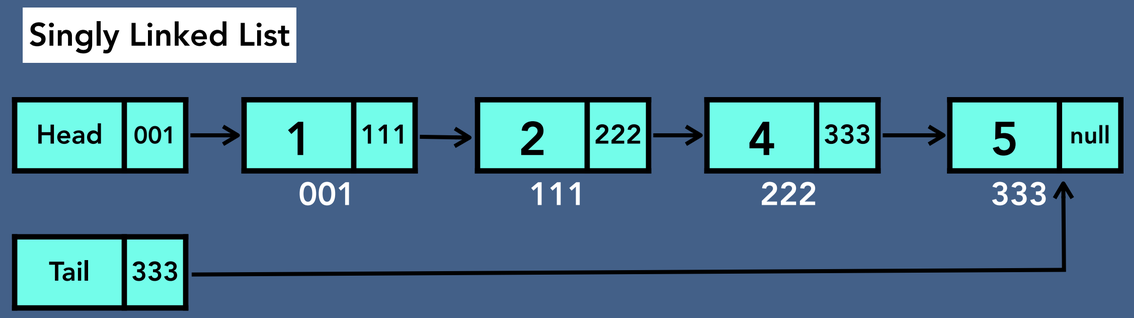
Linked List vs Array

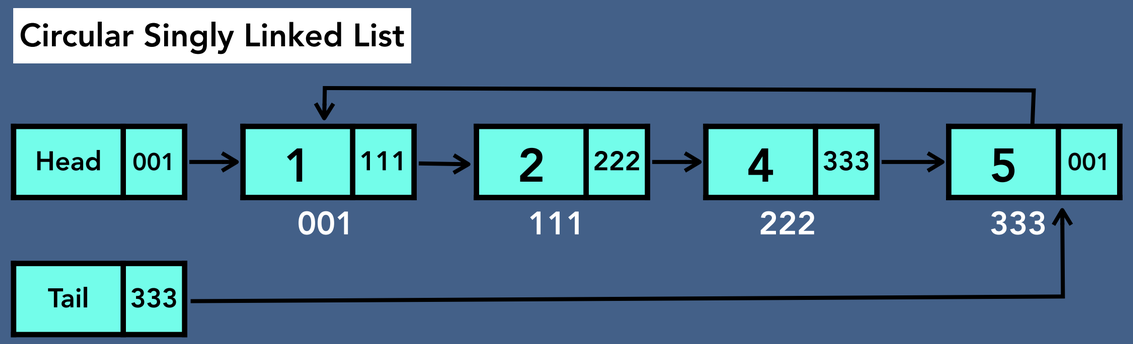


- Elements of Linked list are independent objects
- Variable size - the size of a linked list is not predefined
- Random access - accessing an element is very efficient in arrays

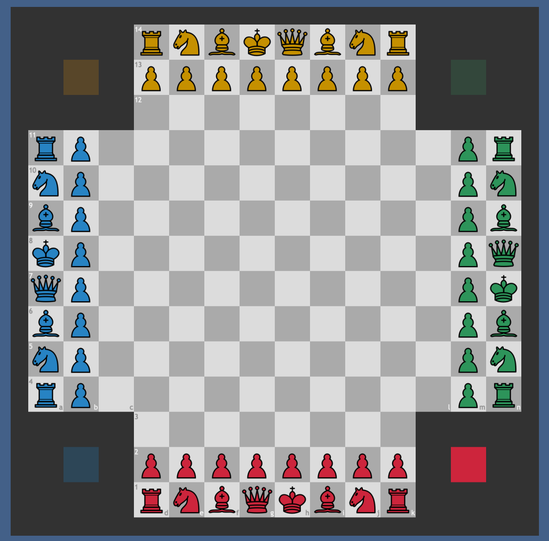
Types of Linked List

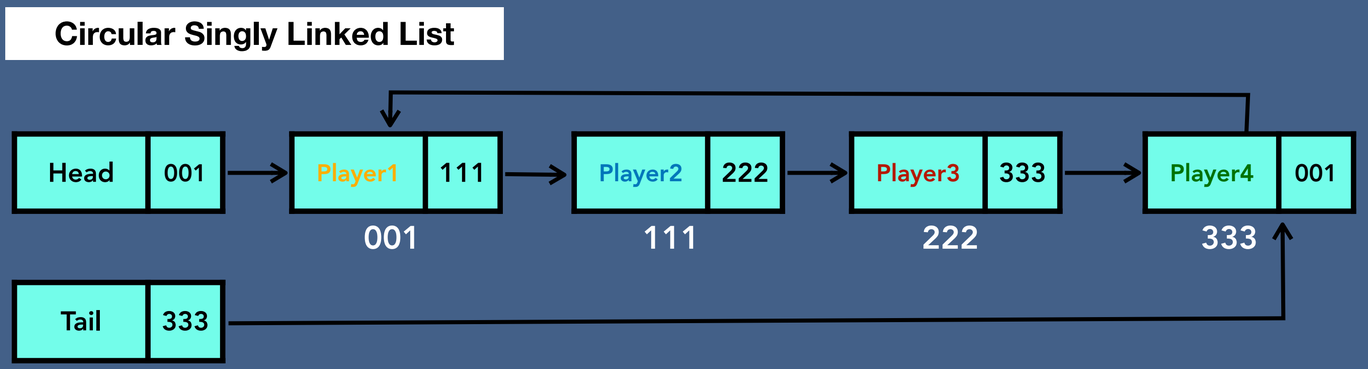
- Singly Linked List
- Circular Singly Linked List
- Doubly Linked List
- Circular Doubly Linked List





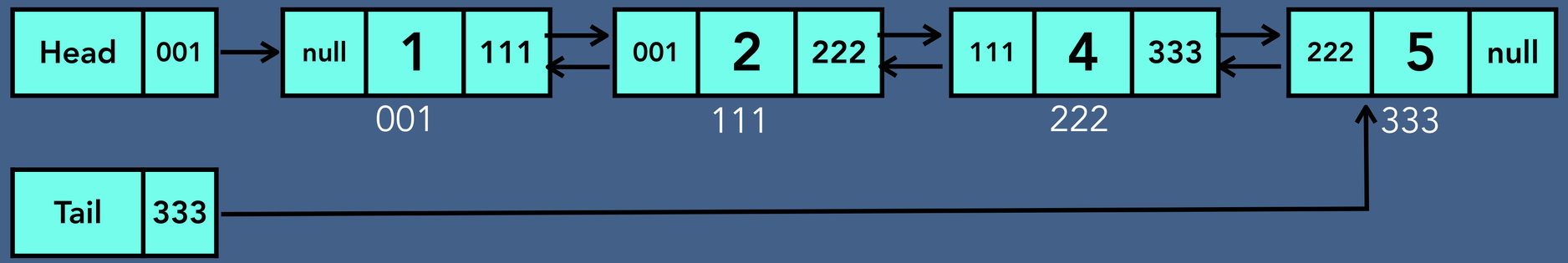
Types of Linked List



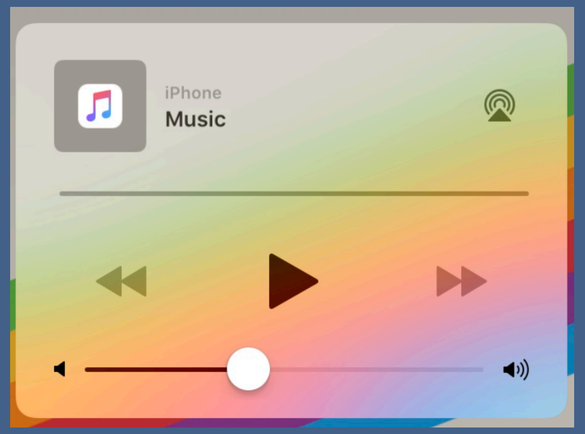


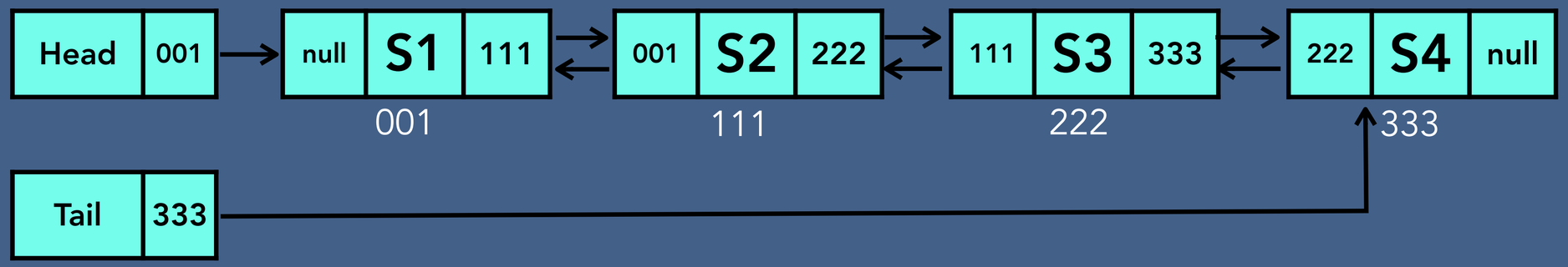
Types of Linked List

Doubly Linked List



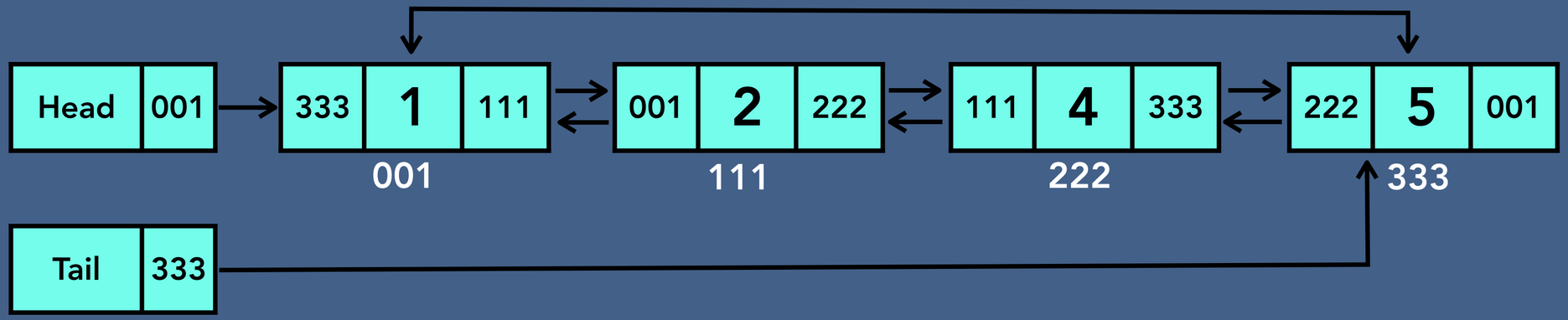
Types of Linked List





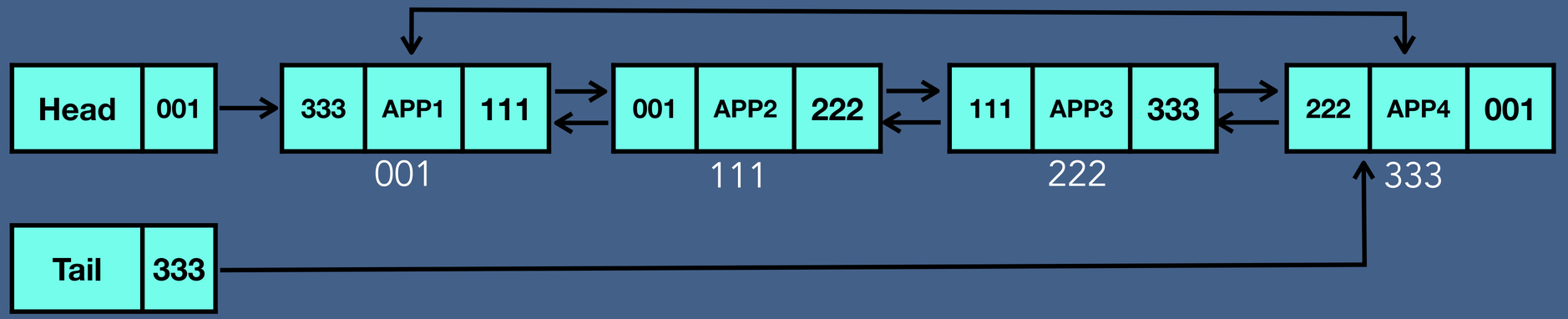
Types of Linked List

Circular Doubly Linked List



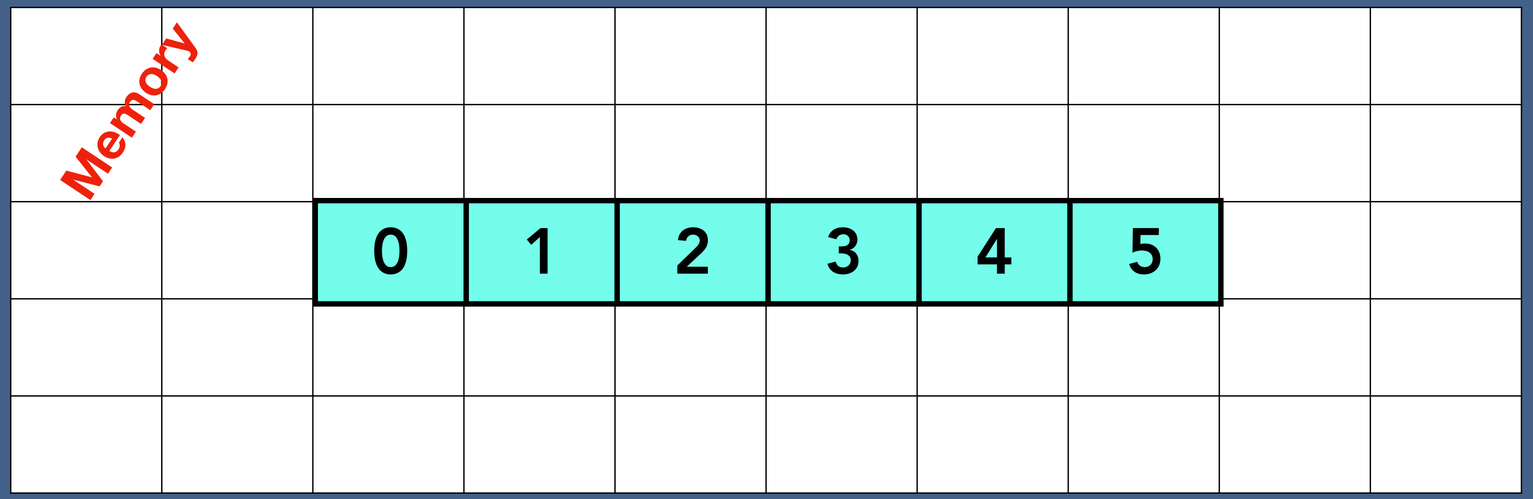
Types of Linked List

Cmd+shift+tab



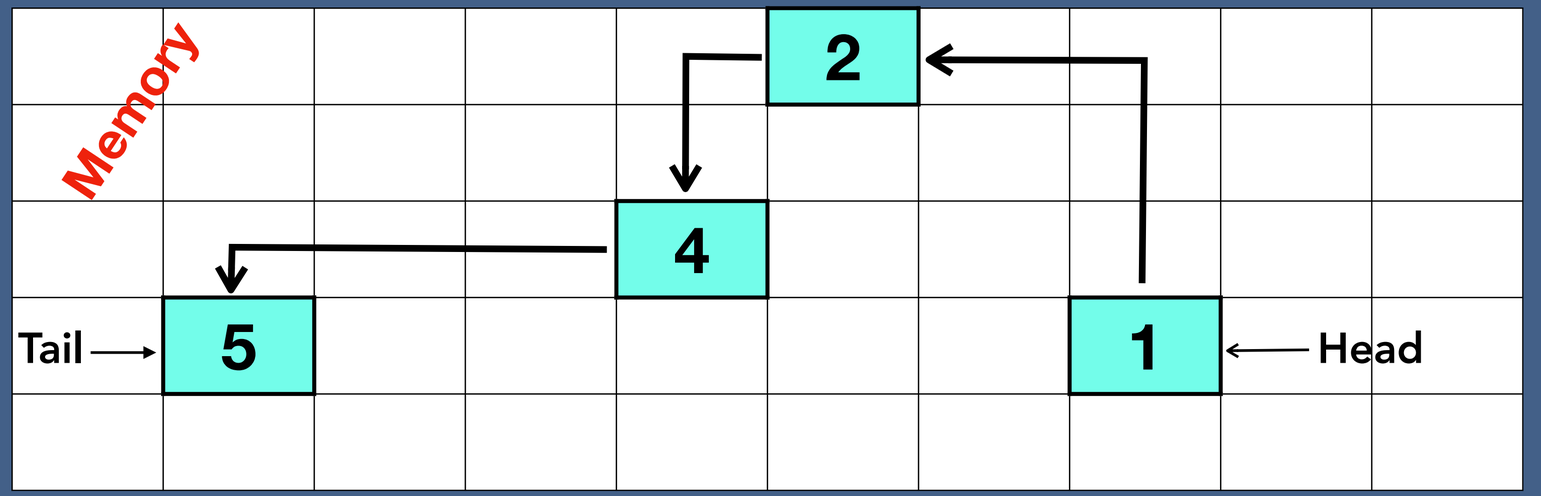
Linked List in Memory

Arrays in memory :

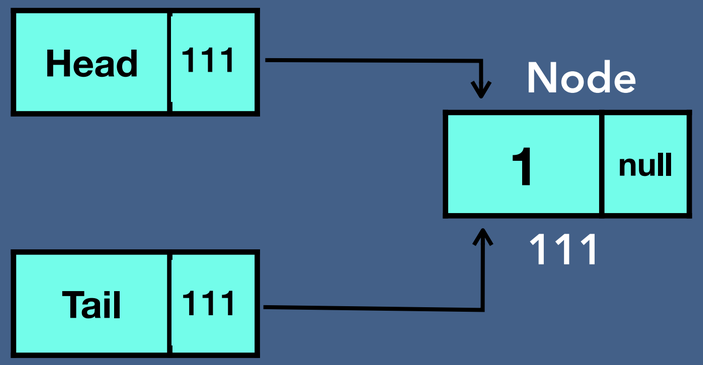


Linked List in Memory

Linked list:



Creation of Singly Linked List

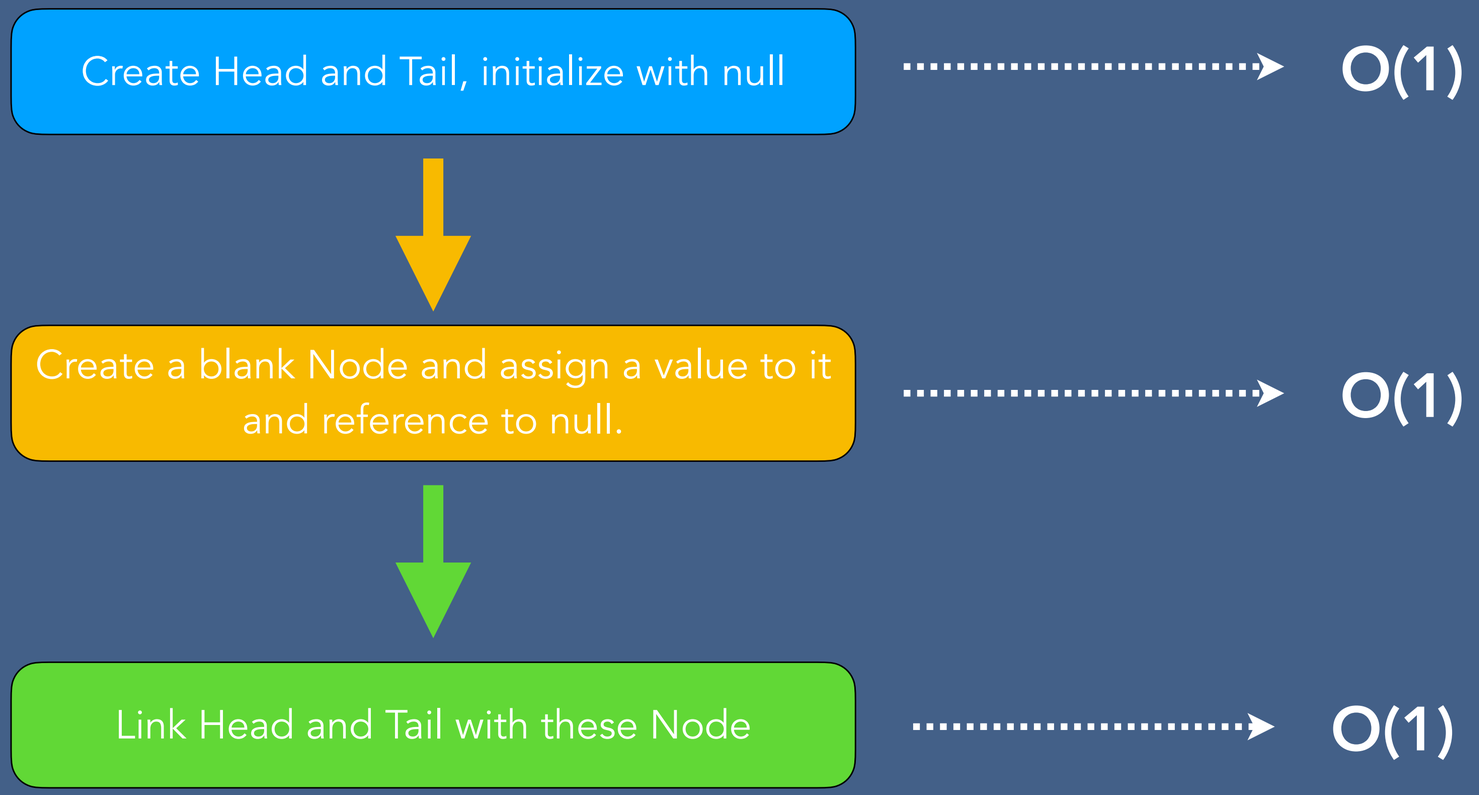


Create Head and Tail, initialize with null

Create a blank Node and assign a value to it and reference to null.

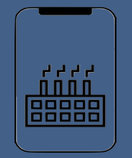
Link Head and Tail with these Node

Creation of Singly Linked List



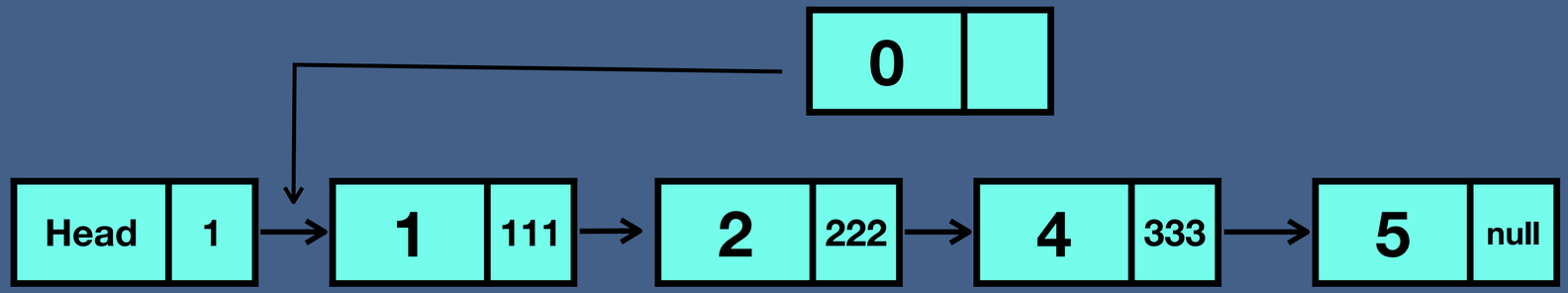
Insertion to Linked List in Memory

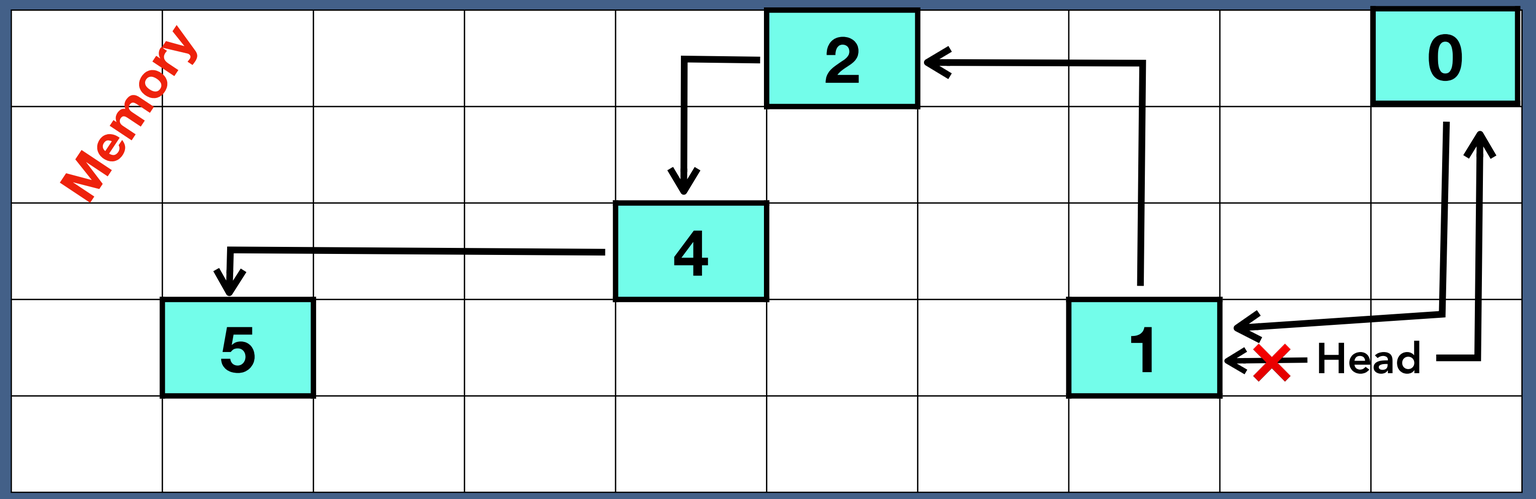
1. At the beginning of the linked list.
2. After a node in the middle of linked list
3. At the end of the linked list.



Insertion to Linked List in Memory

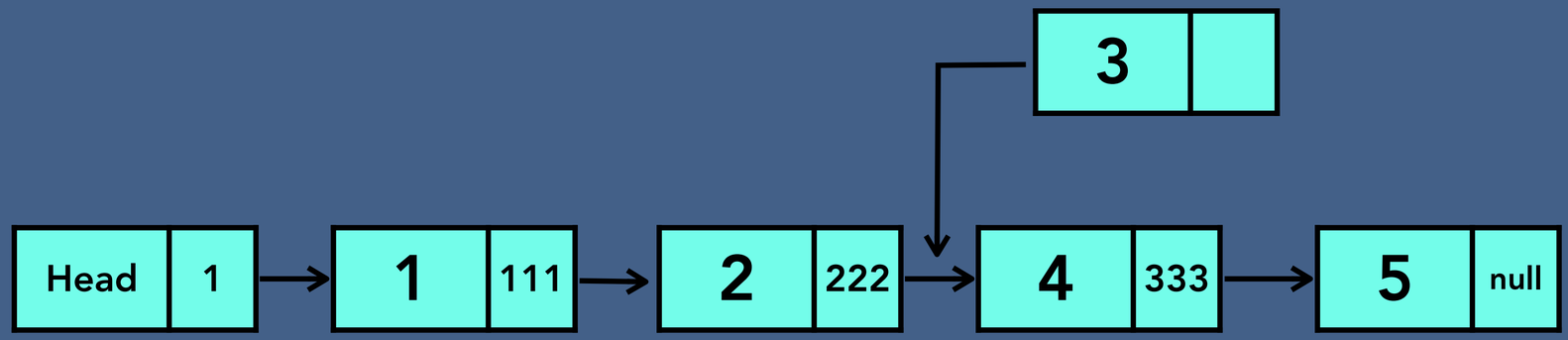
Insert a new node at the beginning

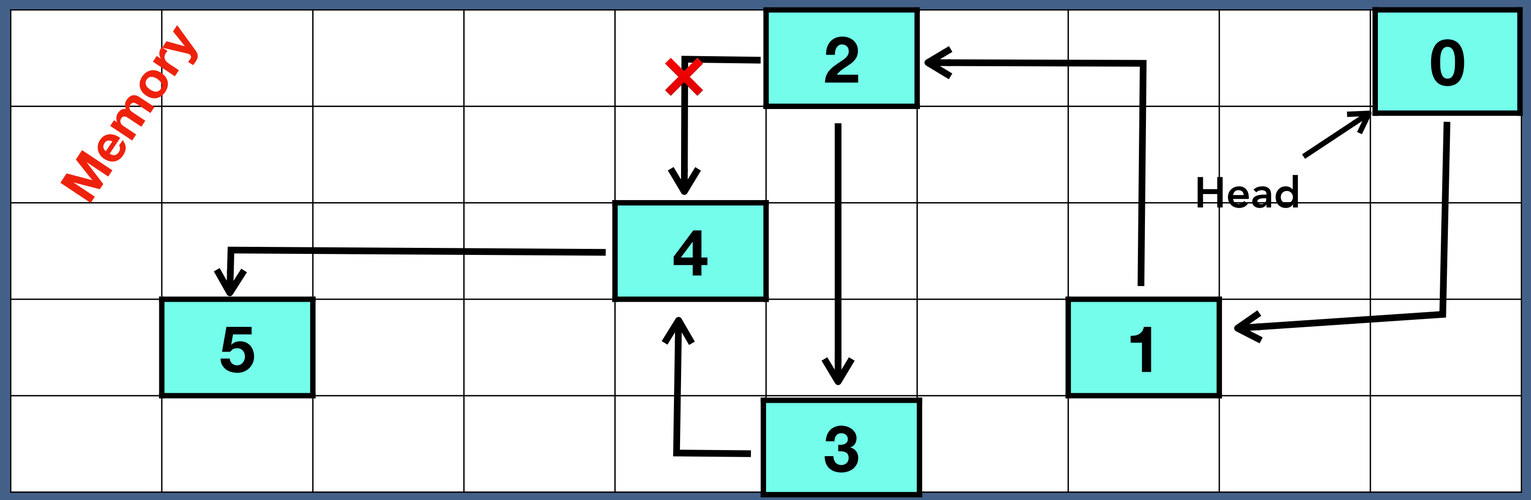




Insertion to Linked List in Memory

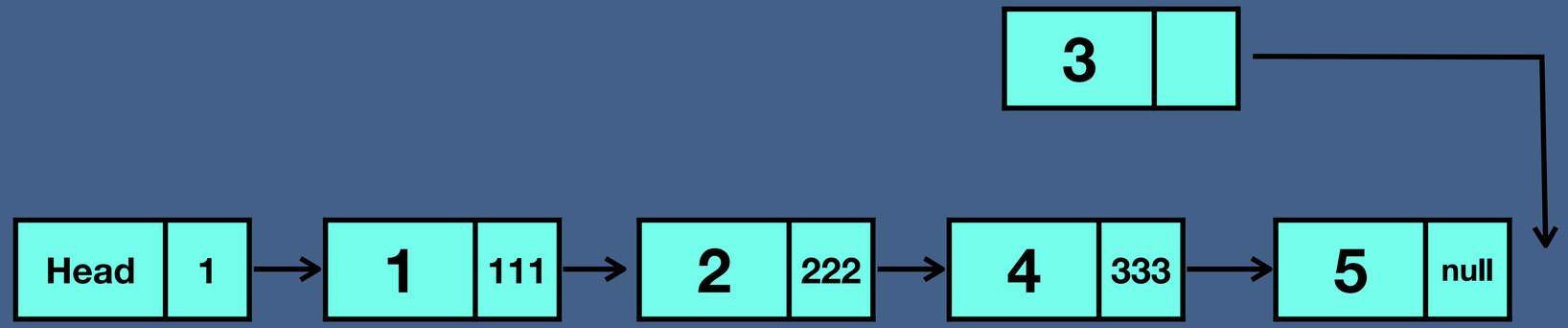
Insert a new node after a node

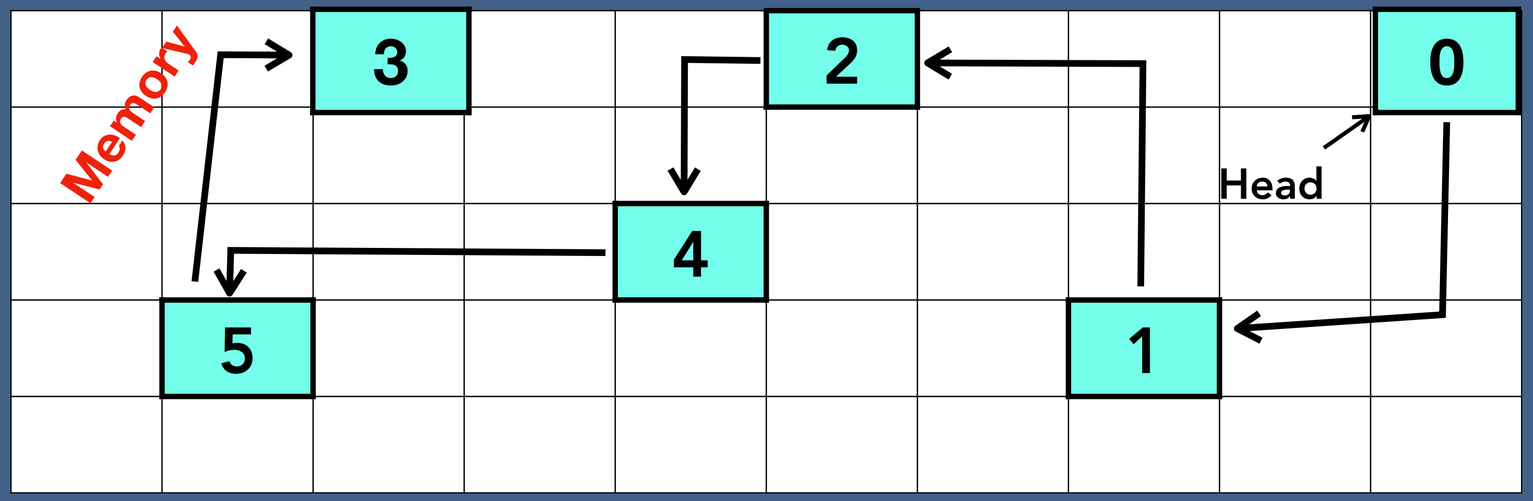




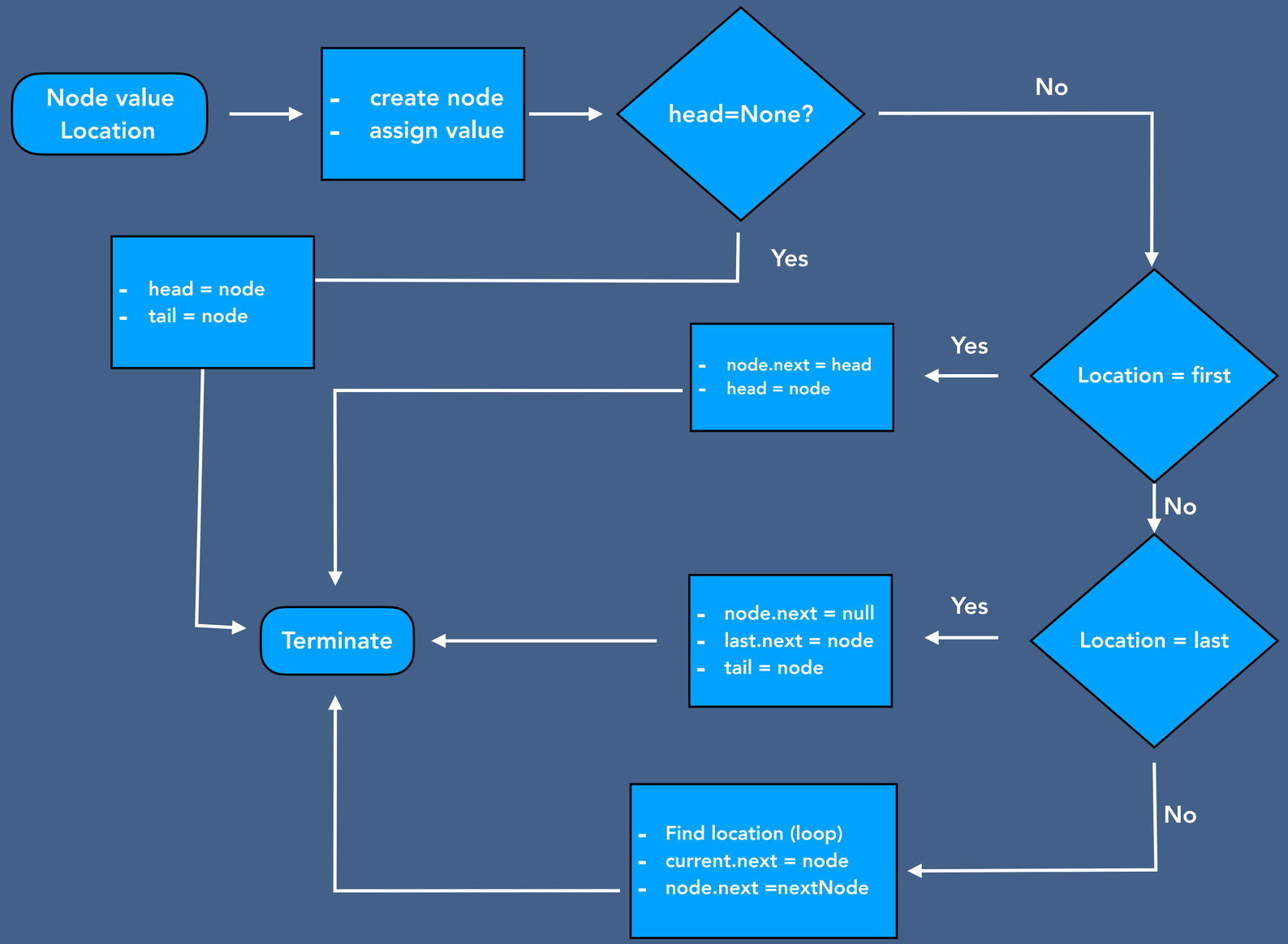
Insertion to Linked List in Memory

Insert a new node at the end of linked list

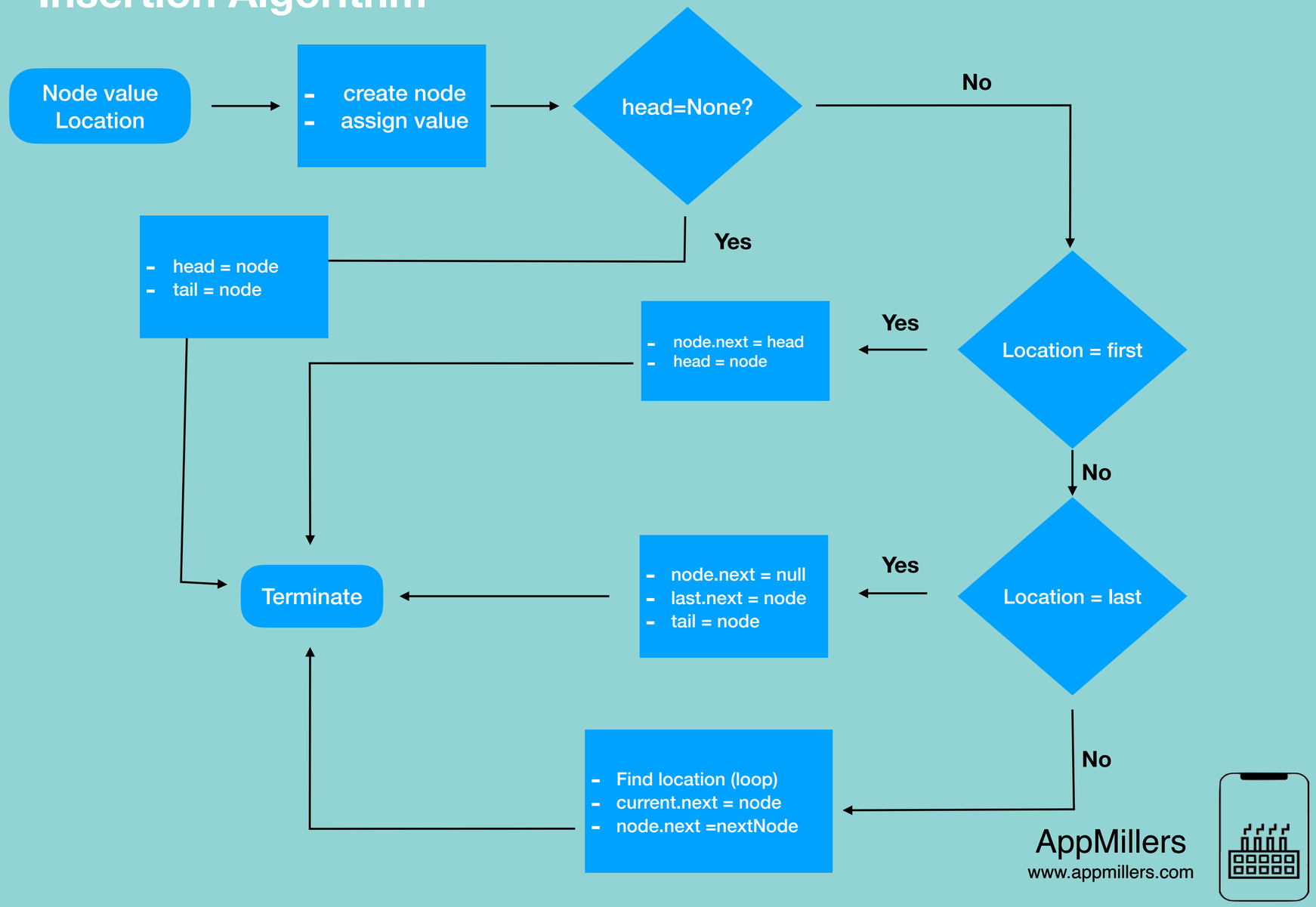




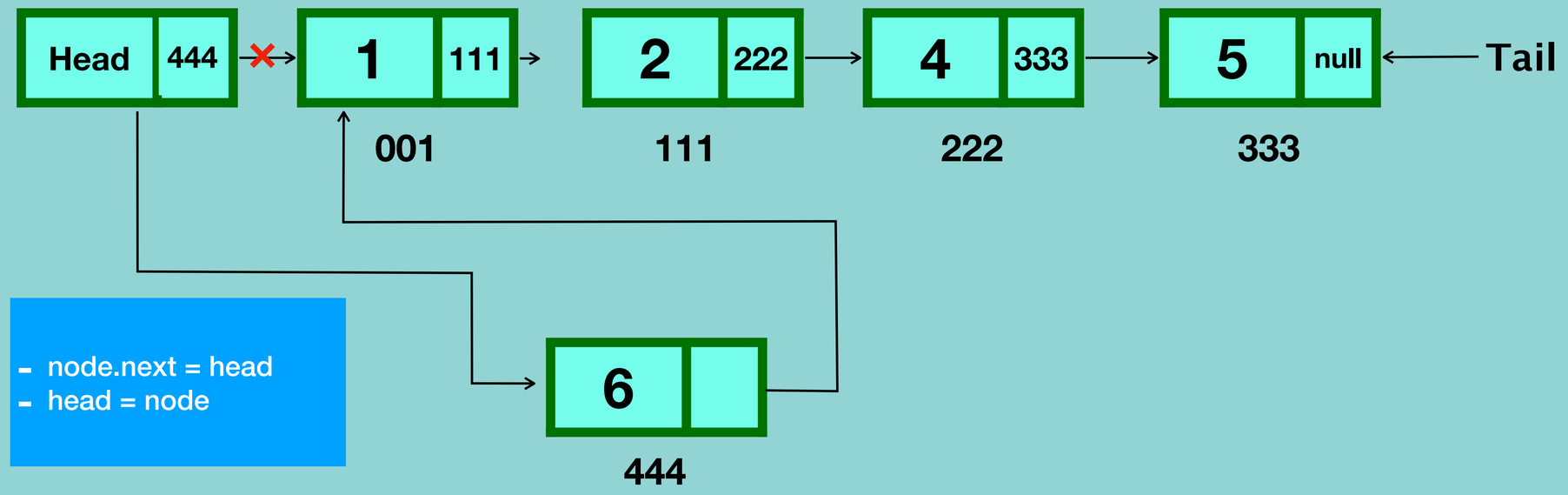
Insertion Algorithm



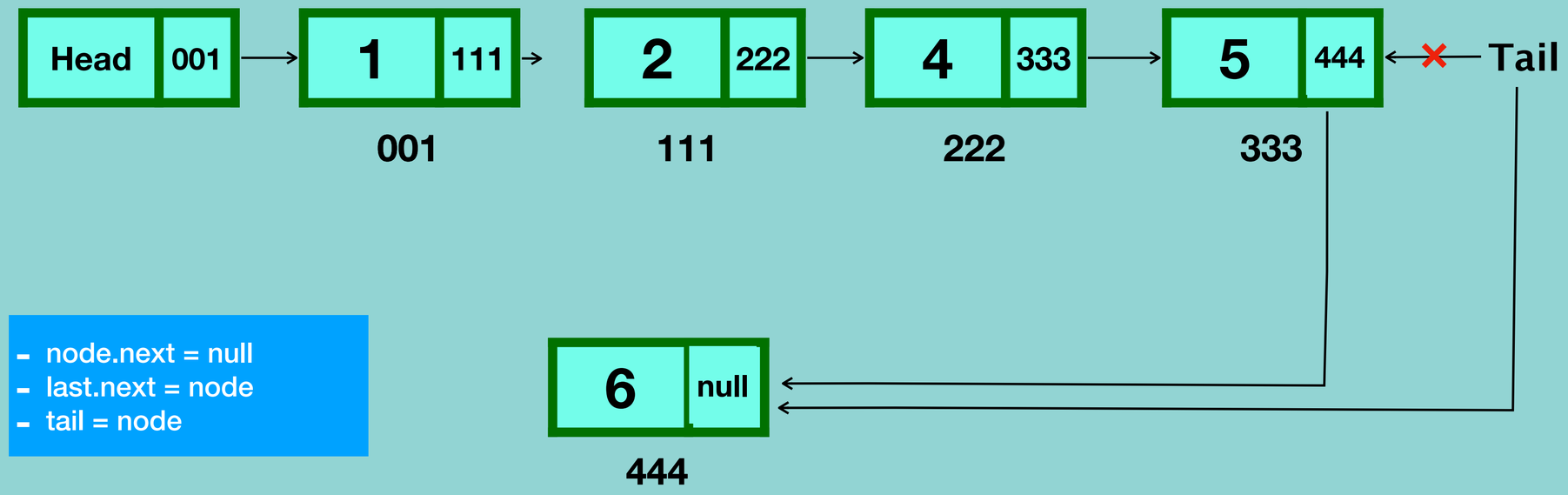
Insertion Algorithm



Singly Linked List Insertion at the beginning

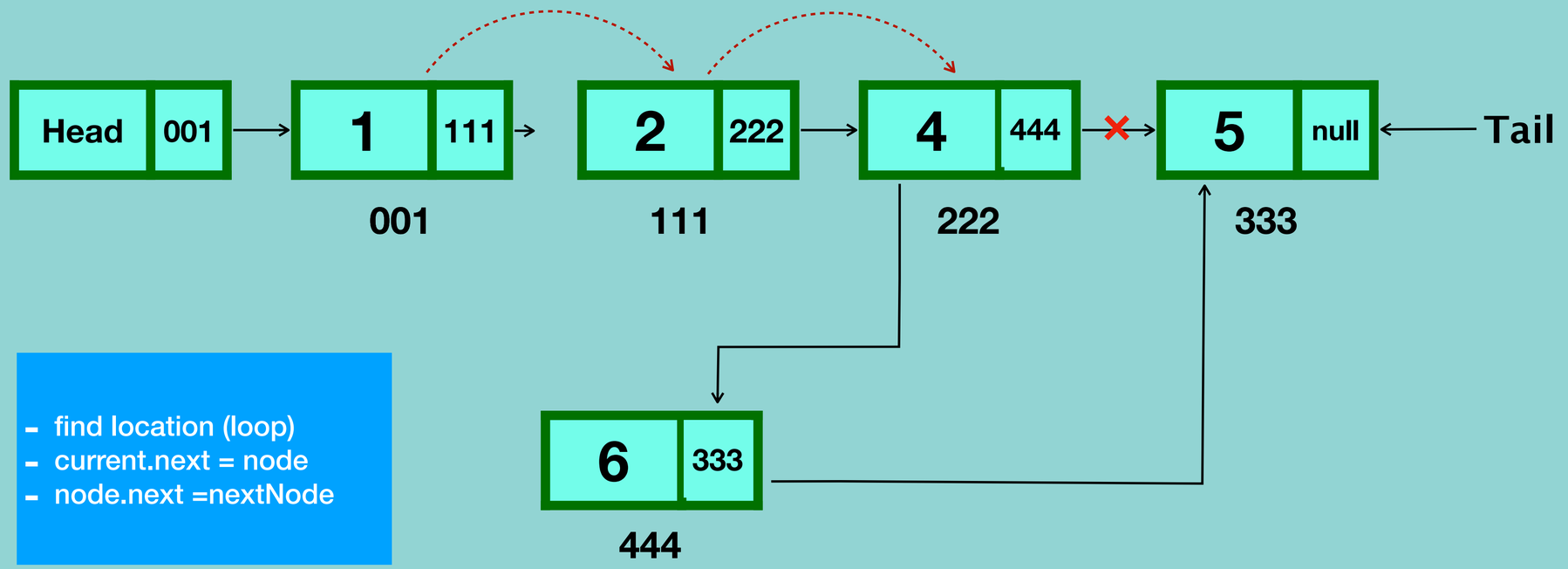


Singly Linked List Insertion at the end



AppMillers
www.appmillers.com

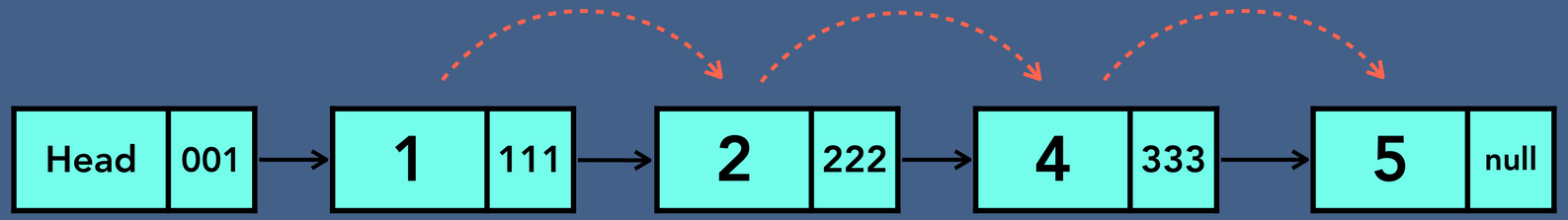
Singly Linked List Insertion in the middle

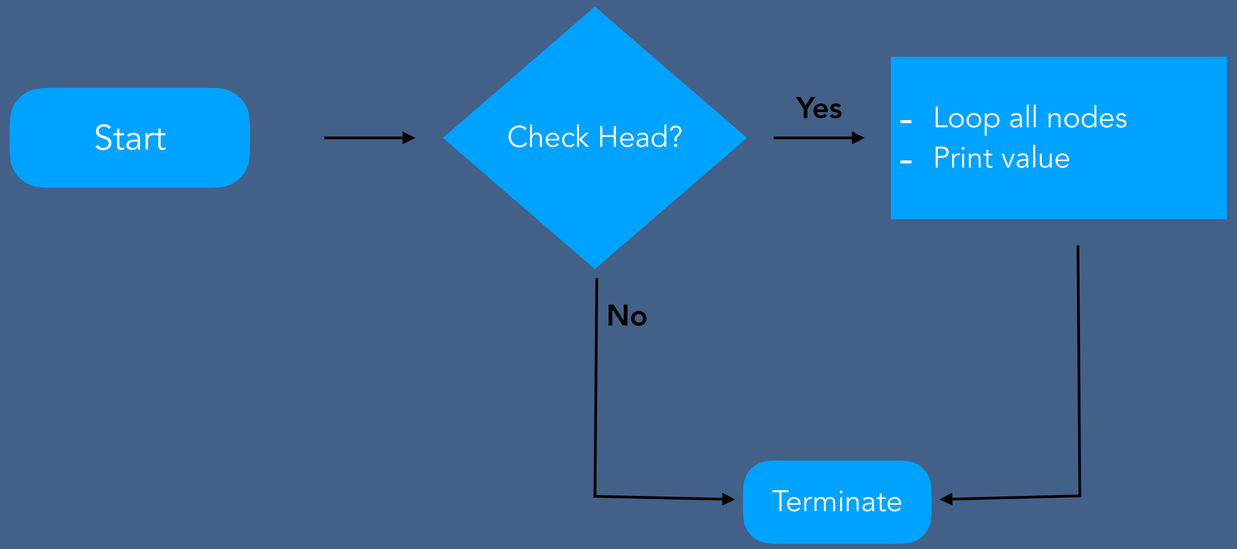


- find location (loop)
- current.next = node
- node.next =nextNode

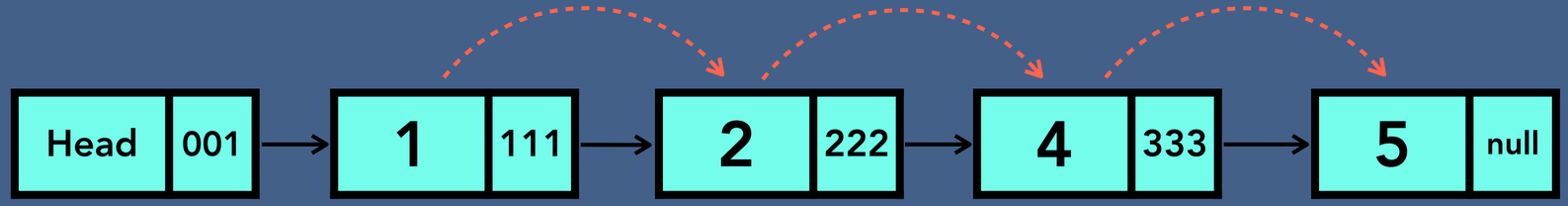
AppMillers
www.appmillers.com

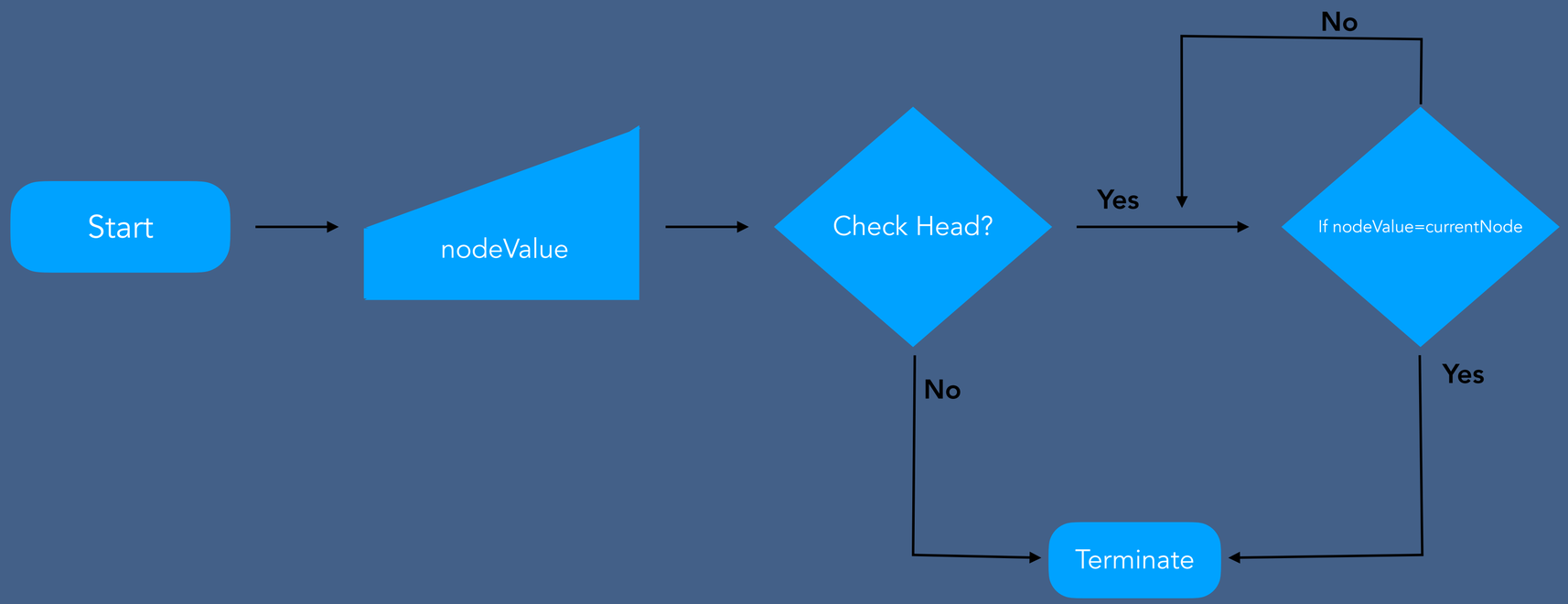
Traversal of Singly Linked List





Search in Singly Linked List

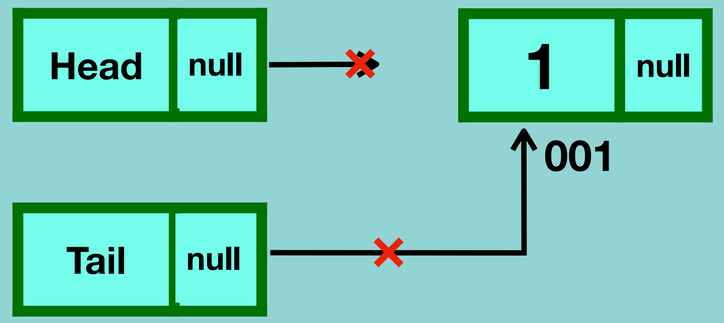




Singly Linked list Deletion

- Deleting the first node
- Deleting any given node
- Deleting the last node

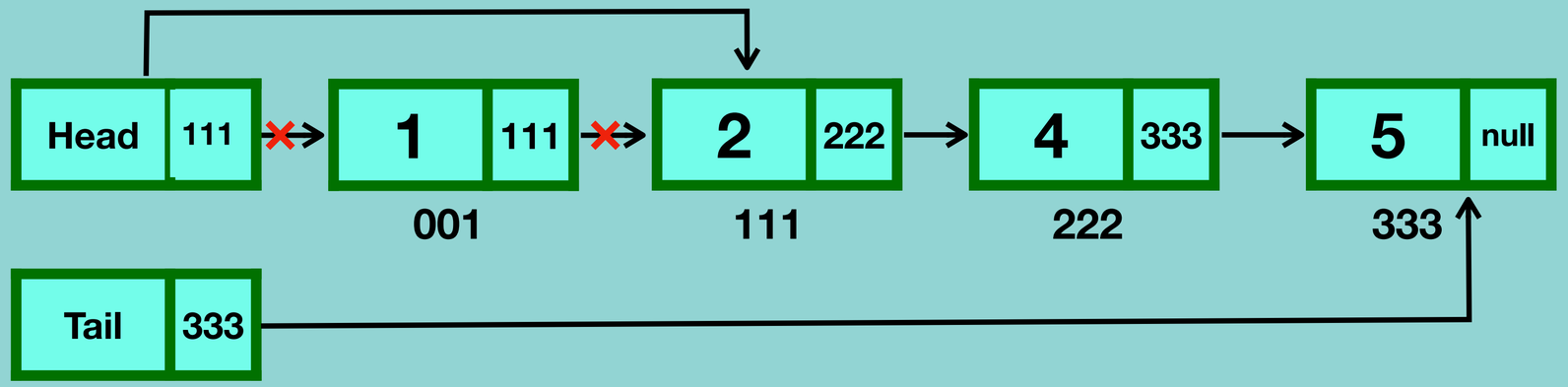
Case 1 - one node



Singly Linked list Deletion

- Deleting the first node
- Deleting any given node
- Deleting the last node

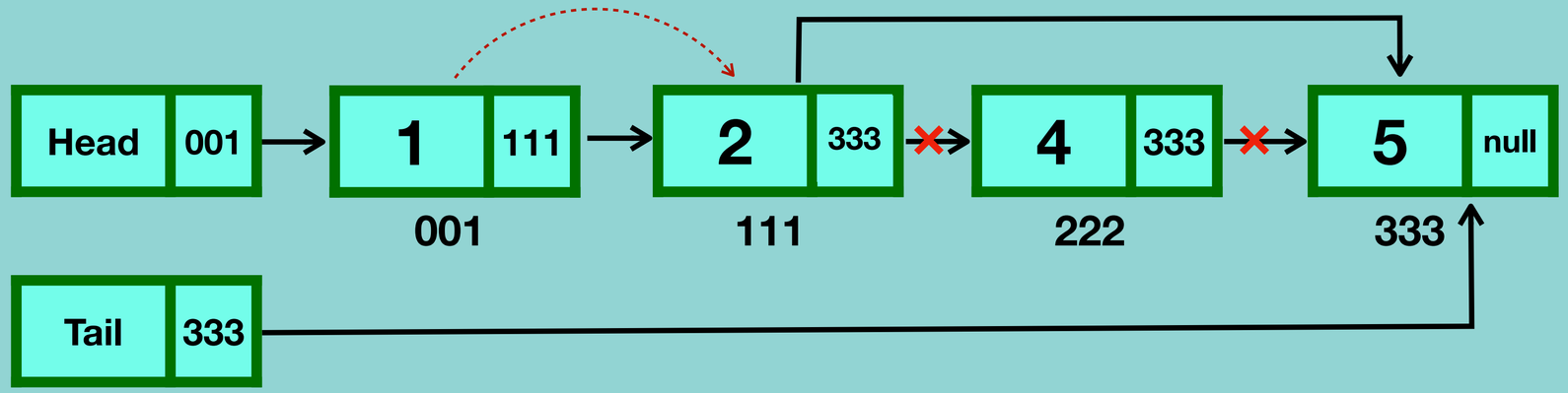
Case 2 - more than one node



AppMillers
www.appmillers.com

Singly Linked list Deletion

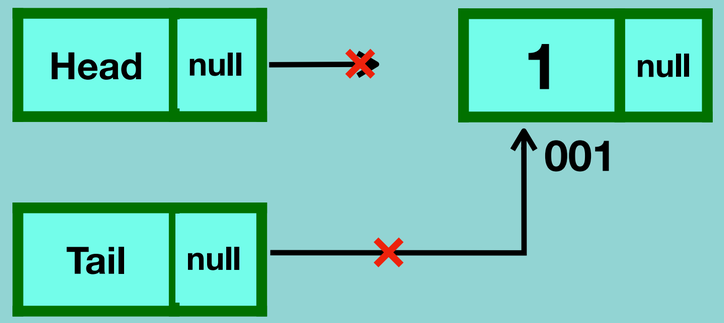
- Deleting the first node
- Deleting any given node
- Deleting the last node



Singly Linked list Deletion

- Deleting the first node
- Deleting any given node
- Deleting the last node

Case 1 - one node

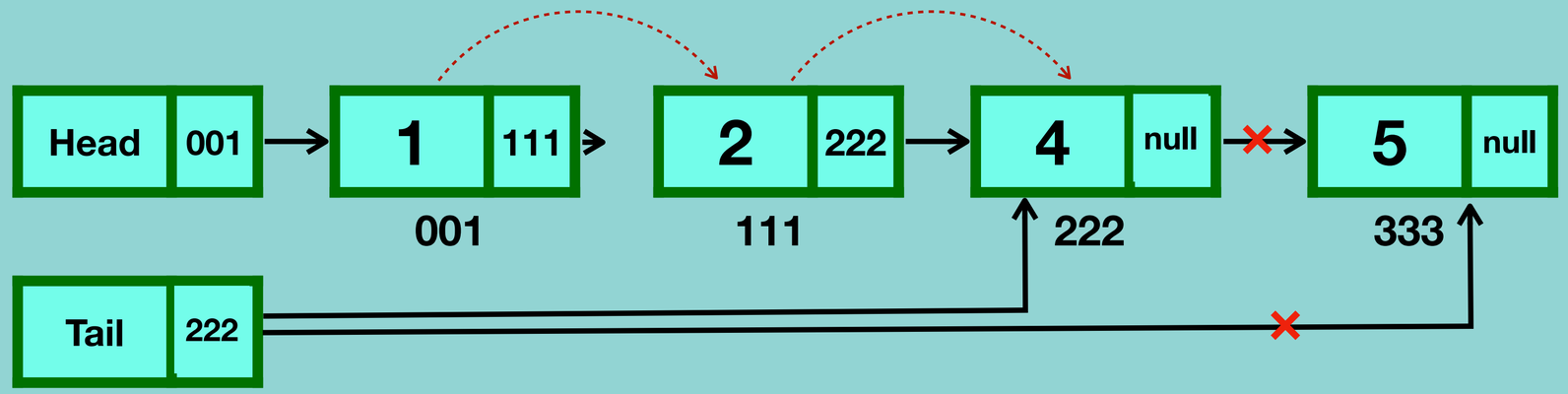


AppMillers
www.appmillers.com

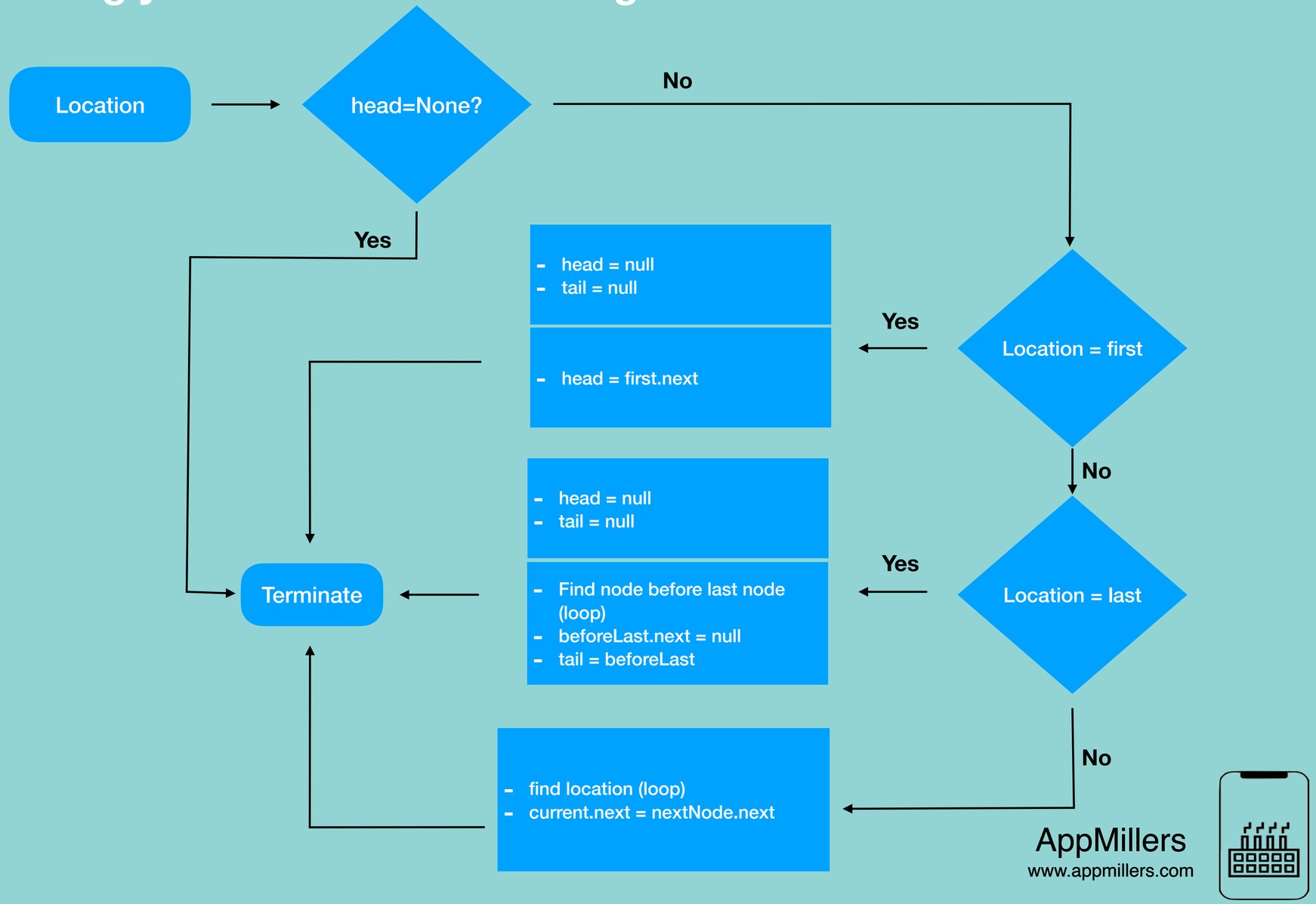
Singly Linked list Deletion

- Deleting the first node
- Deleting any given node
- Deleting the last node

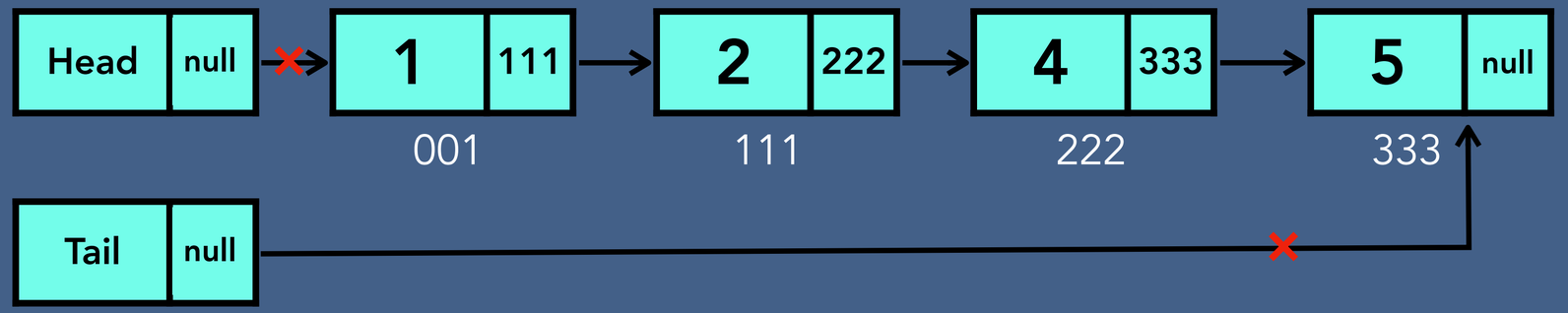
Case 2 - more than one node



Singly Linked list Deletion Algorithm

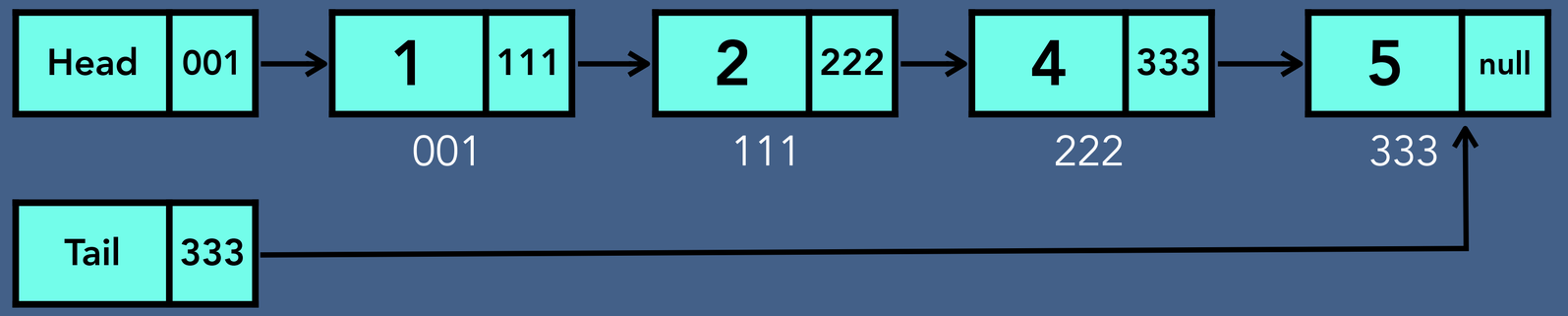


Delete entire Singly Linked List

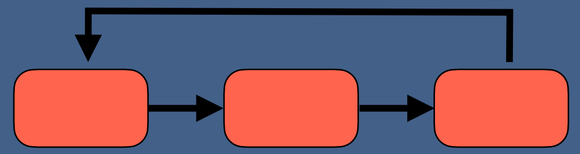


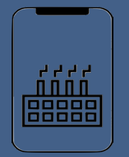
Time and Space Complexity of Singly Linked List

Singly Linked List	Time complexity	Space complexity
Creation	O(1)	O(1)
Insertion	O(n)	O(1)
Searching	O(n)	O(1)
Traversing	O(n)	O(1)
Deletion of a node	O(n)	O(1)
Deletion of linked list	O(1)	O(1)



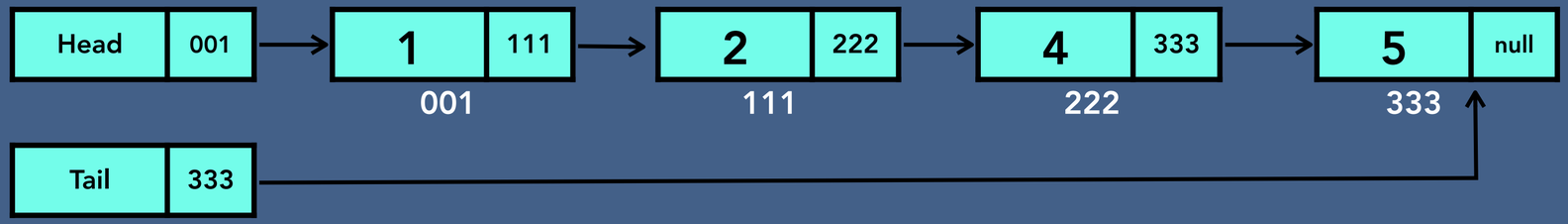
Circular Singly Linked List



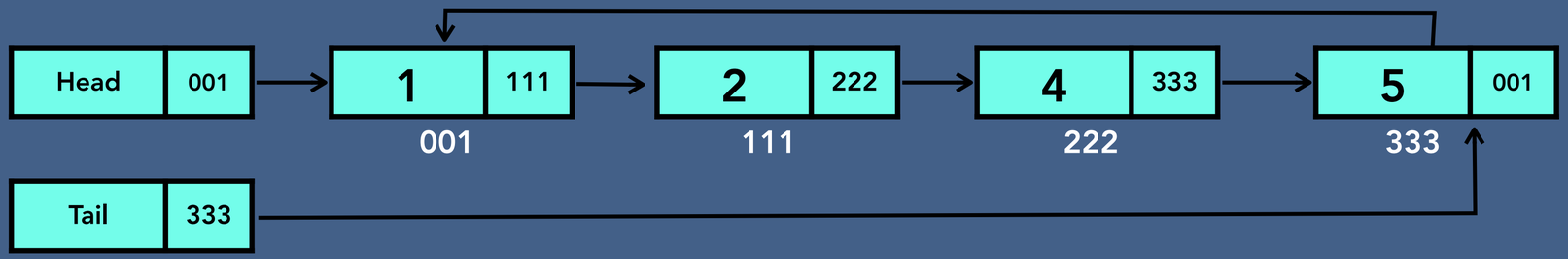


Circular Singly Linked List

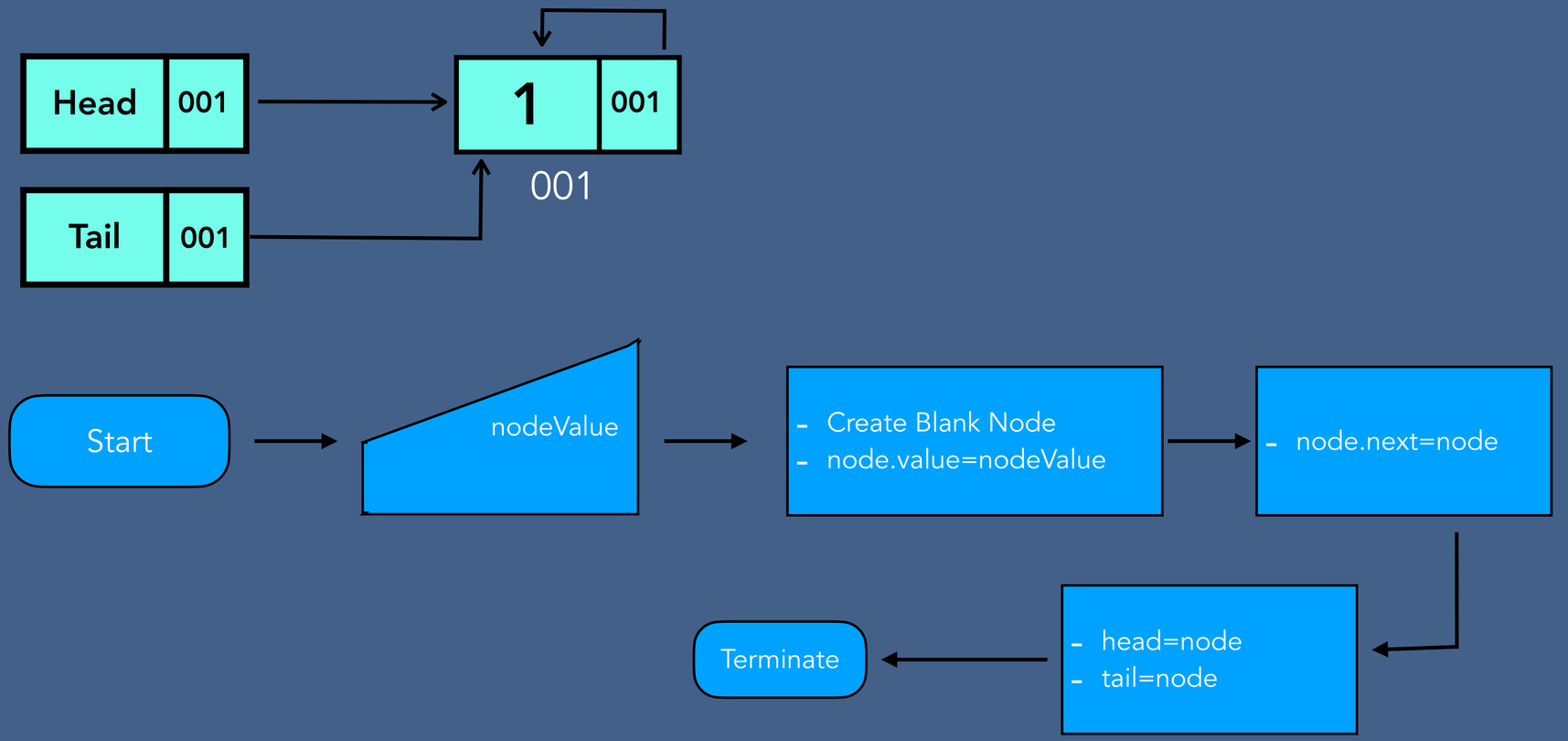
Singly Linked List



Circular Singly Linked List

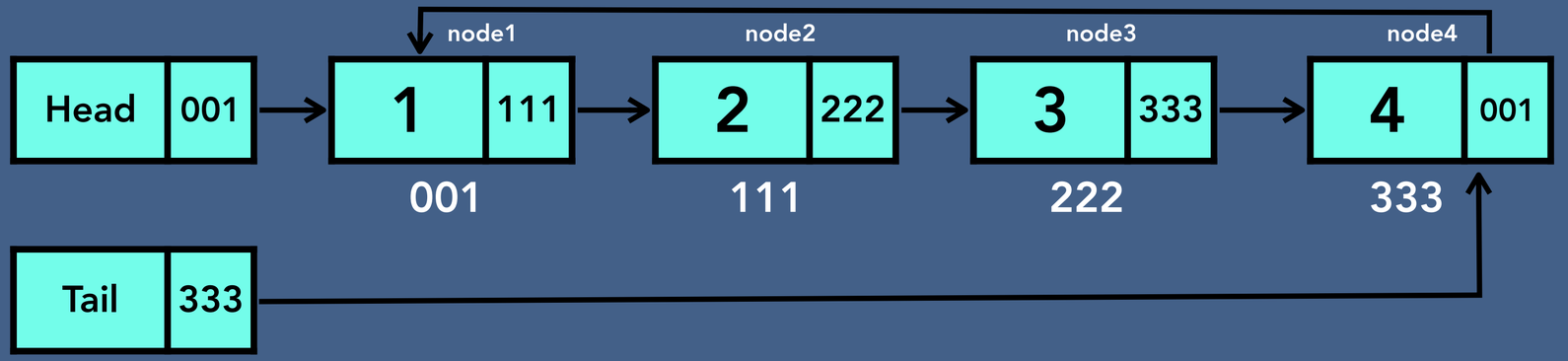


Create - Circular Singly Linked List



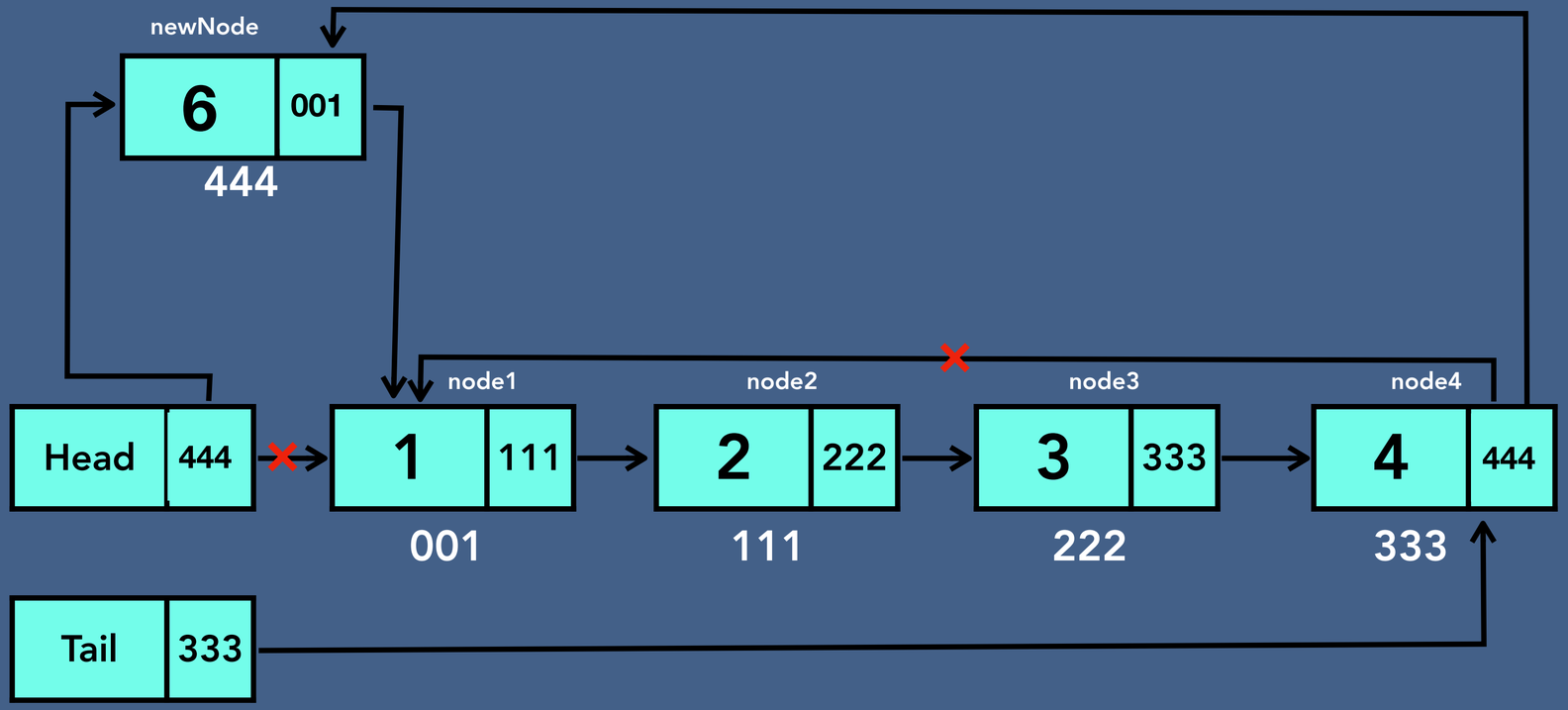
Insertion - Circular Singly Linked List

- Insert at the beginning of linked list
- Insert at the specified location of linked list
- Insert at the end of linked list



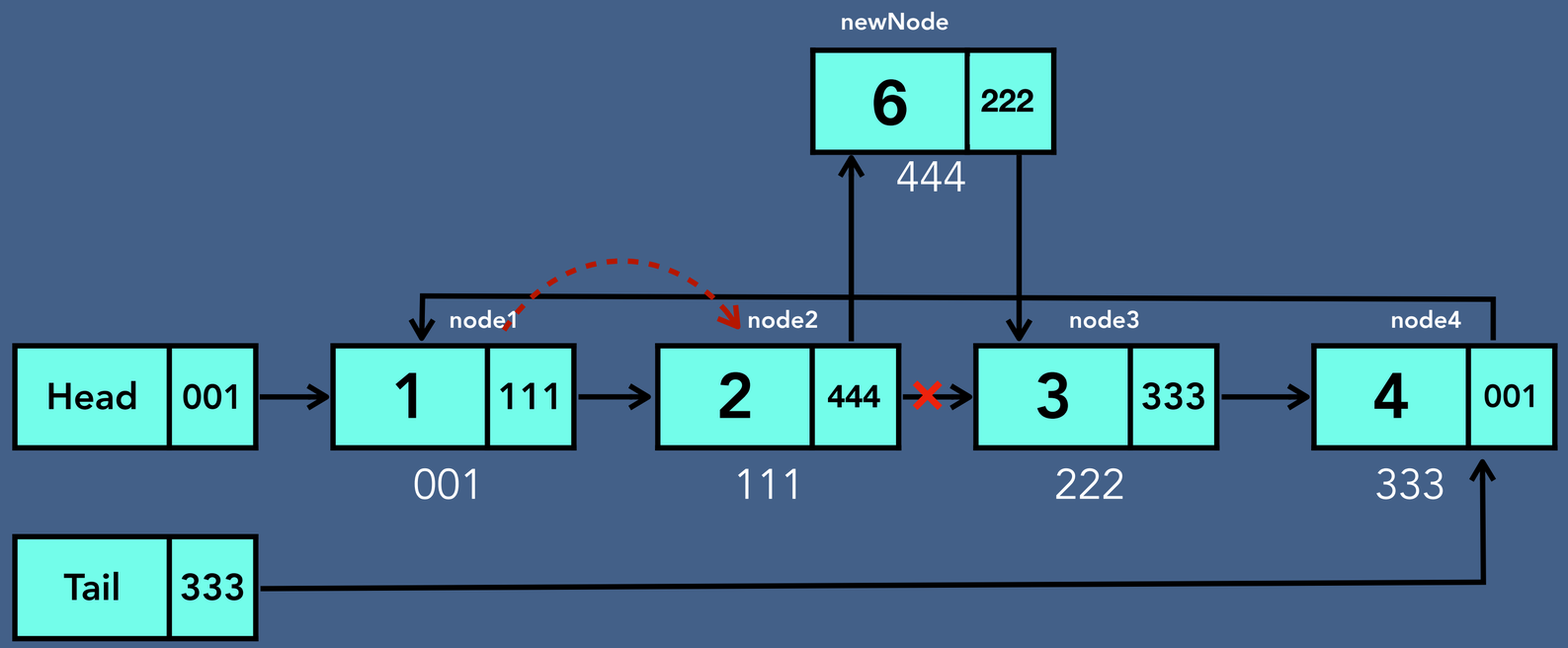
Insertion - Circular Singly Linked List

- Insert at the beginning of linked list



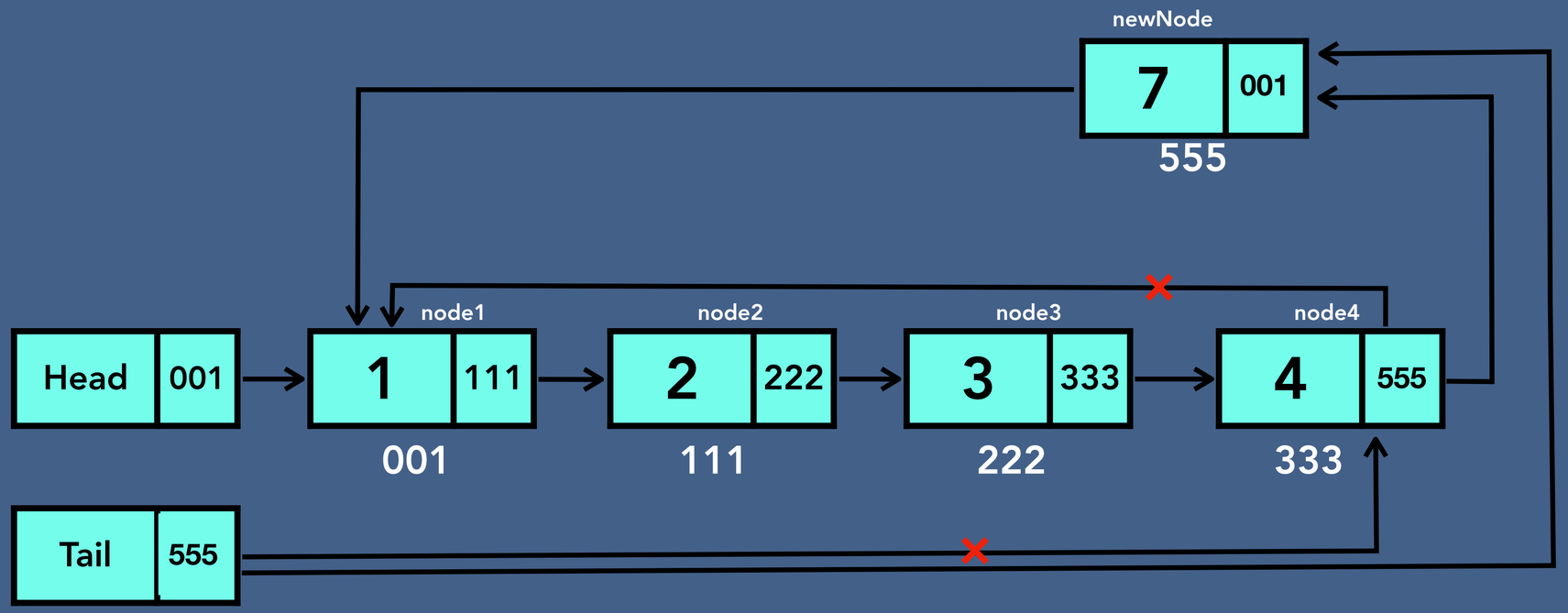
Insertion - Circular Singly Linked List

- Insert at the specified location of linked list

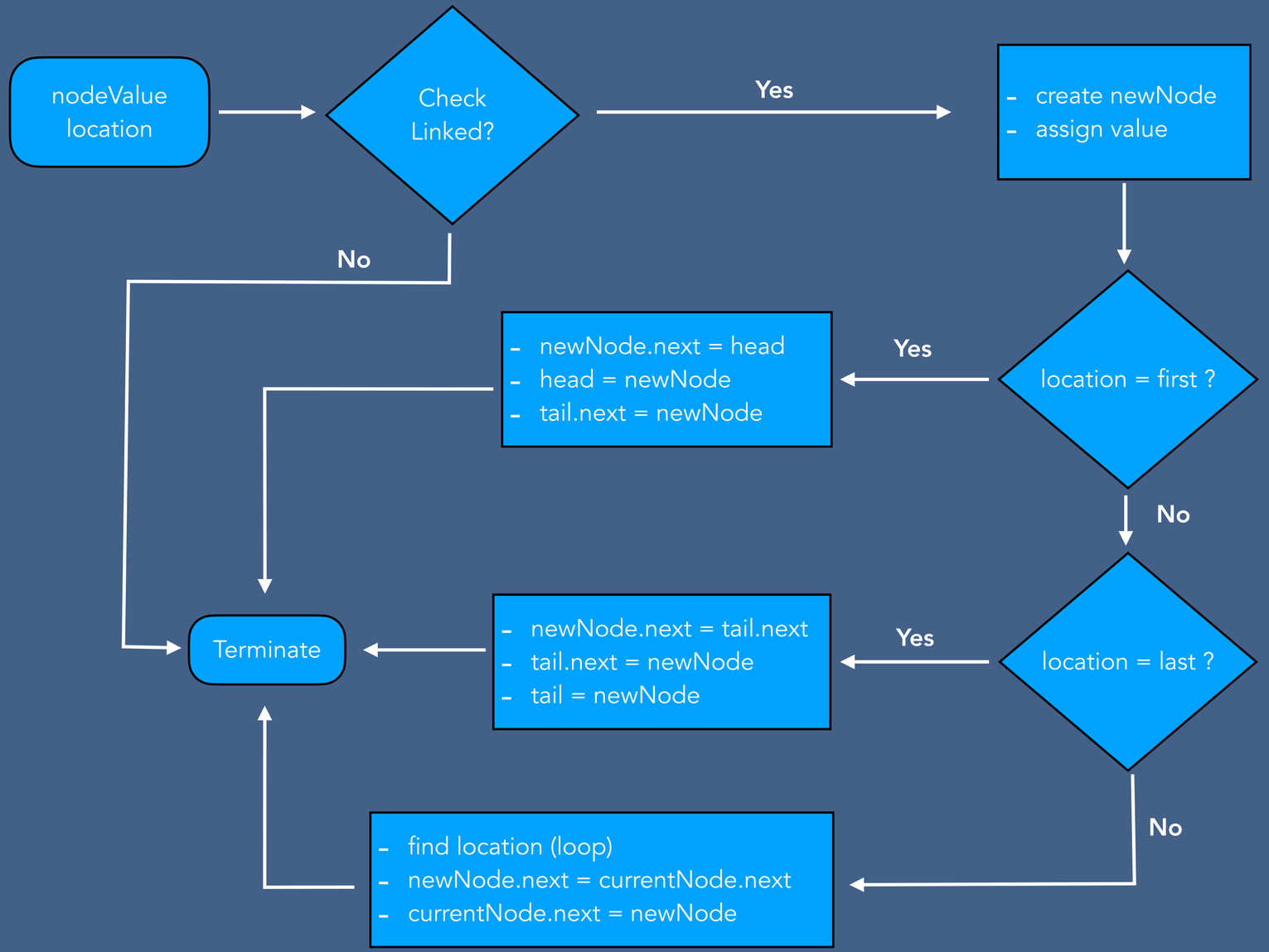


Insertion - Circular Singly Linked List

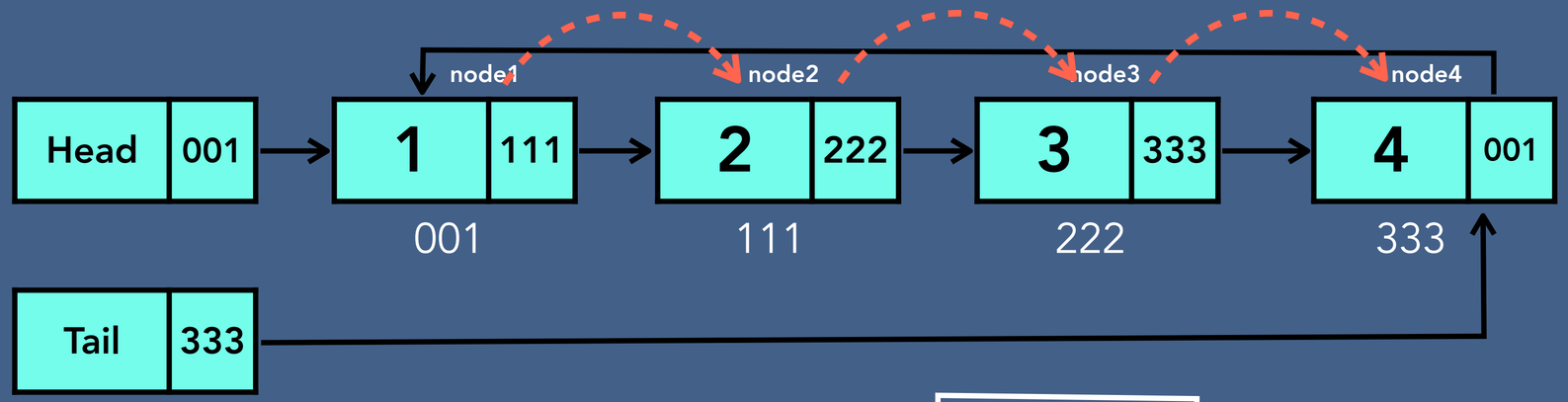
- Insert at the end of linked list

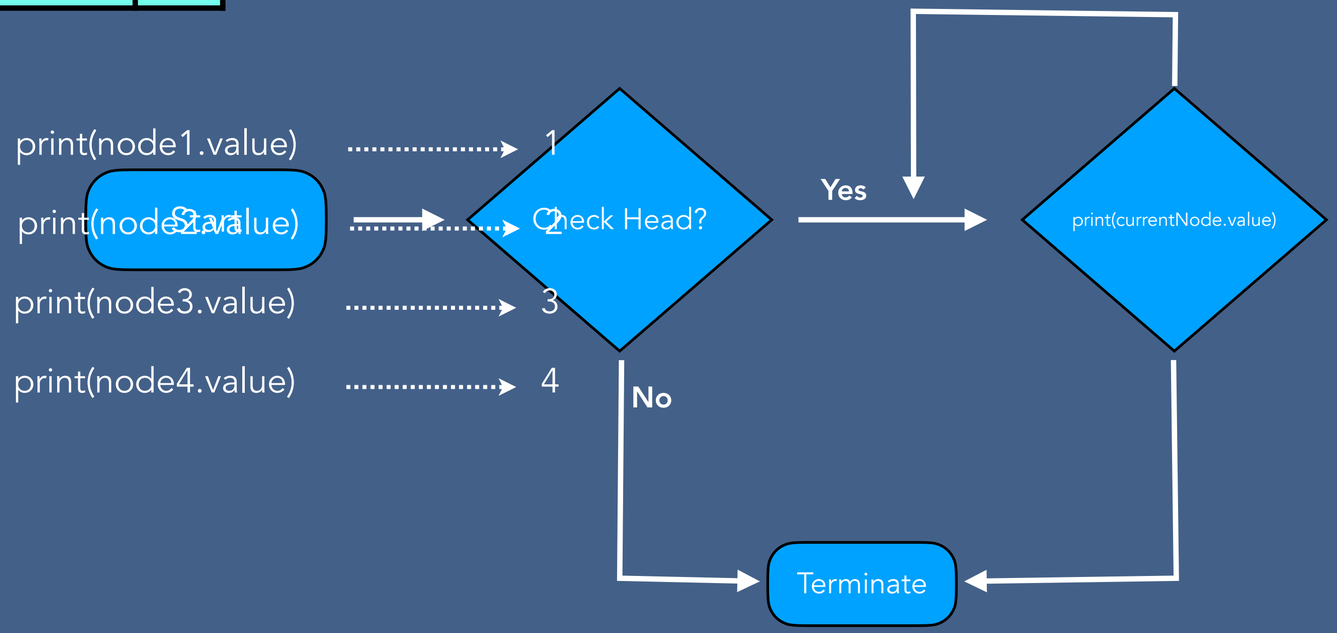


Insertion Algorithm - Circular Singly Linked List

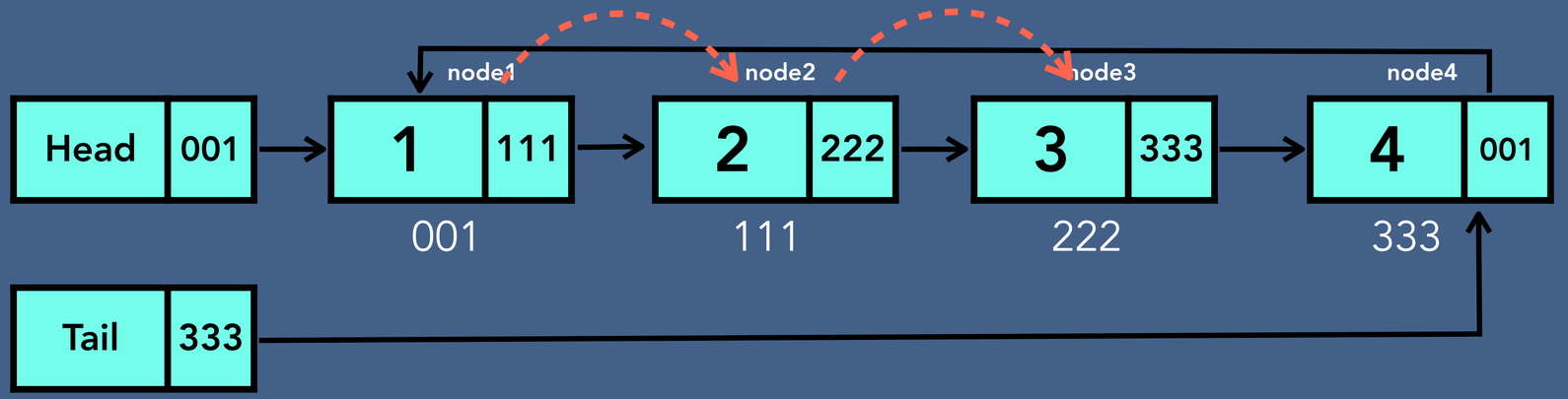


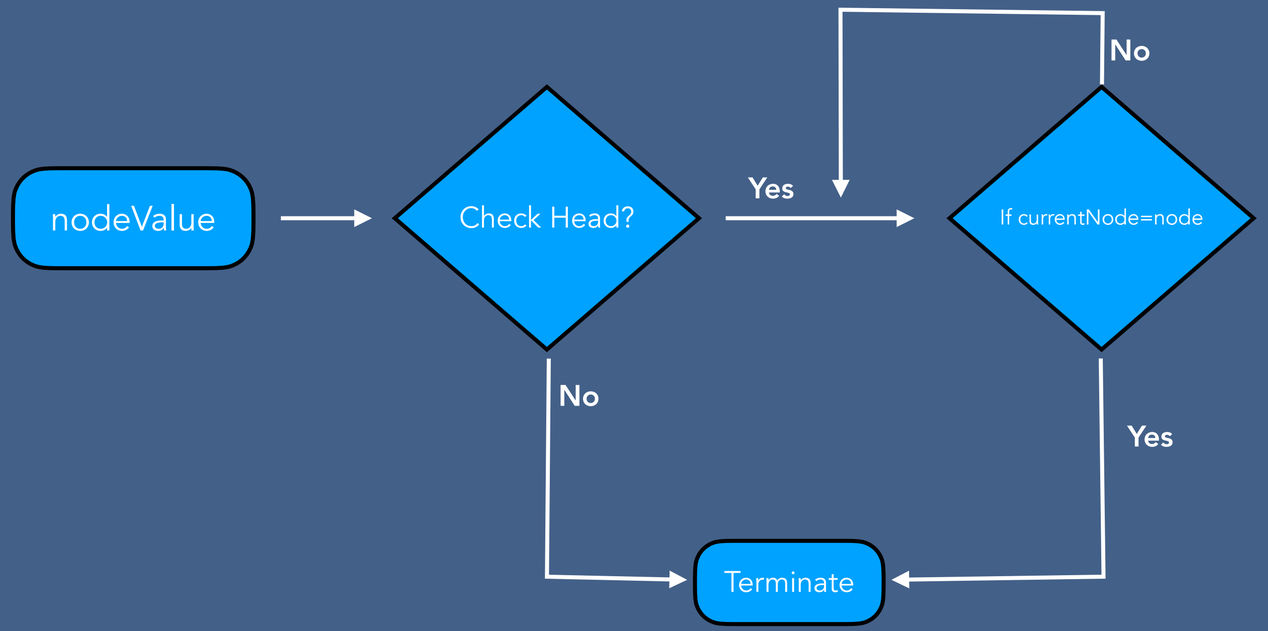
Traversal - Circular Singly Linked List





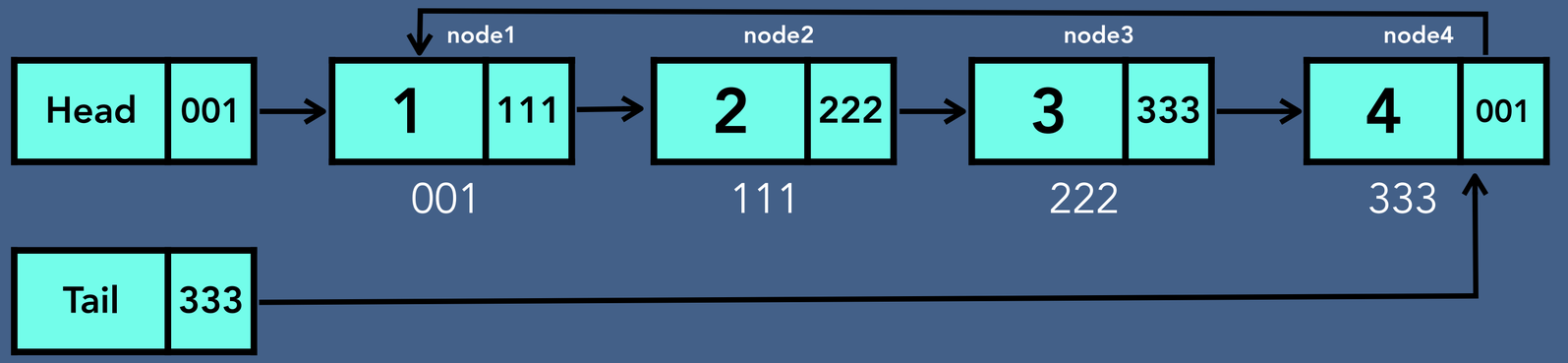
Searching - Circular Singly Linked List





Deletion - Circular Singly Linked List

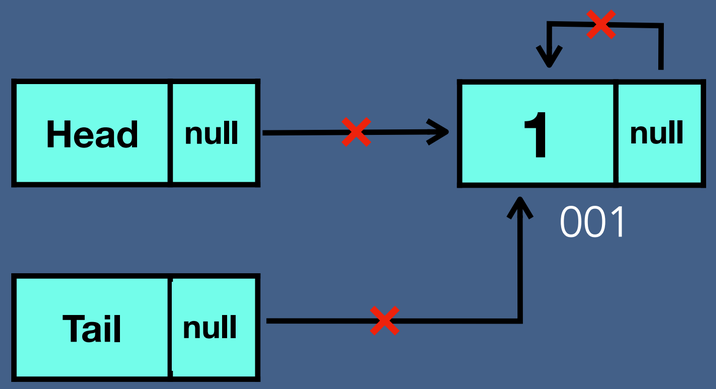
- Deleting the first node
- Deleting any given node
- Deleting the last node



Deletion - Circular Singly Linked List

Deleting the first node

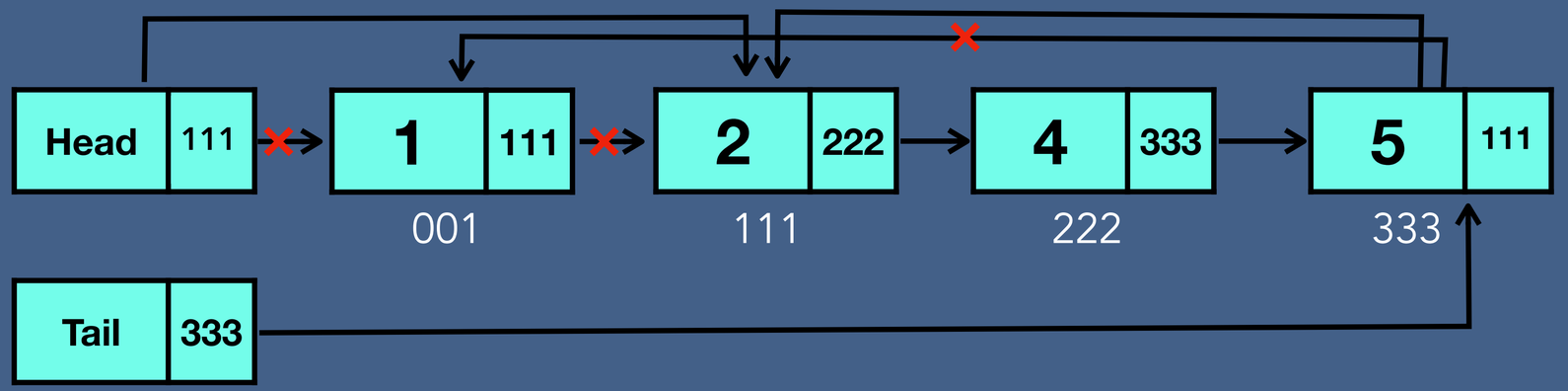
Case 1 - one node

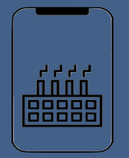


Deletion - Circular Singly Linked List

Deleting the first node

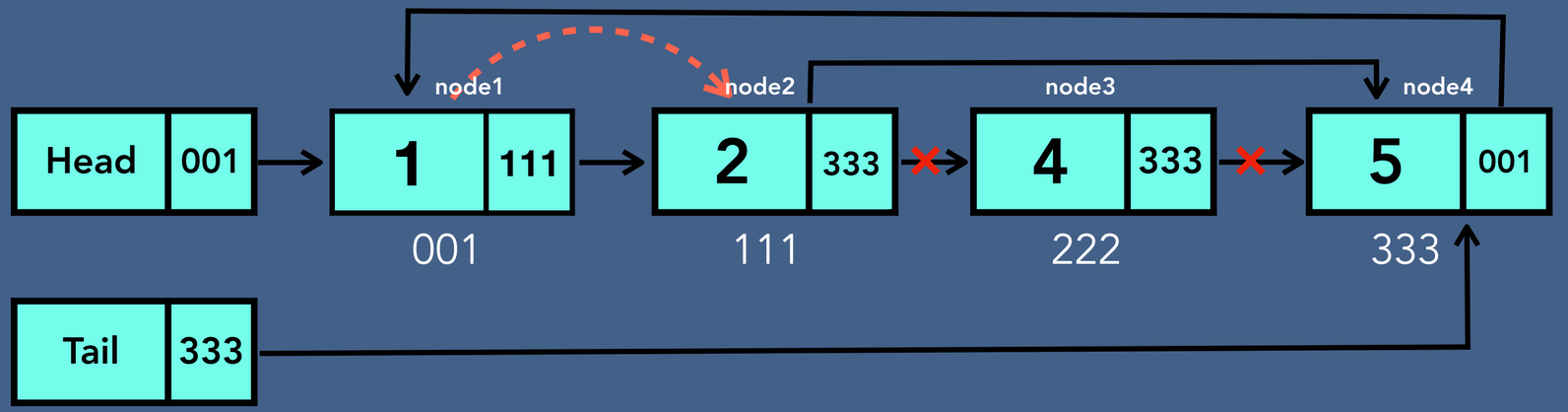
Case 2 - more than one node





Deletion - Circular Singly Linked List

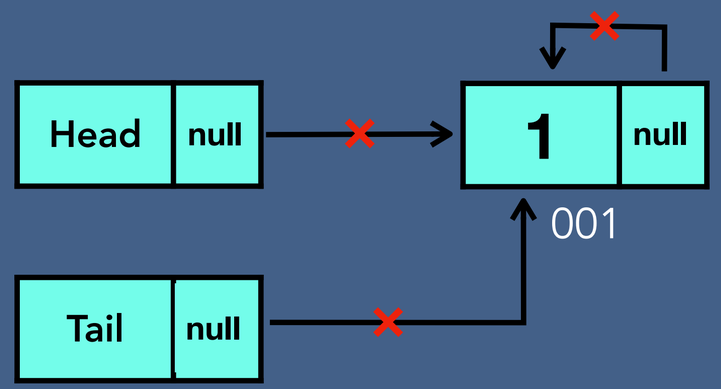
Deleting any given node



Deletion - Circular Singly Linked List

Deleting the last node

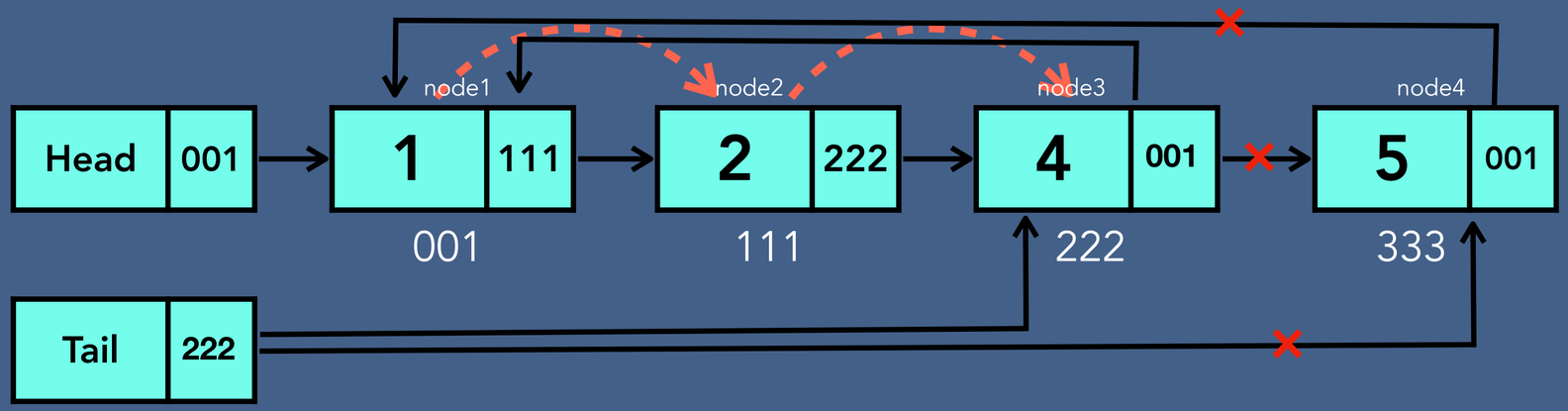
Case 1 - one node



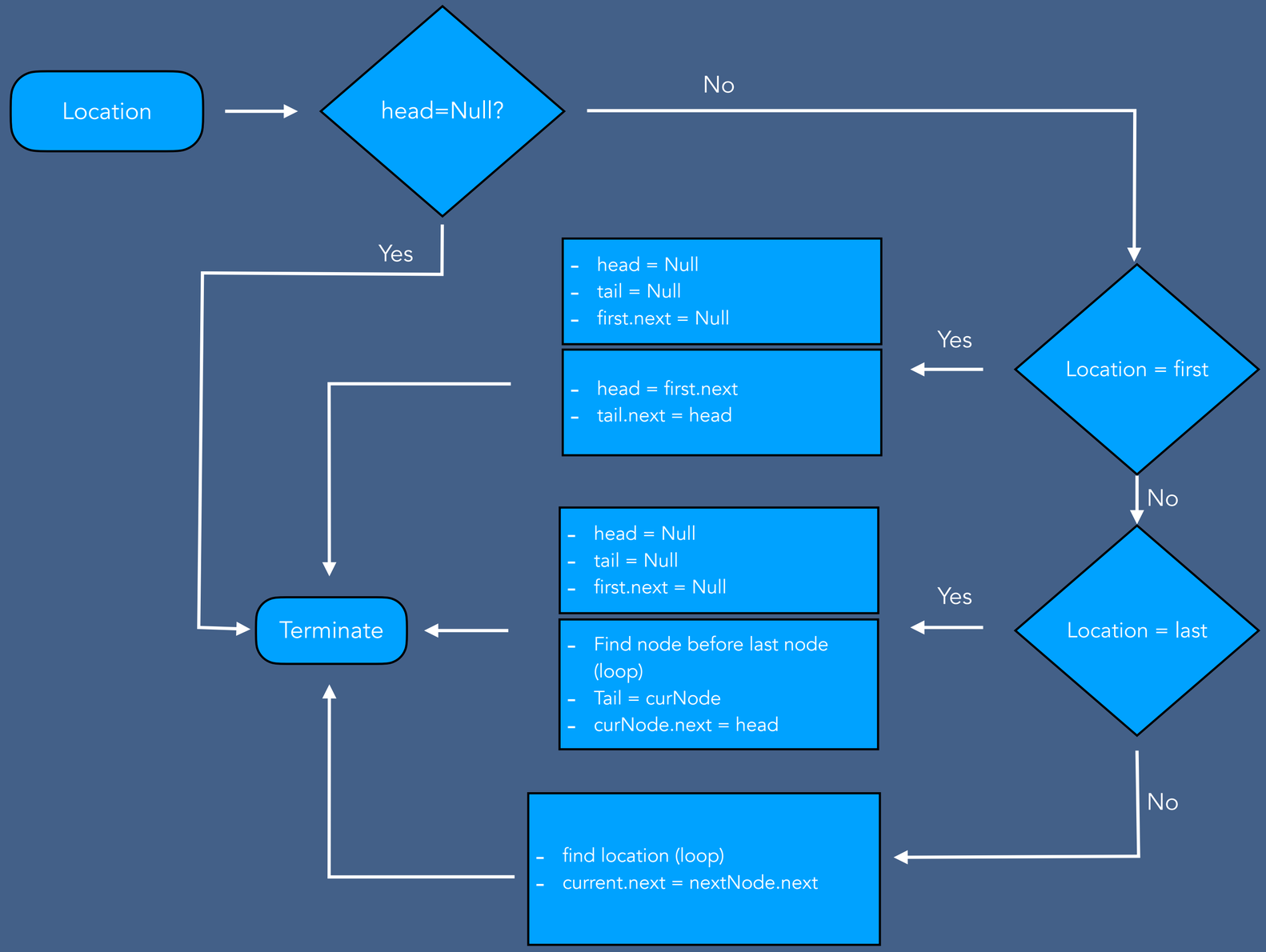
Deletion - Circular Singly Linked List

Deleting the last node

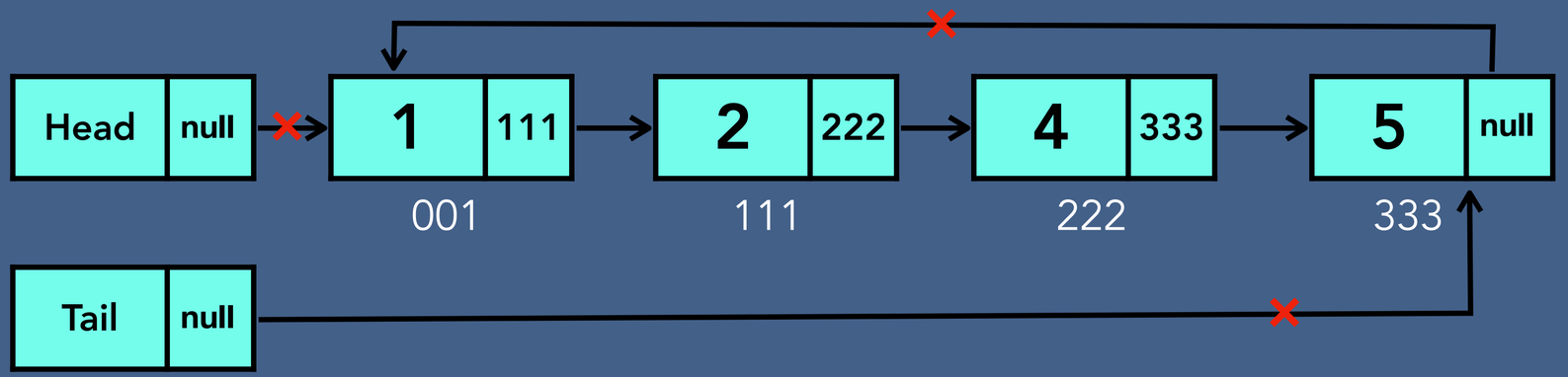
Case 2 - more than one node



Deletion Algorithm - Circular Singly Linked List

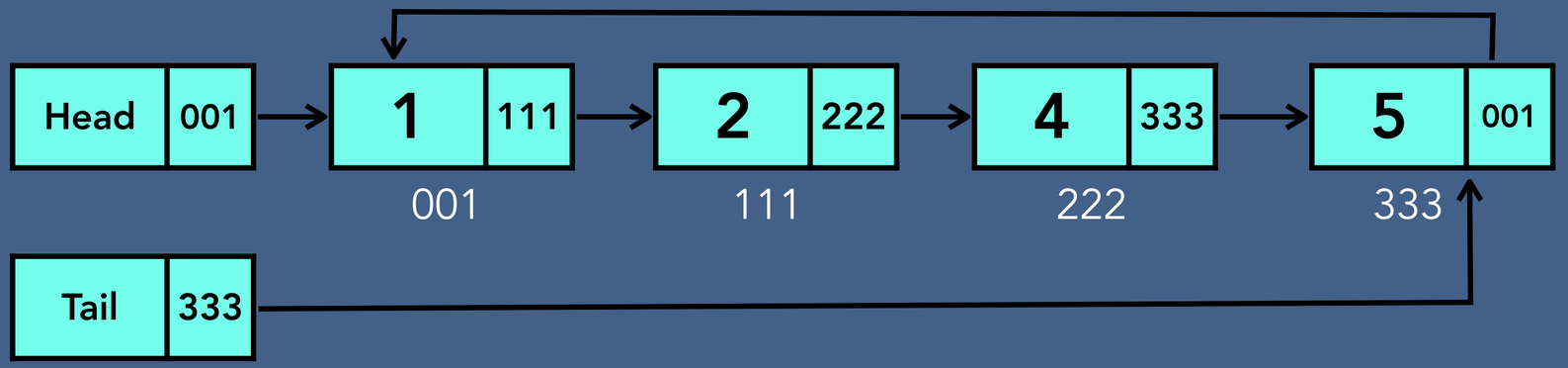


Delete Entire Circular Singly Linked List

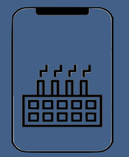


Time and Space Complexity of Circular Singly Linked List

Circular Singly Linked List	Time complexity	Space complexity
Creation	O(1)	O(1)
Insertion	O(n)	O(1)
Searching	O(n)	O(1)
Traversing	O(n)	O(1)
Deletion of a node	O(n)	O(1)
Deletion of CSLL	O(1)	O(1)

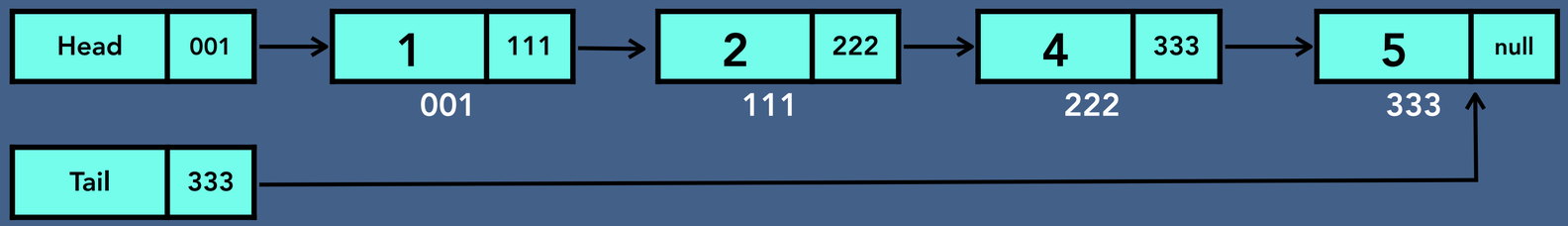


Doubly Linked List

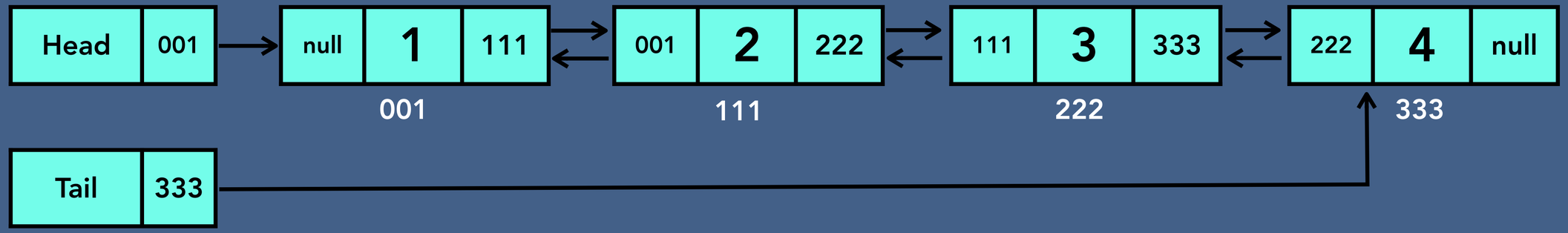


Doubly Linked List

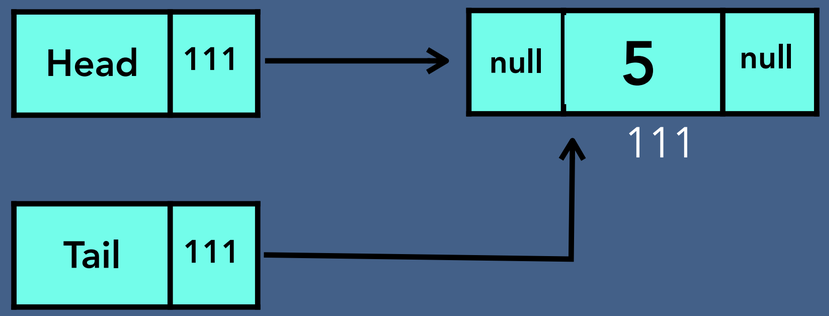
Singly Linked List

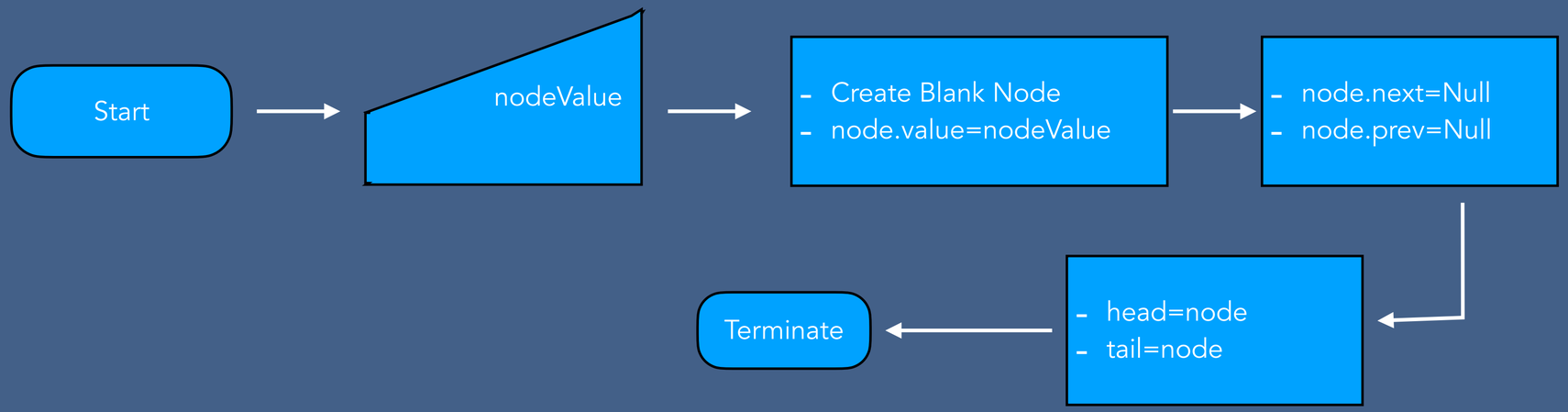


Doubly Linked List



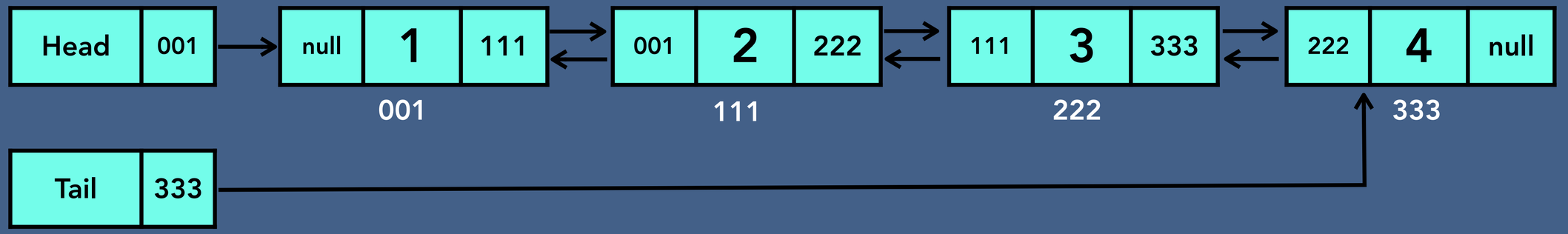
Create - Doubly Linked List





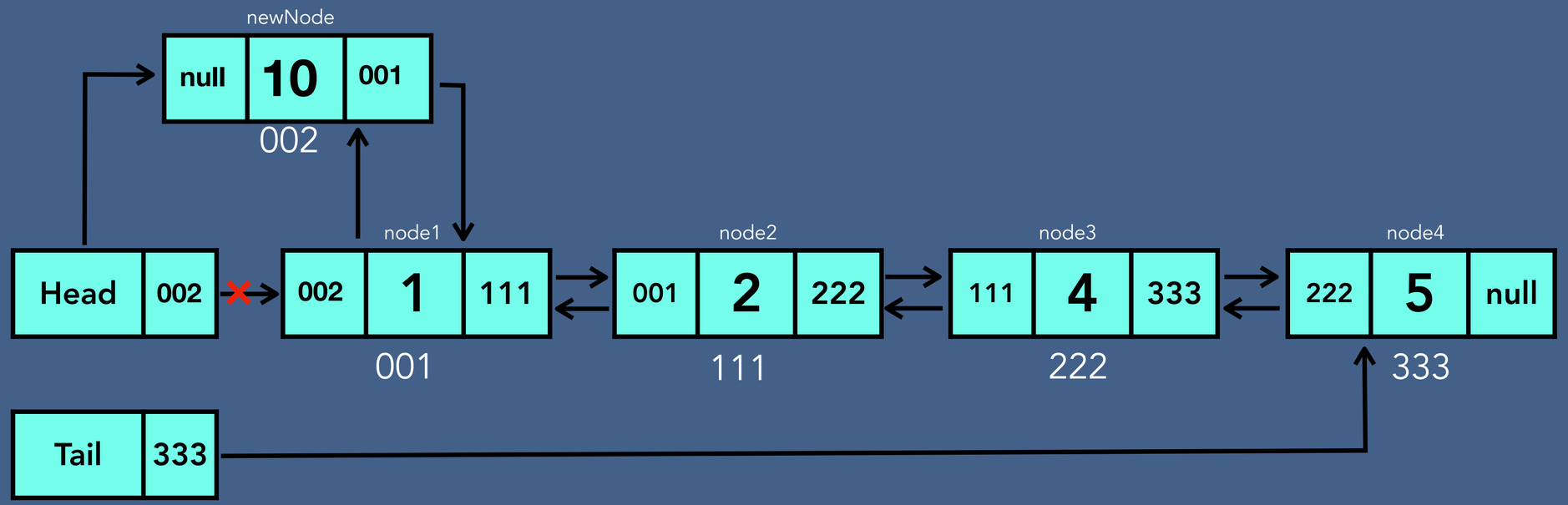
Insertion - Doubly Linked List

- Insert at the beginning of linked list
- Insert at the specified location of linked list
- Insert at the end of linked list



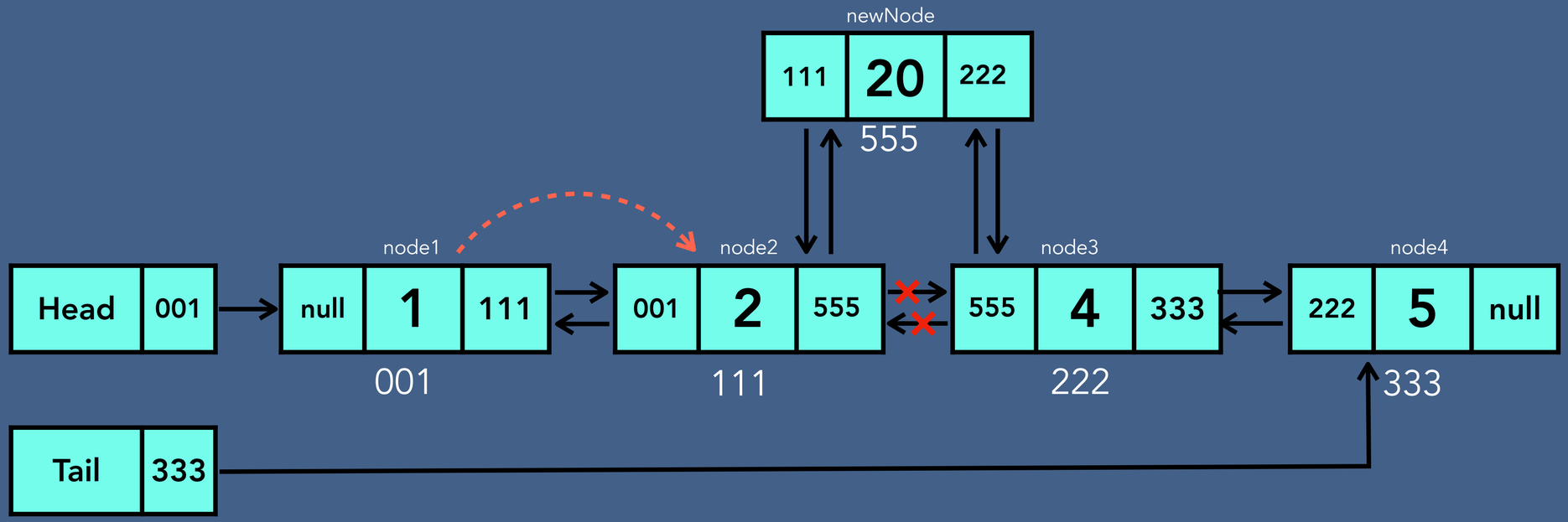
Insertion - Doubly Linked List

- Insert at the beginning of linked list



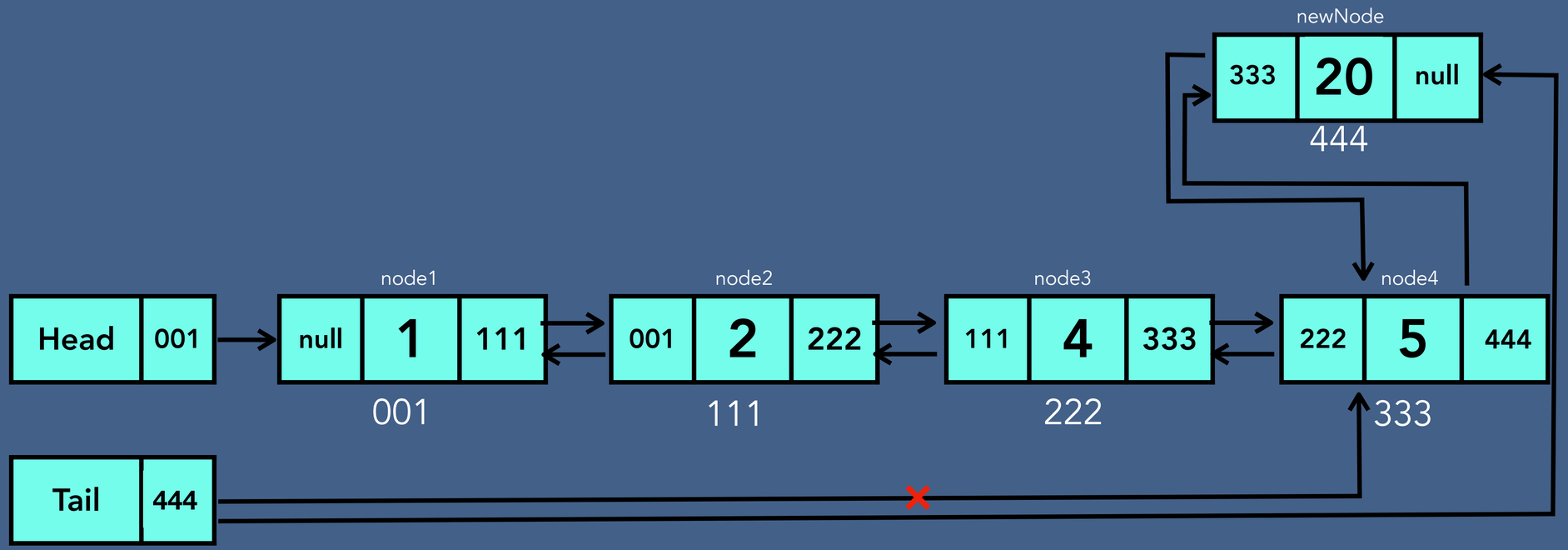
Insertion - Doubly Linked List

- Insert at the specified location of linked list

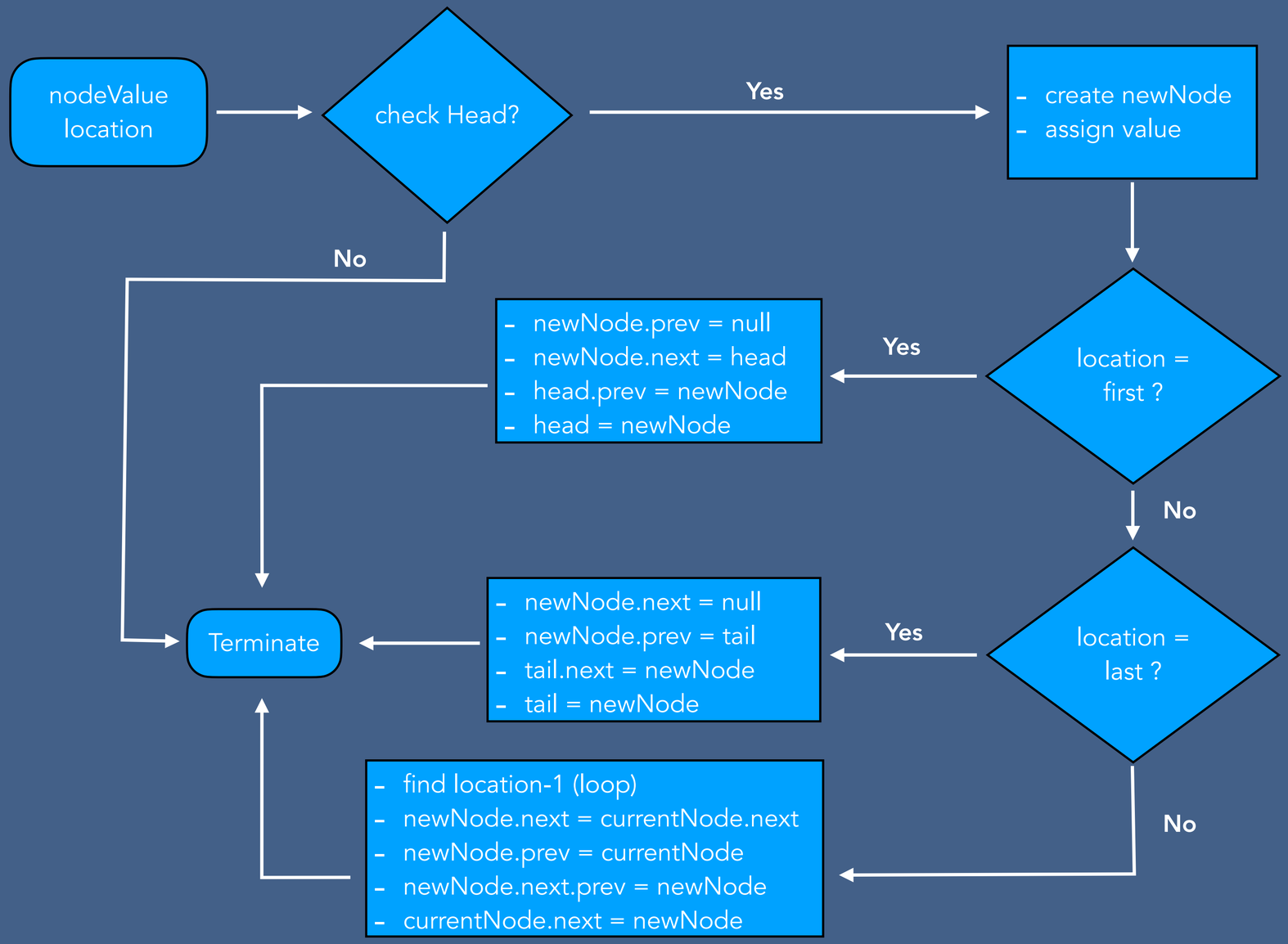


Insertion - Doubly Linked List

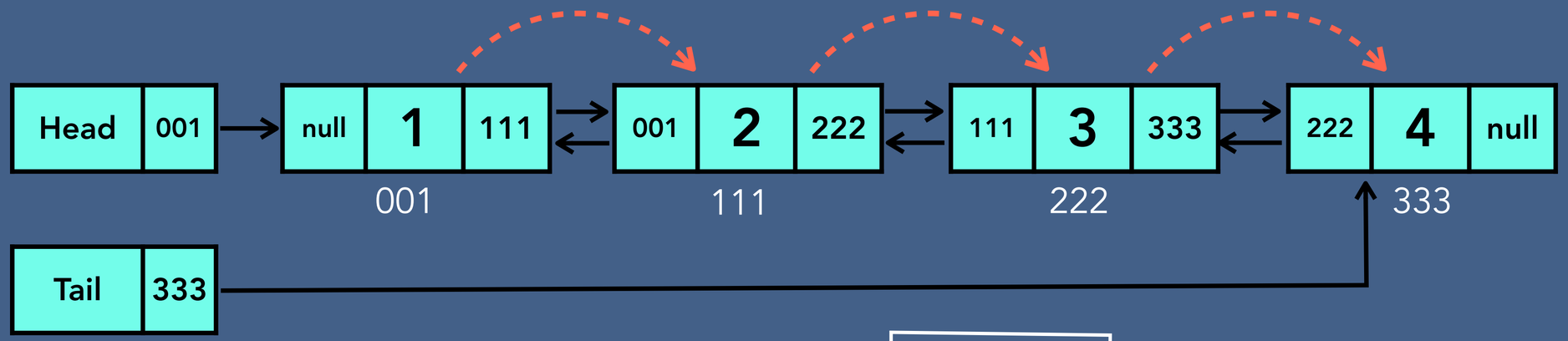
- Insert at the end of linked list

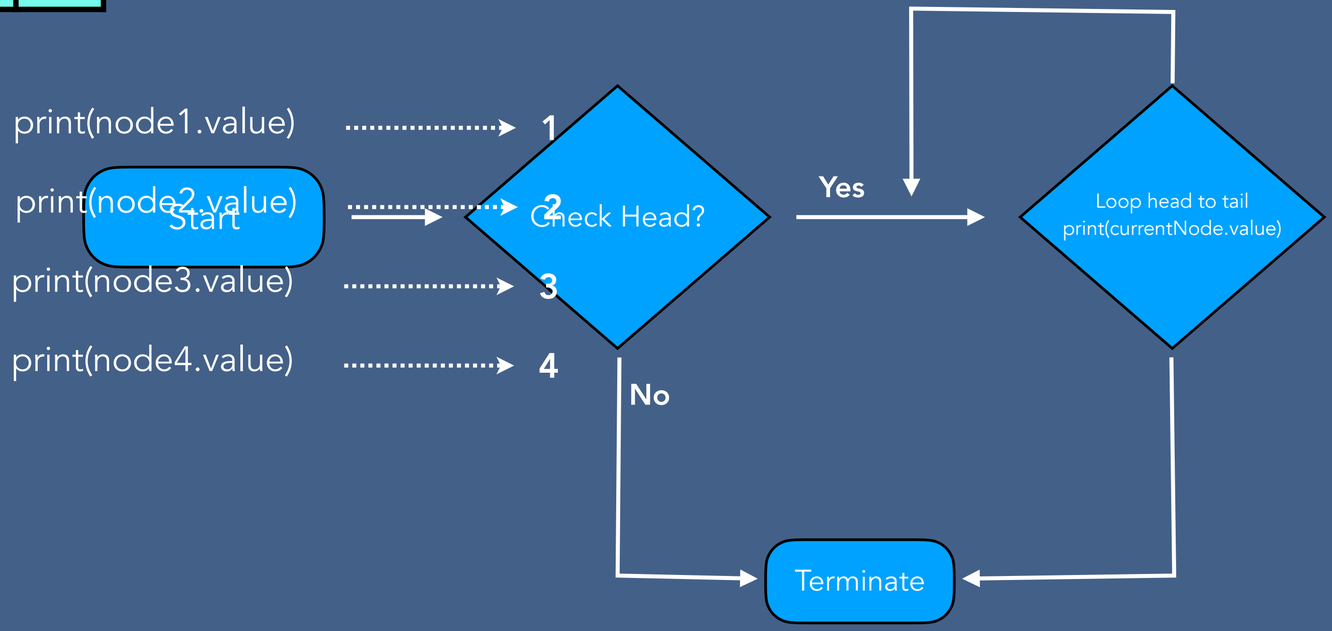


Insertion Algorithm - Doubly Linked List

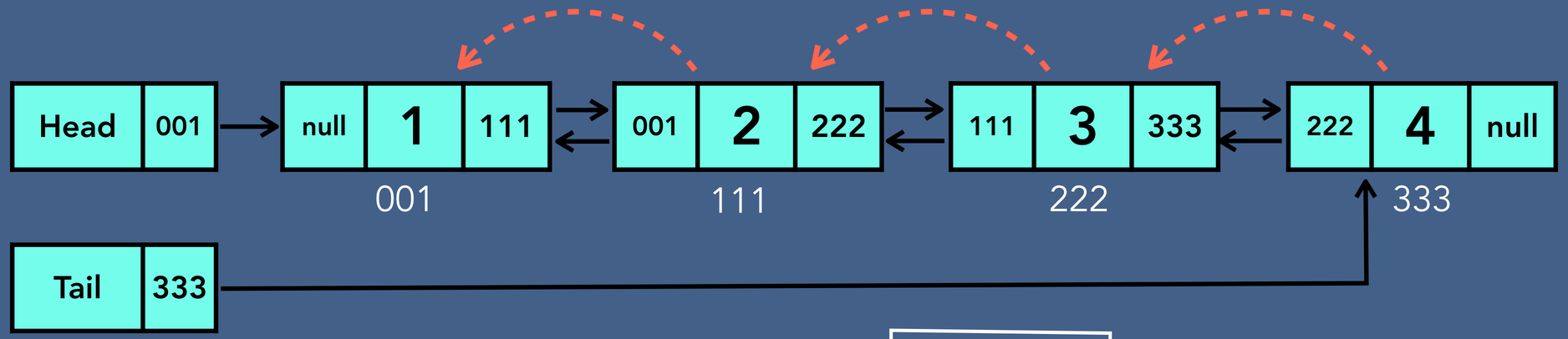


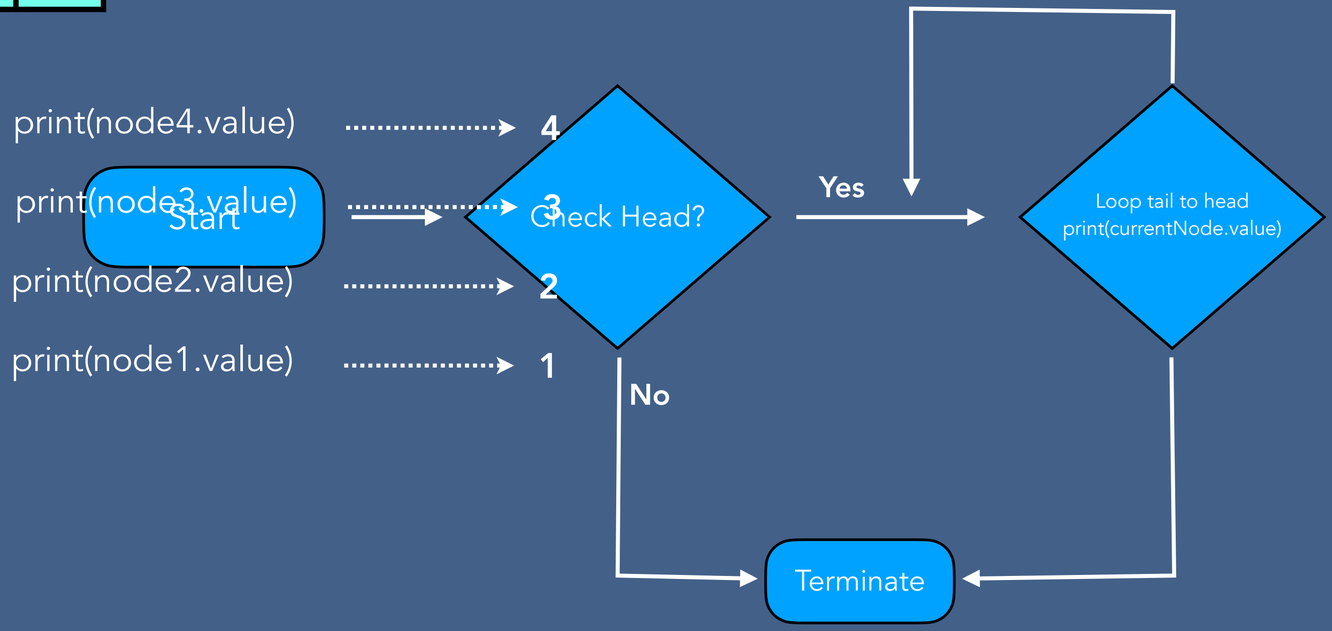
Traversal - Doubly Linked List



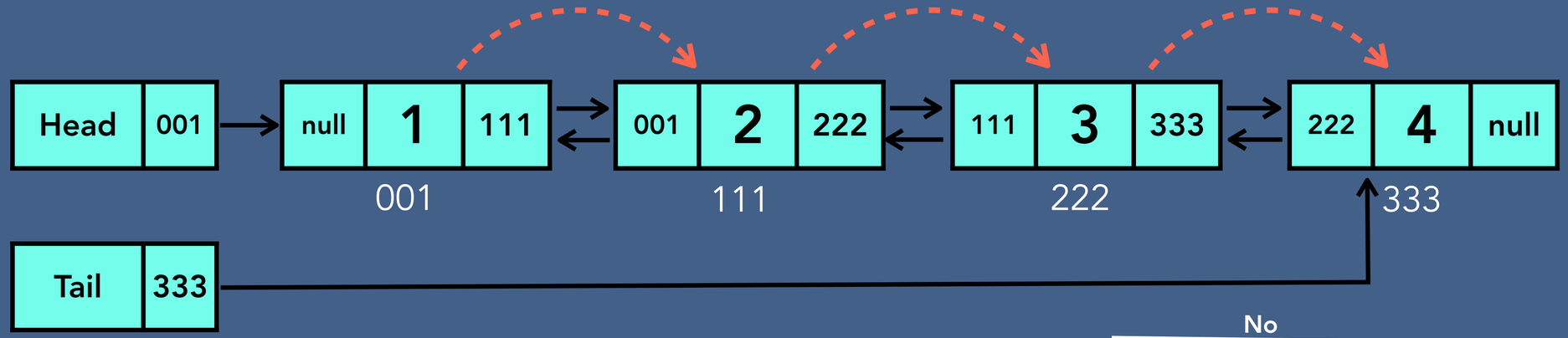


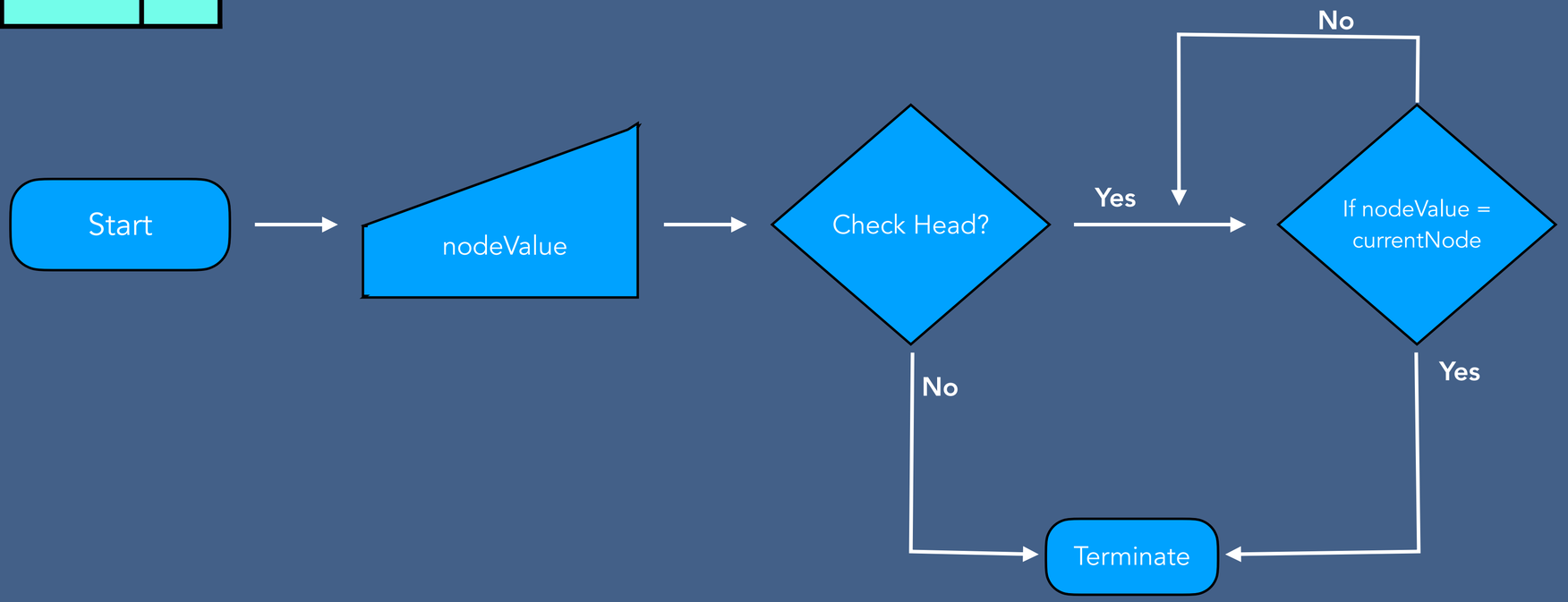
Reverse Traversal - Doubly Linked List





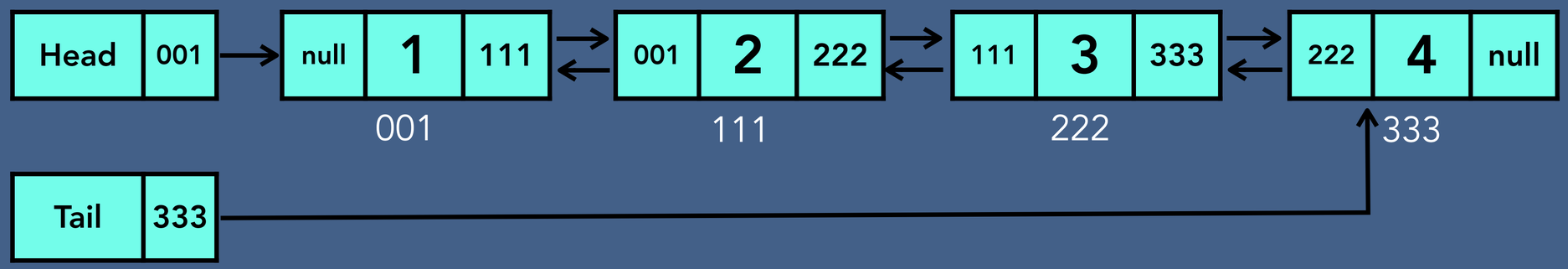
Searching- Doubly Linked List





Deletion - Doubly Linked List

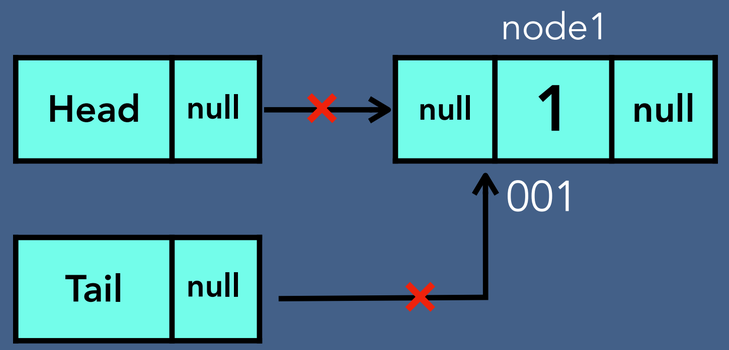
- Deleting the first node
- Deleting any given node
- Deleting the last node



Deletion - Doubly Linked List

Deleting the first node

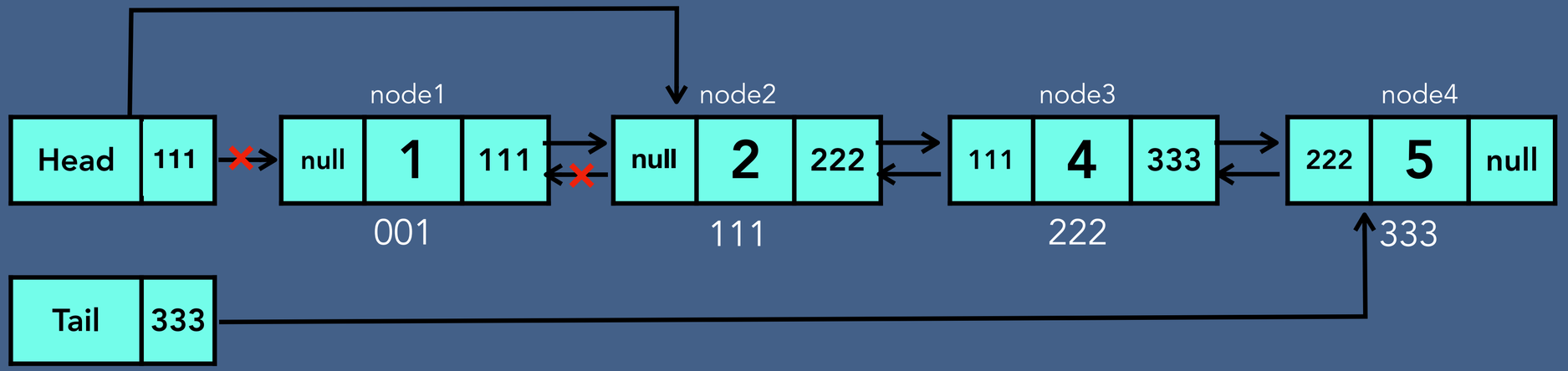
Case 1 - one node



Deletion - Doubly Linked List

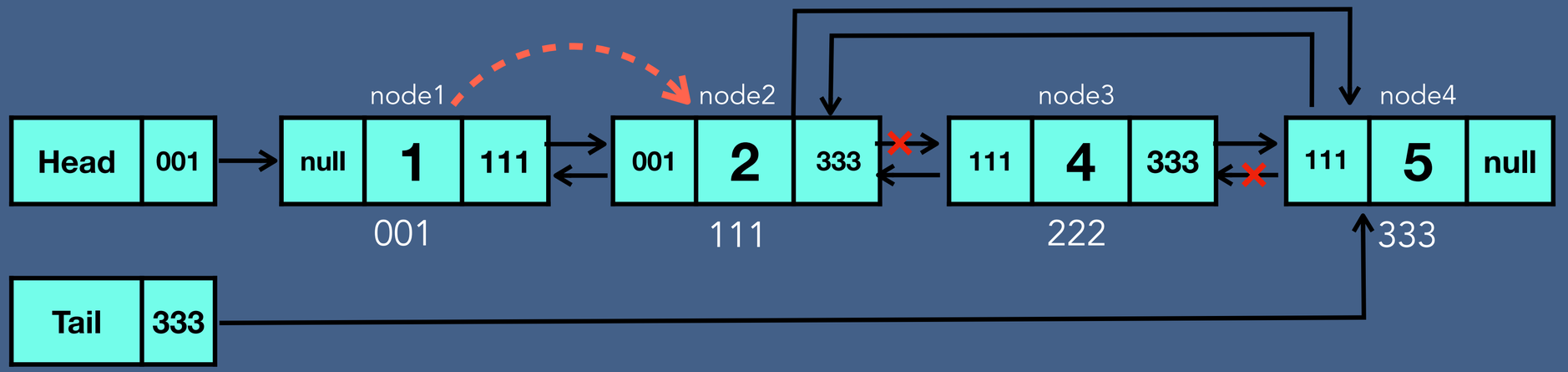
Deleting the first node

Case 2 - more than one node



Deletion - Doubly Linked List

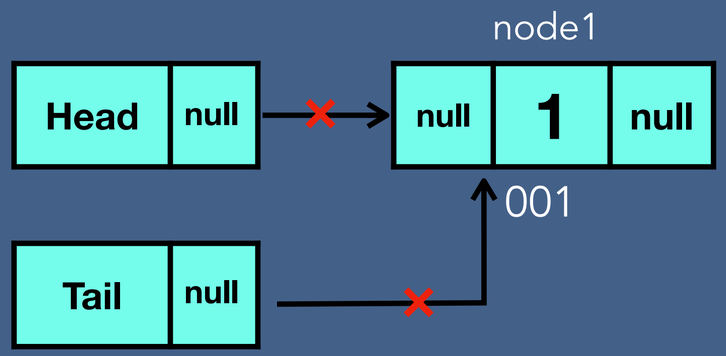
Deleting any given node



Deletion - Doubly Linked List

Deleting the last node

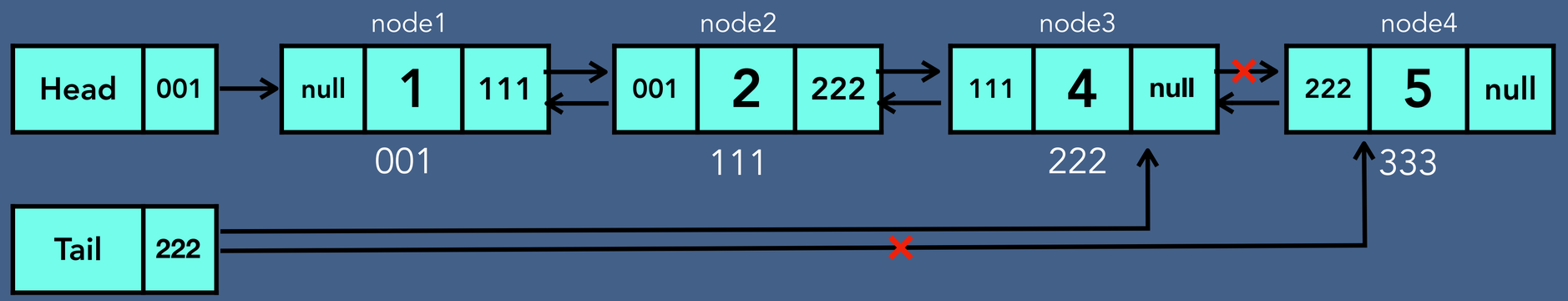
Case 1 - one node



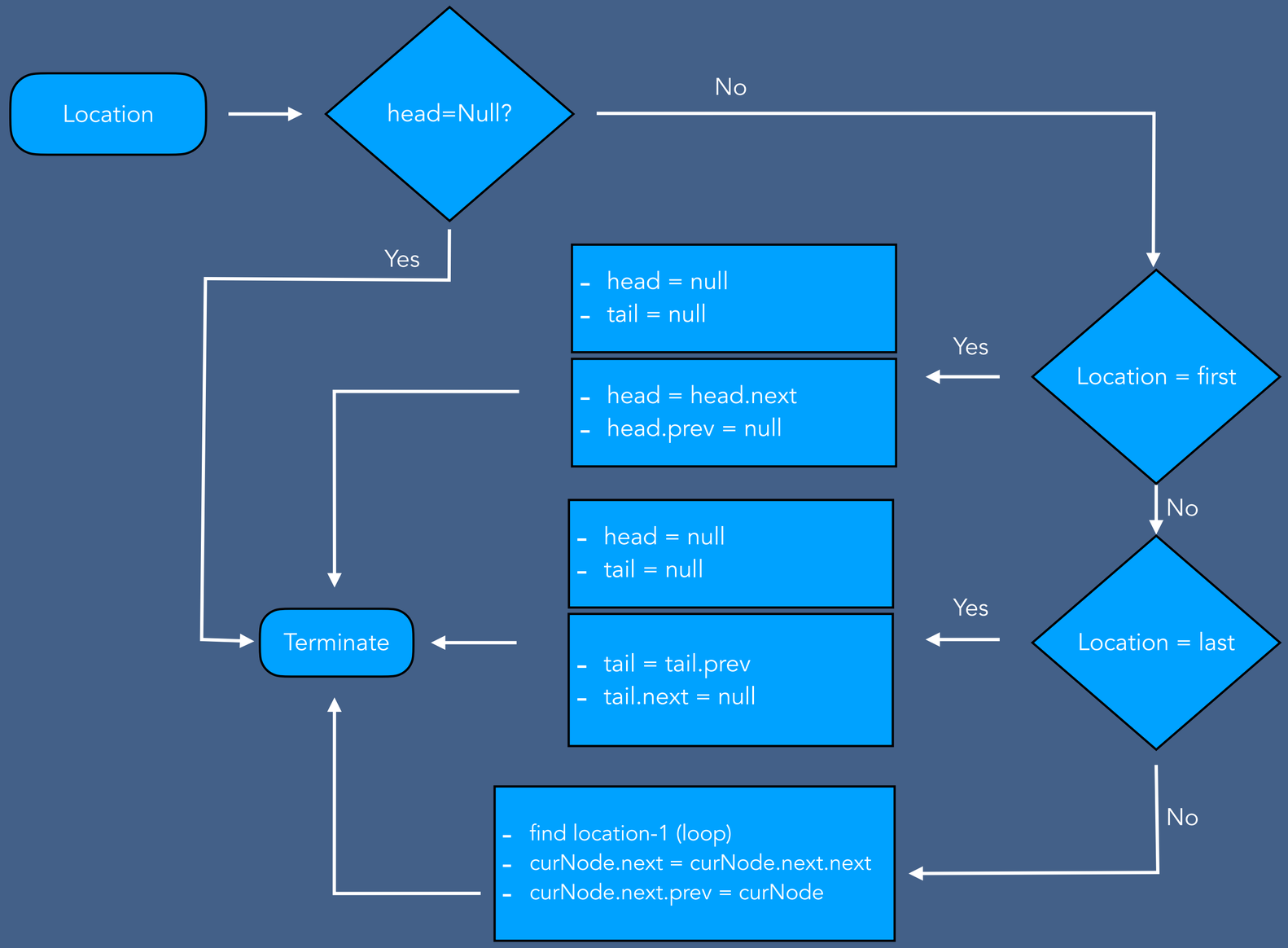
Deletion - Doubly Linked List

Deleting the last node

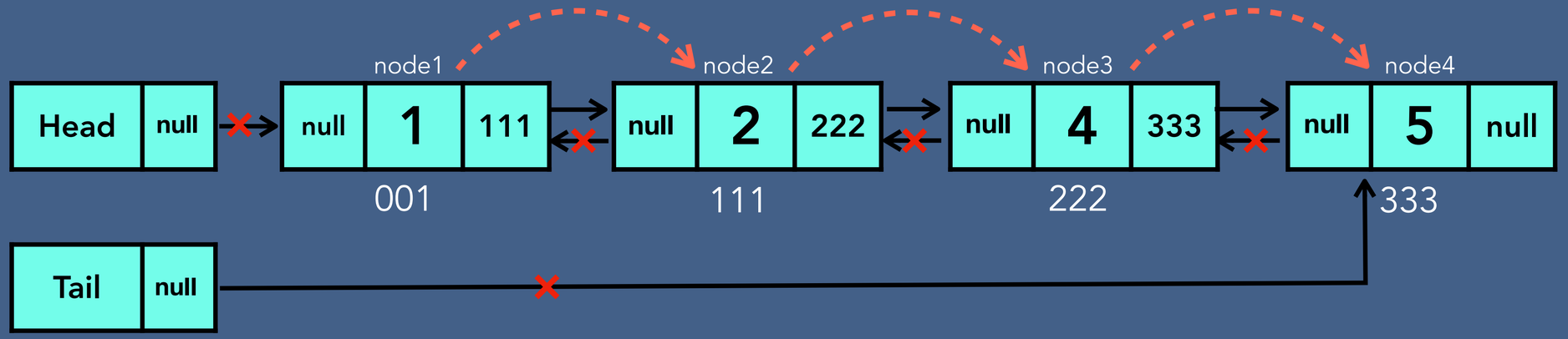
Case 2 - more than one node

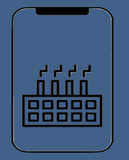


Deletion Algorithm - Doubly Linked List



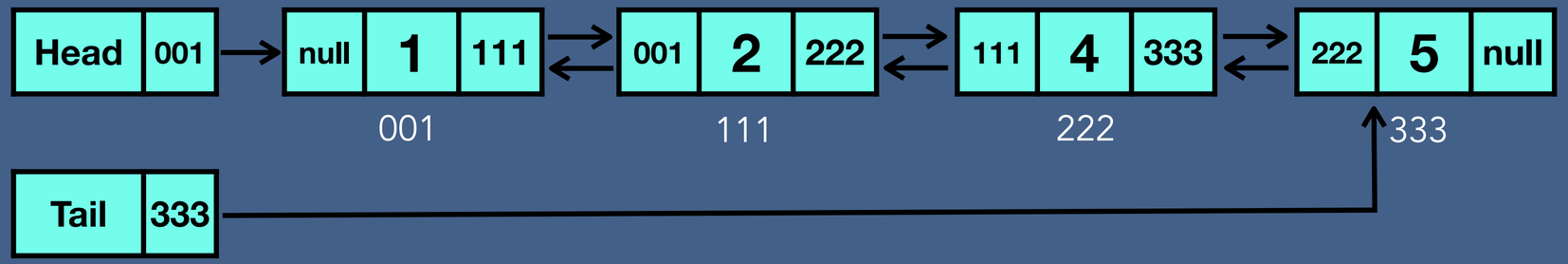
Delete Entire Doubly Linked List



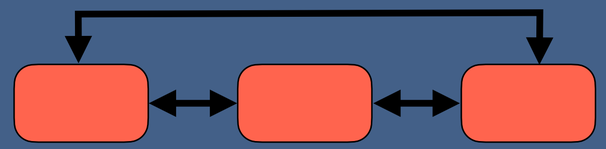


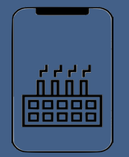
Time and Space Complexity of Doubly Linked List

Doubly Linked List	Time complexity	Space complexity
Creation	O(1)	O(1)
Insertion	O(n)	O(1)
Searching	O(n)	O(1)
Traversing (forward ,backward)	O(n)	O(1)
Deletion of a node	O(n)	O(1)
Deletion of DLL	O(n)	O(1)



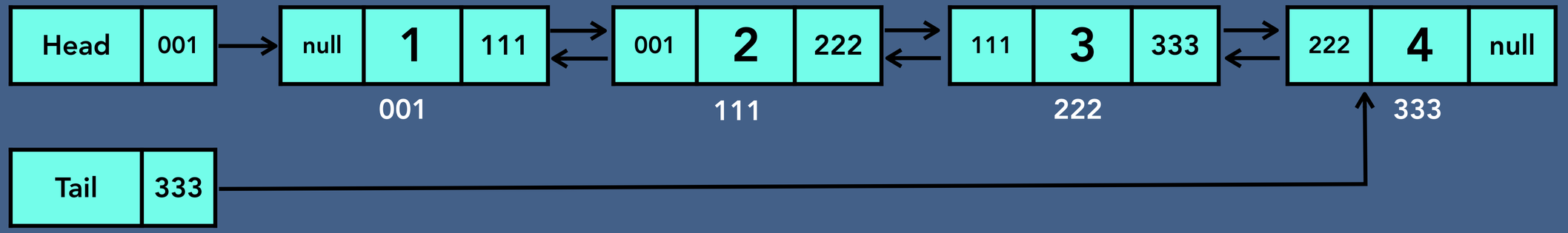
Circular Doubly Linked List



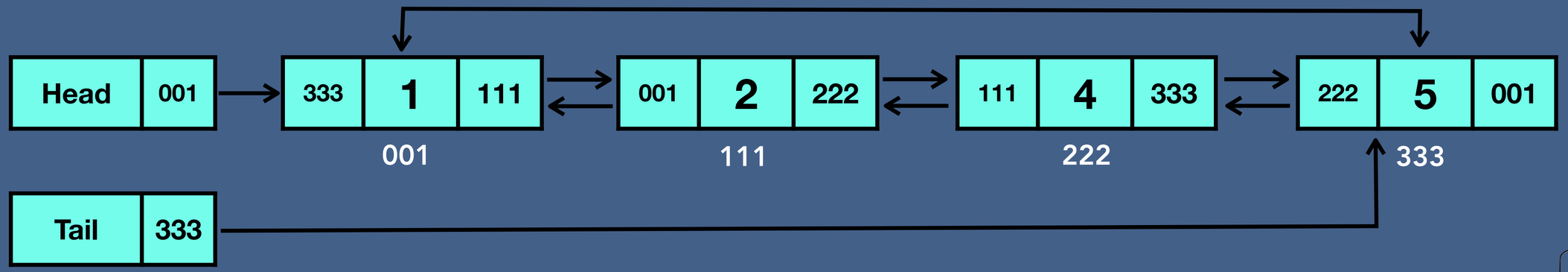


Circular Doubly Linked List

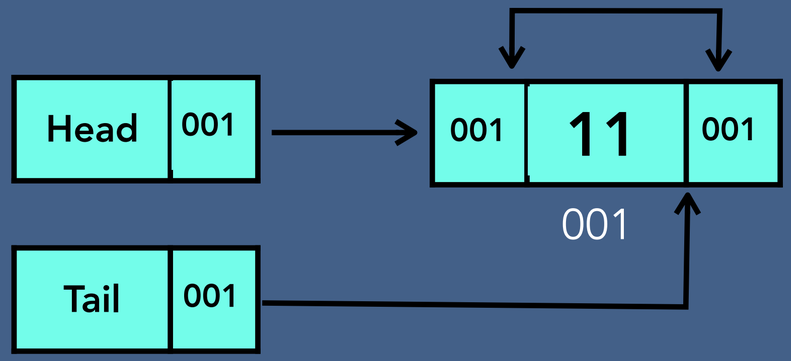
Doubly Linked List

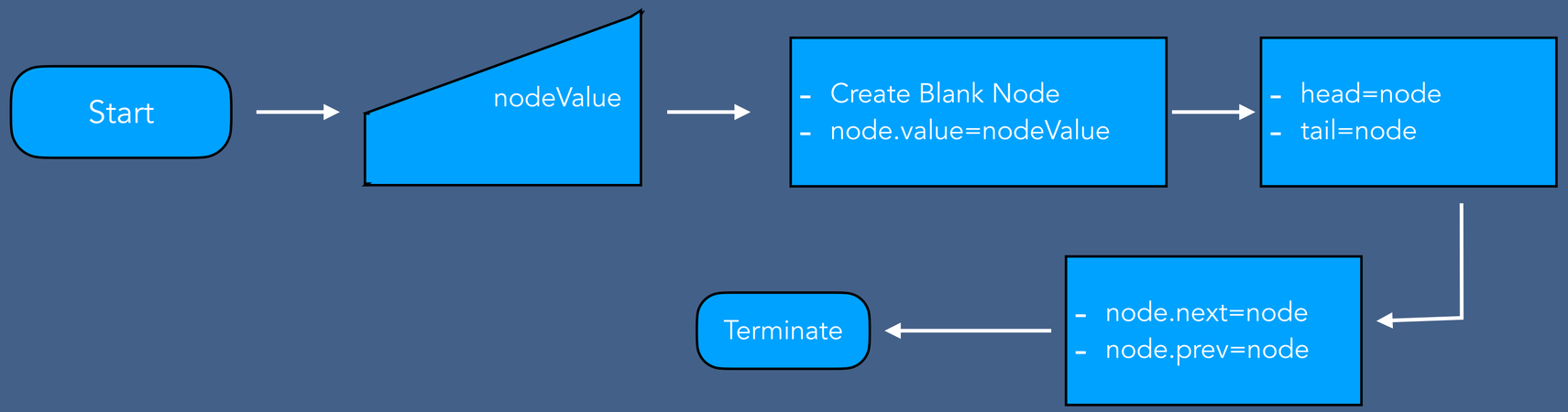


Circular Doubly linked list



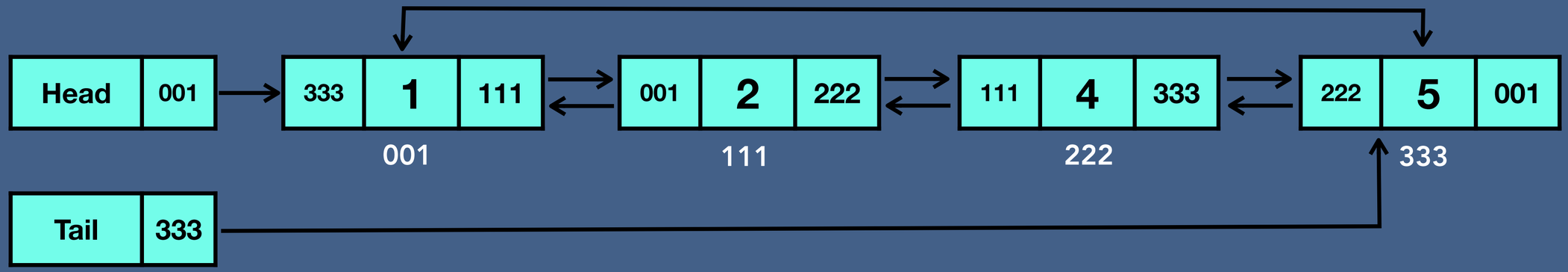
Create - Circular Doubly Linked List





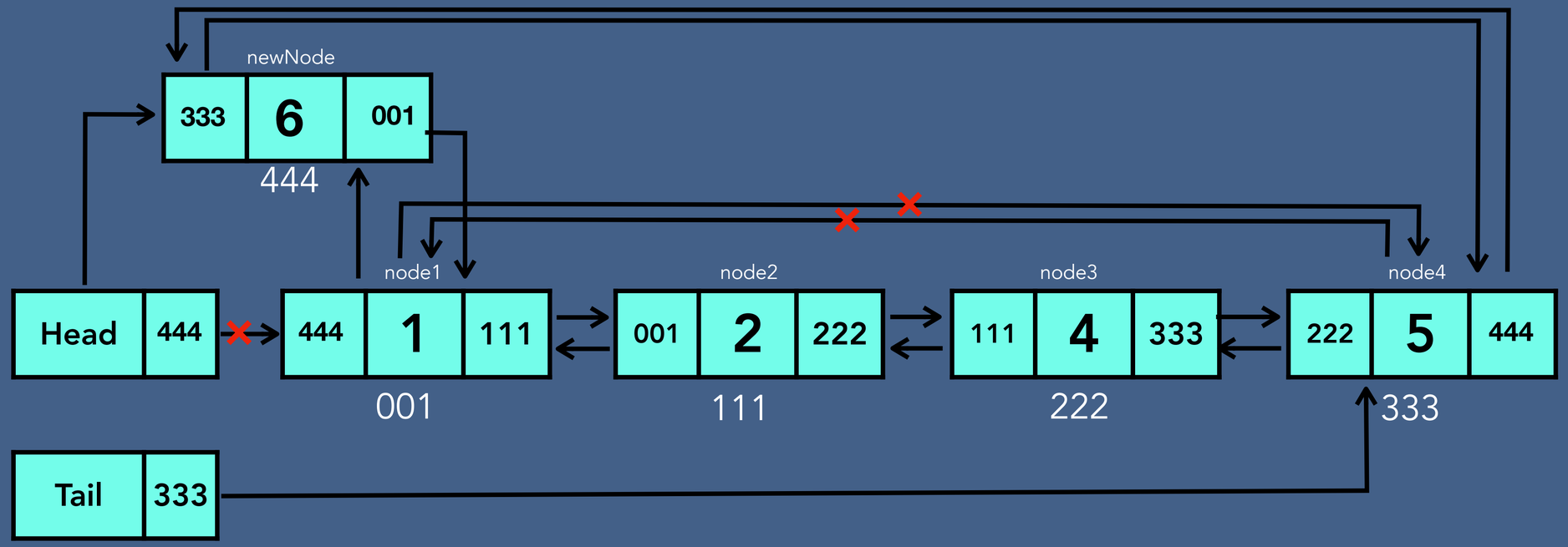
Insertion - Circular Doubly Linked List

- Insert at the beginning of linked list
- Insert at the specified location of linked list
- Insert at the end of linked list



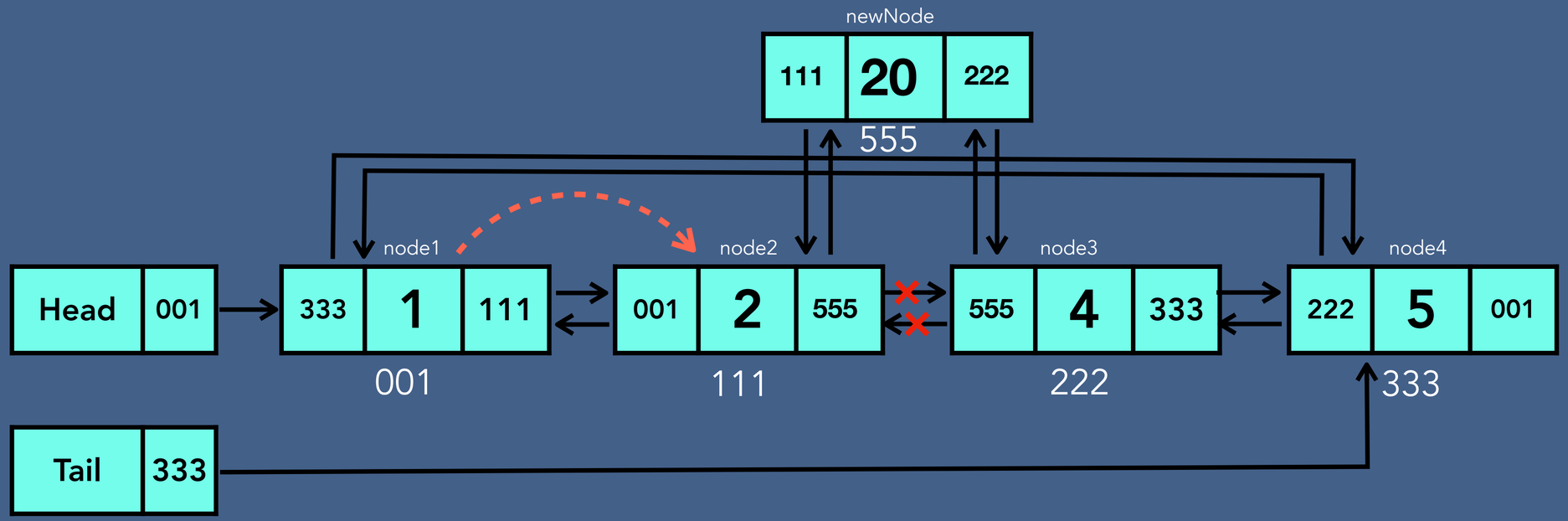
Insertion - Circular Doubly Linked List

- Insert at the beginning of linked list



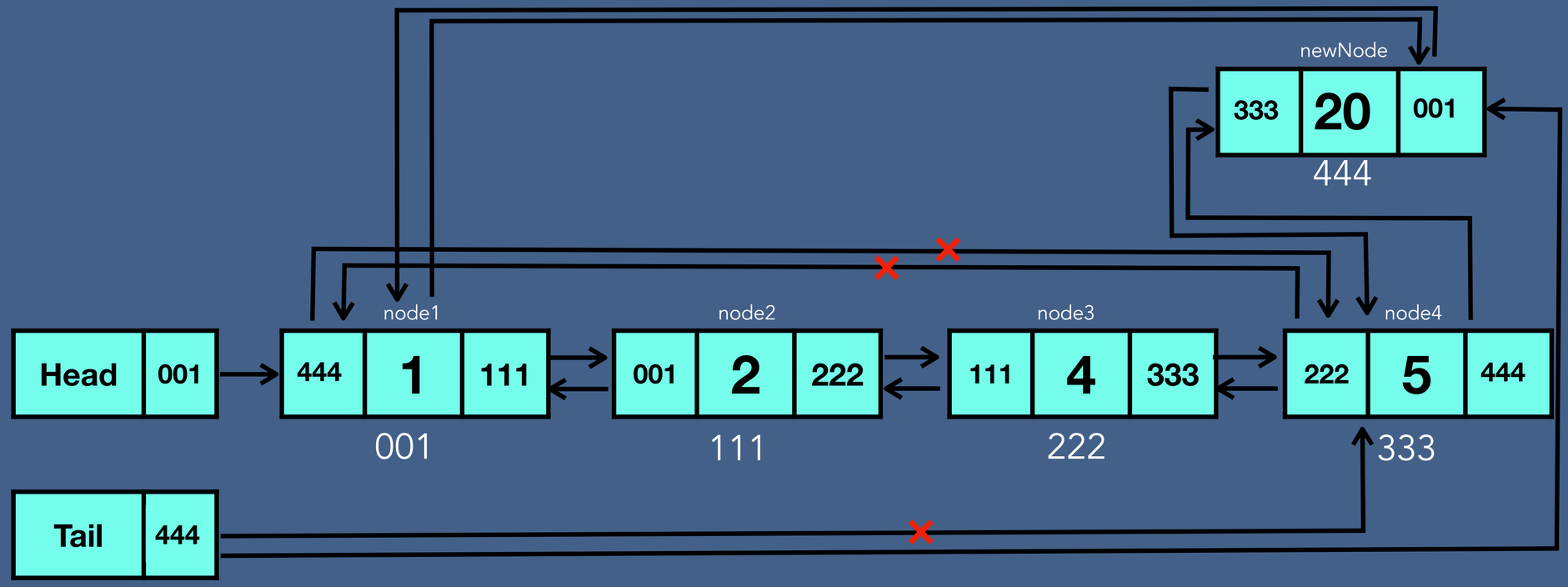
Insertion - Circular Doubly Linked List

- Insert at the specified location of linked list

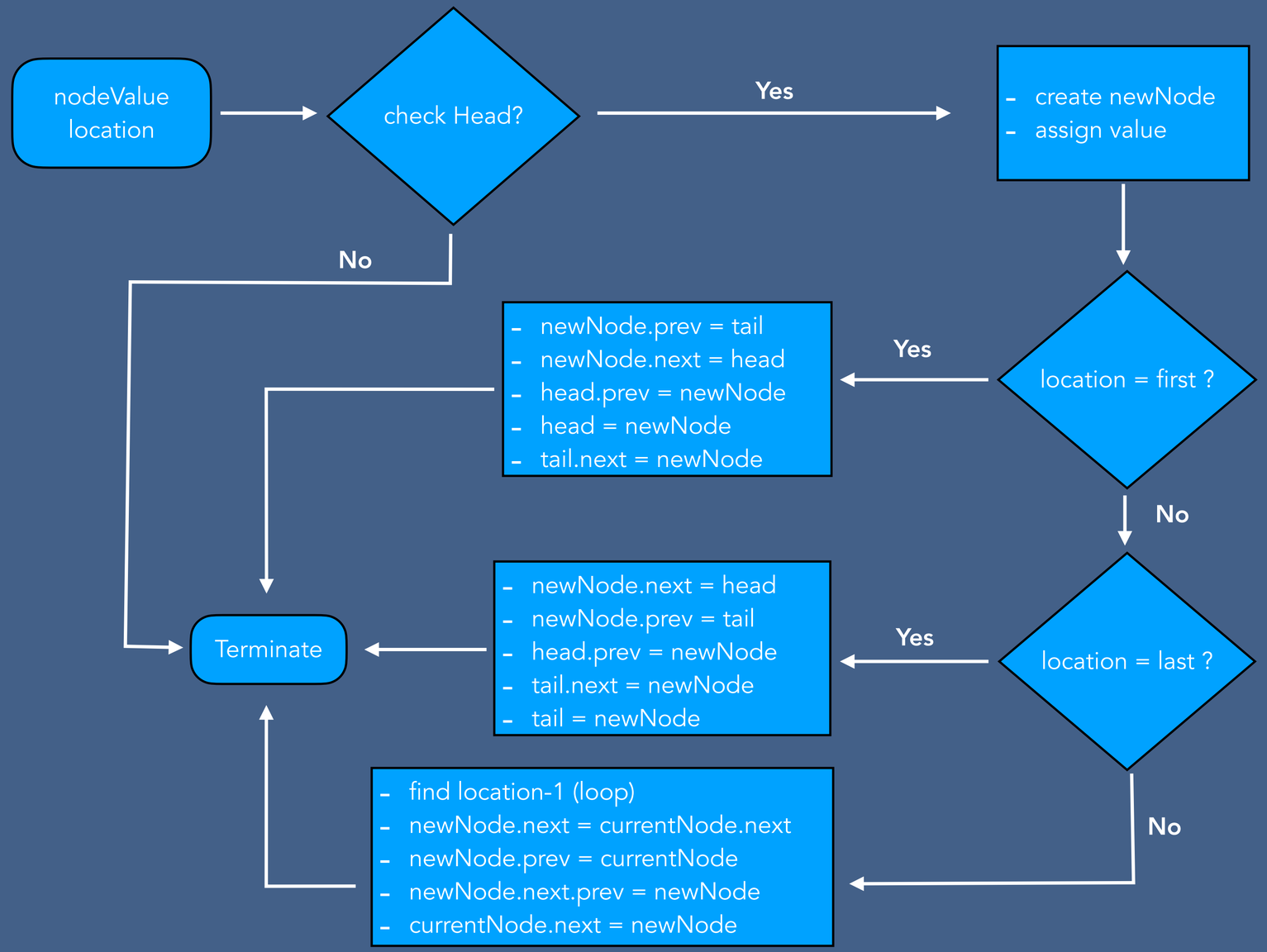


Insertion - Circular Doubly Linked List

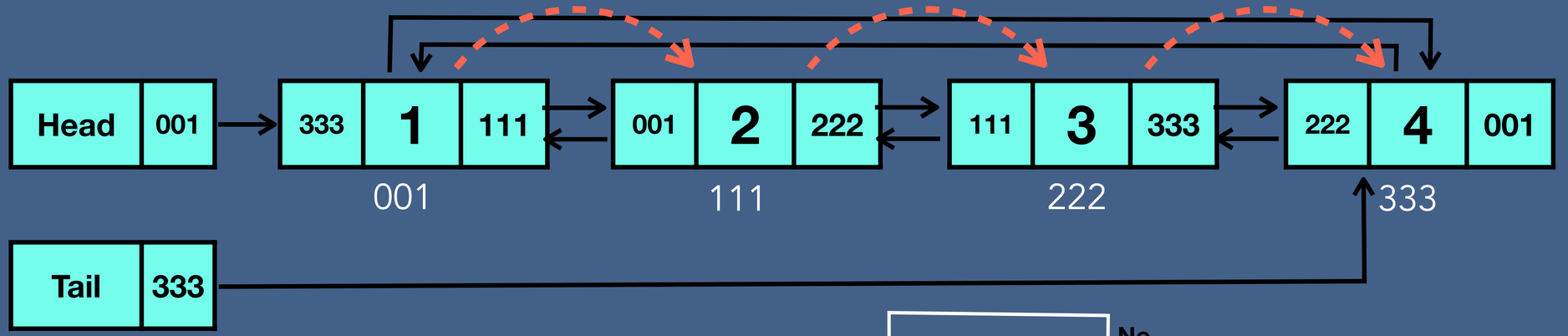
- Insert at the end of linked list

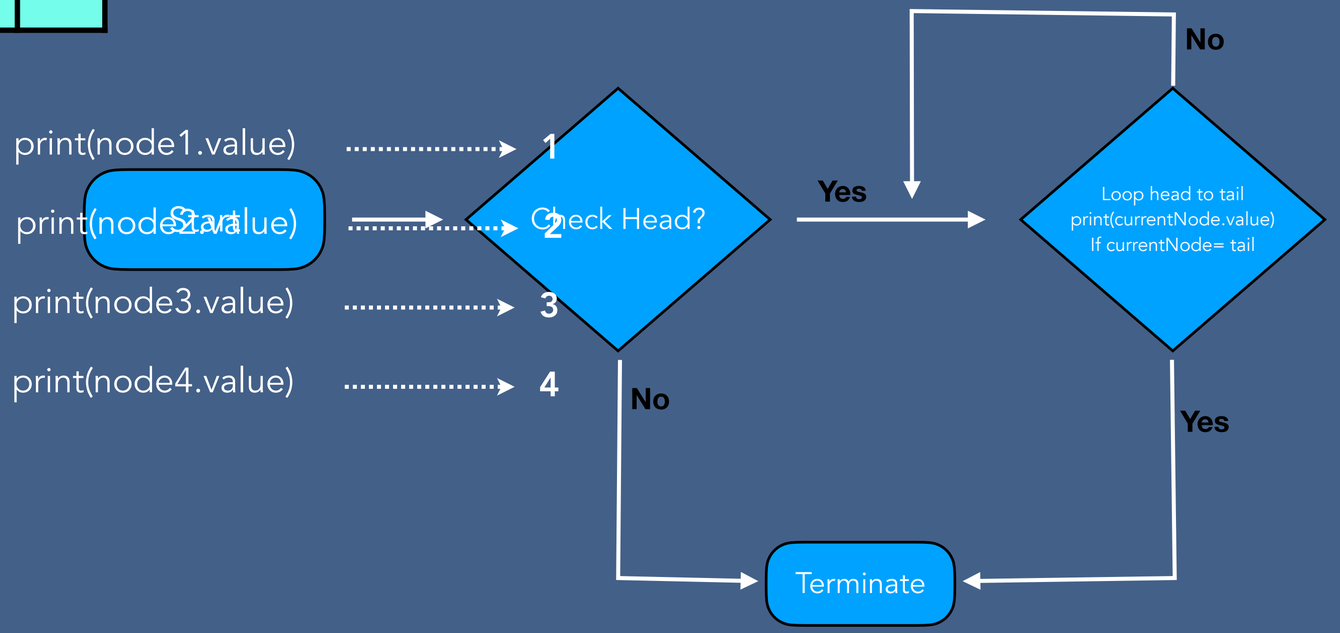


Insertion Algorithm - Circular Doubly Linked List

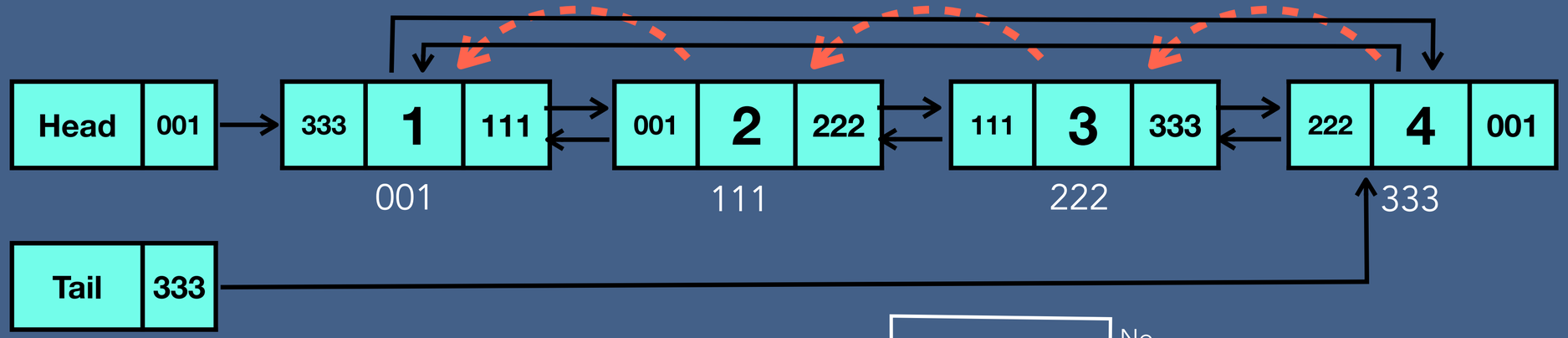


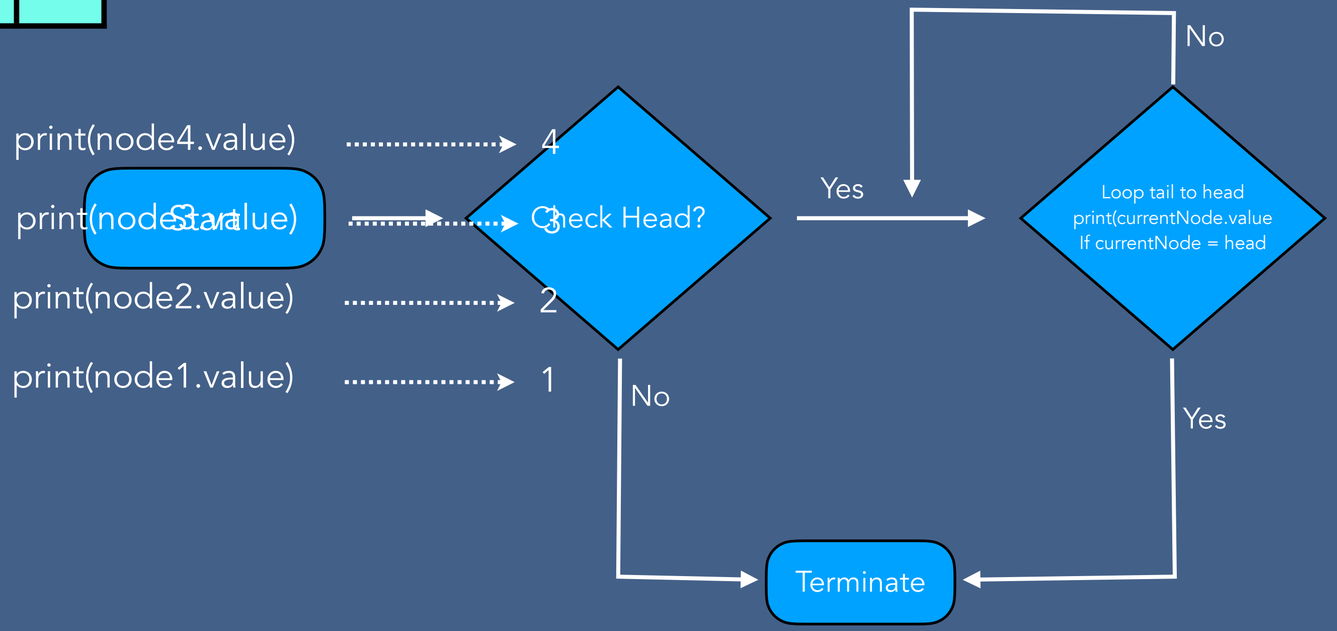
Traversal - Circular Doubly Linked List



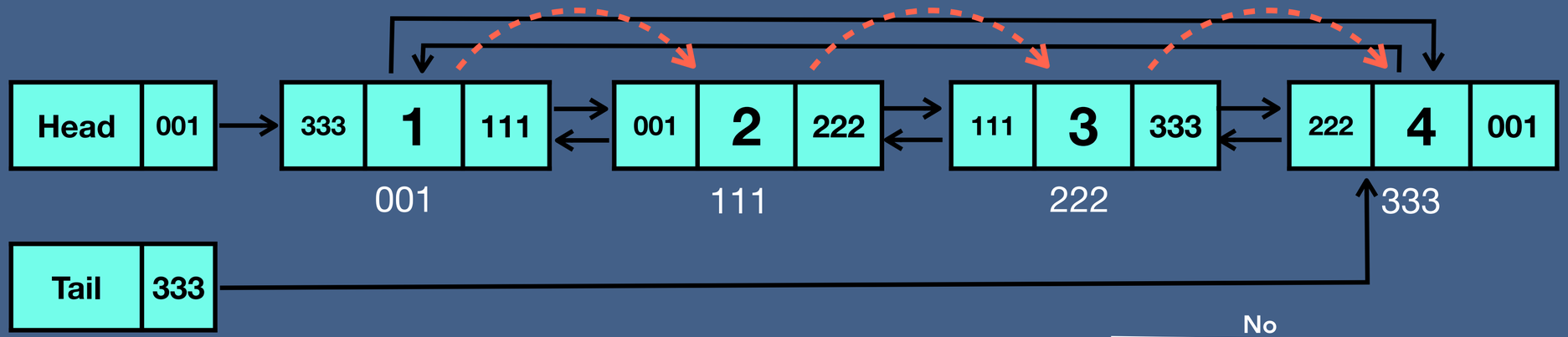


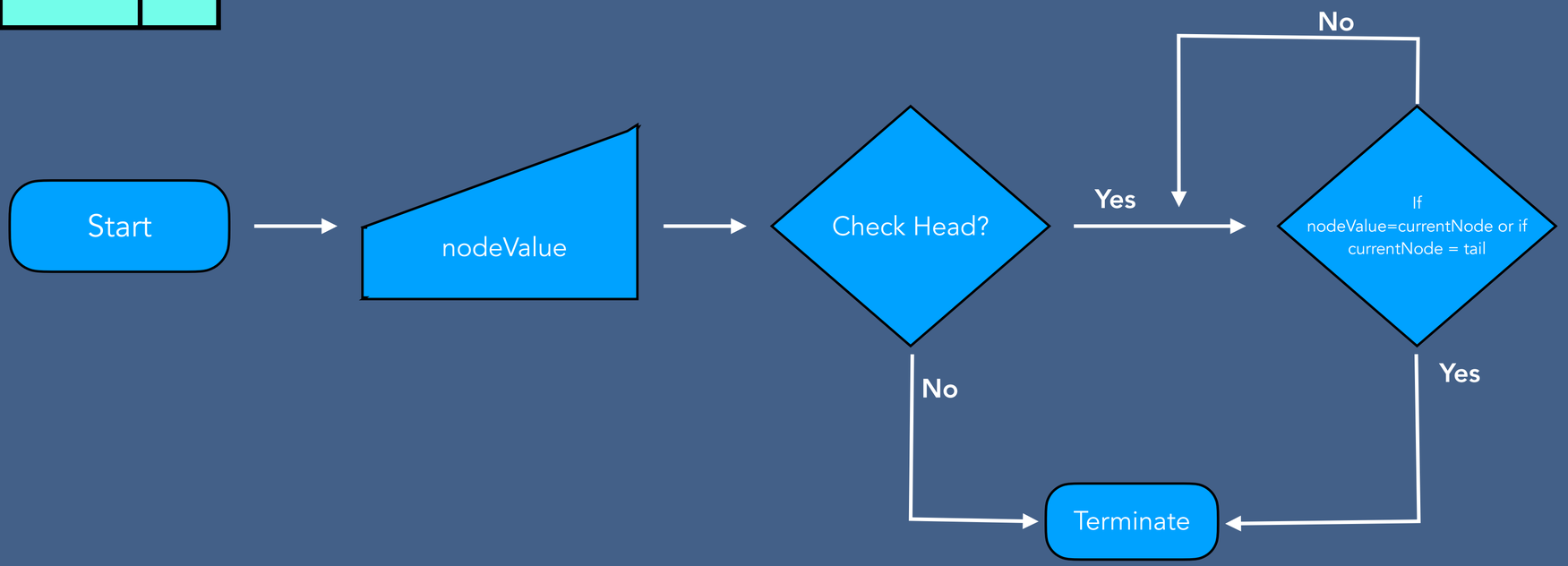
Reverse Traversal - Circular Doubly Linked List





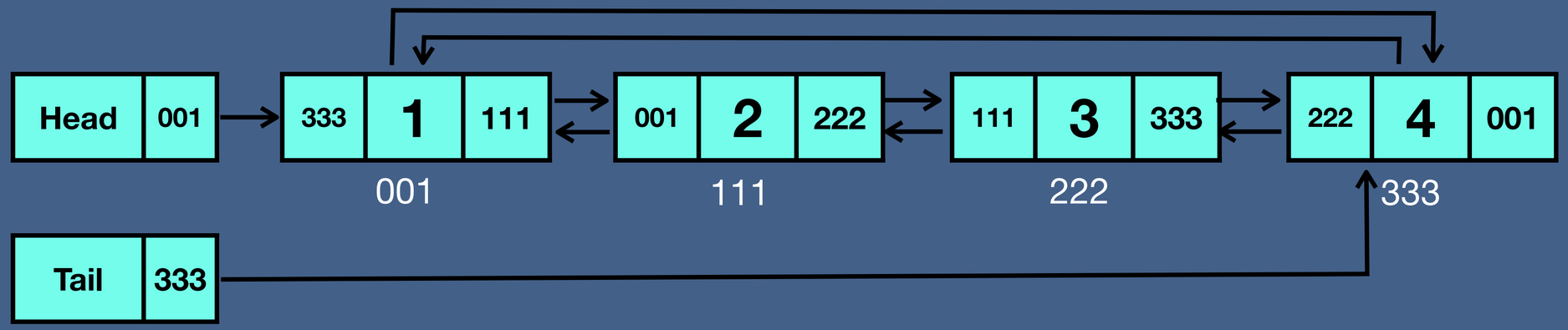
Searching- Circular Doubly Linked List





Deletion - Circular Doubly Linked List

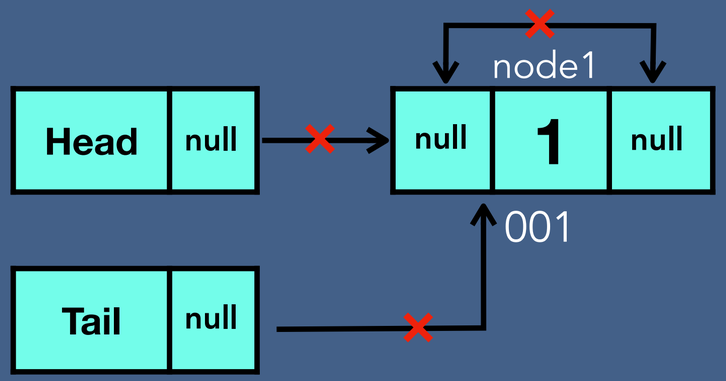
- Deleting the first node
- Deleting any given node
- Deleting the last node



Deletion - Circular Doubly Linked List

Deleting the first node

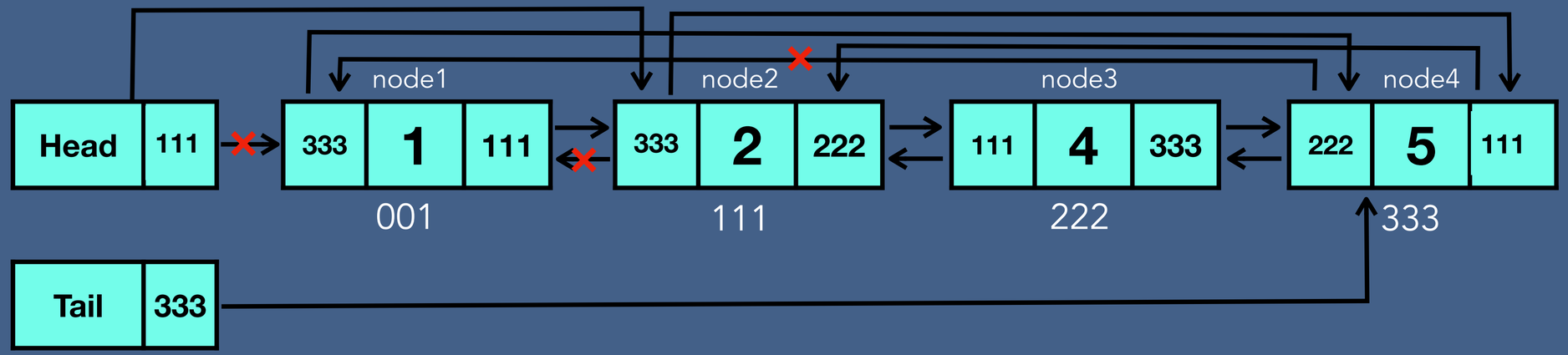
Case 1 - one node



Deletion - Circular Doubly Linked List

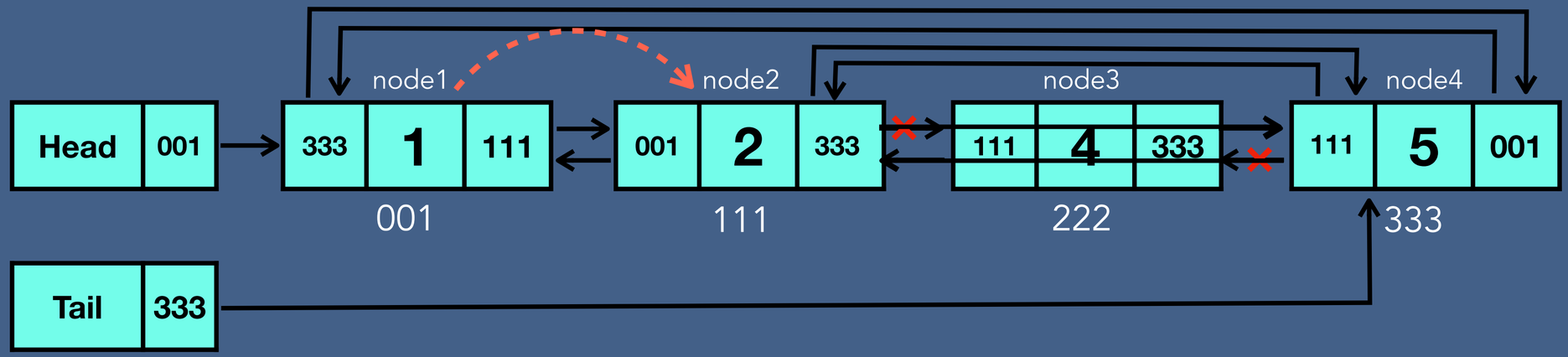
Deleting the first node

Case 2 - more than one node



Deletion - Circular Doubly Linked List

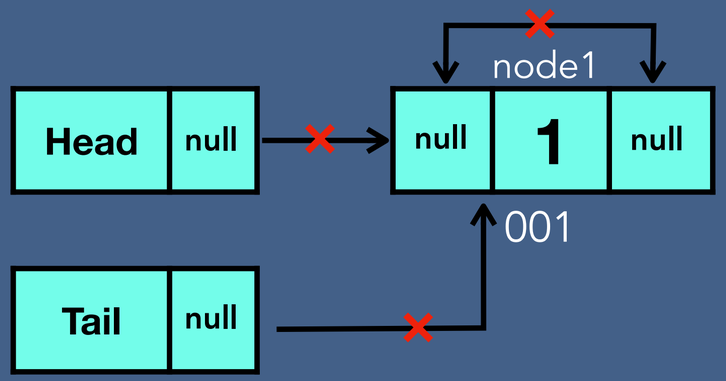
Deleting any given node



Deletion - Circular Doubly Linked List

Deleting the last node

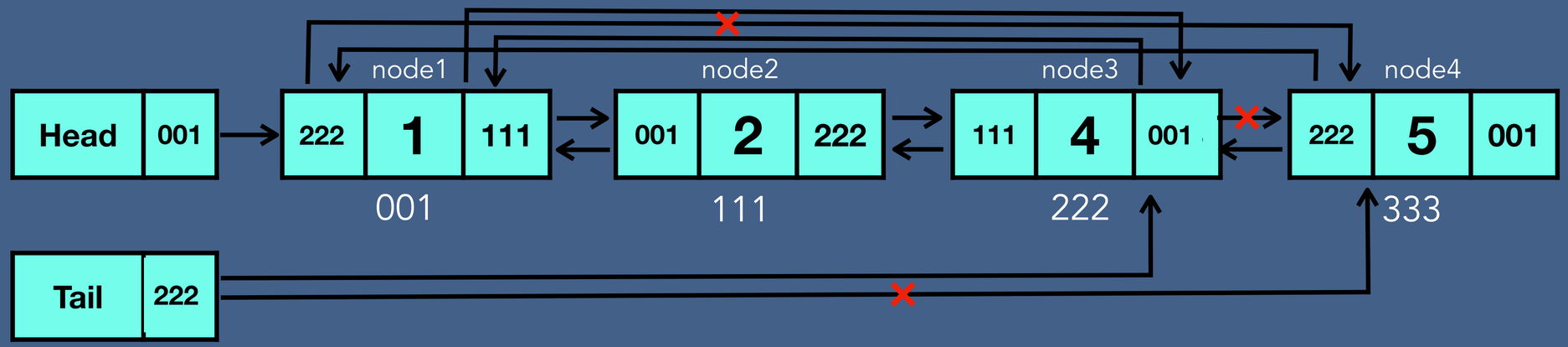
Case 1 - one node



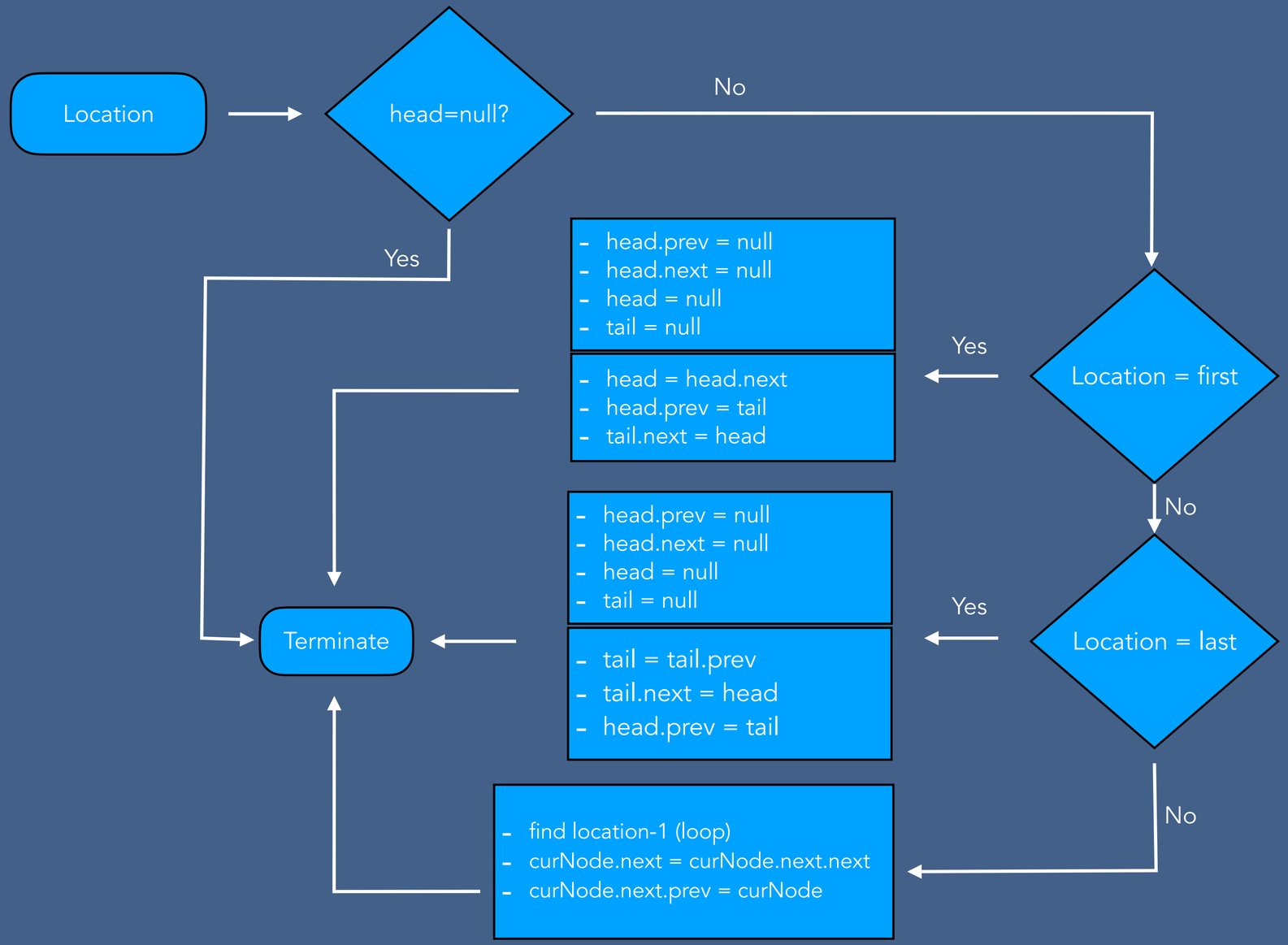
Deletion - Circular Doubly Linked List

Deleting the last node

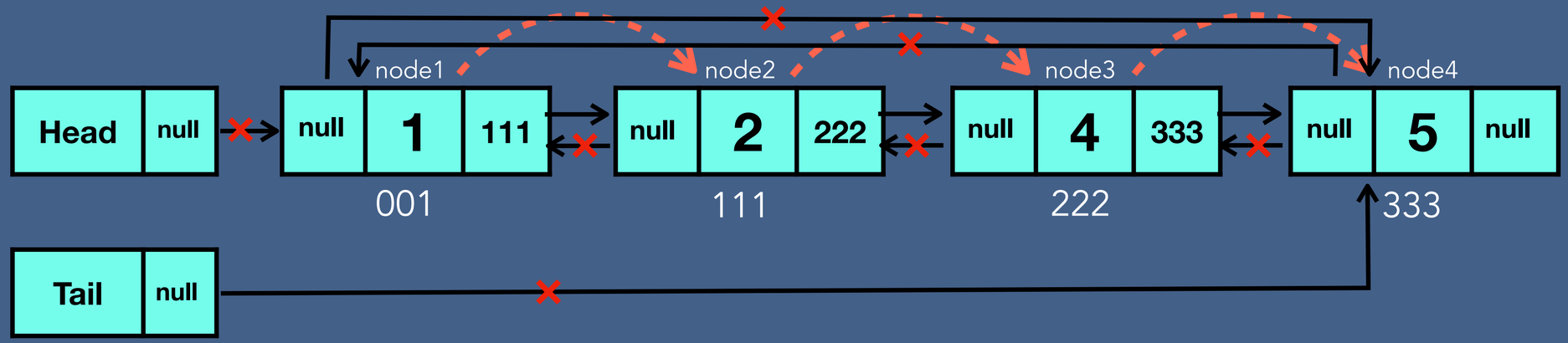
Case 2 - more than one node



Deletion Algorithm - Circular Doubly Linked List

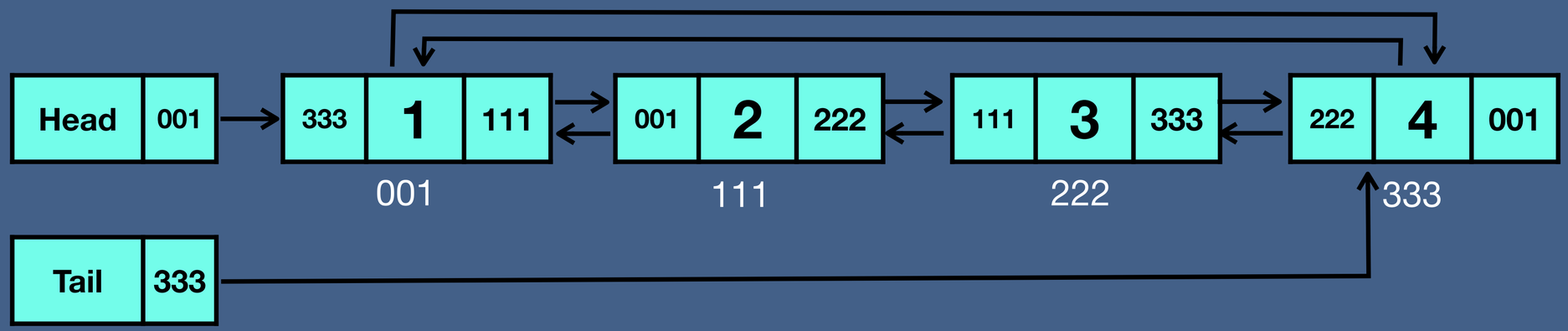


Delete Entire Circular Doubly Linked List



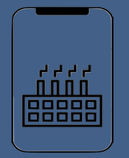
Time and Space Complexity of Circular Doubly Linked List

Circular Doubly Linked List	Time complexity	Space complexity
Creation	O(1)	O(1)
Insertion	O(n)	O(1)
Searching	O(n)	O(1)
Traversing (forward ,backward)	O(n)	O(1)
Deletion of a node	O(n)	O(1)
Deletion of CDLL	O(n)	O(1)

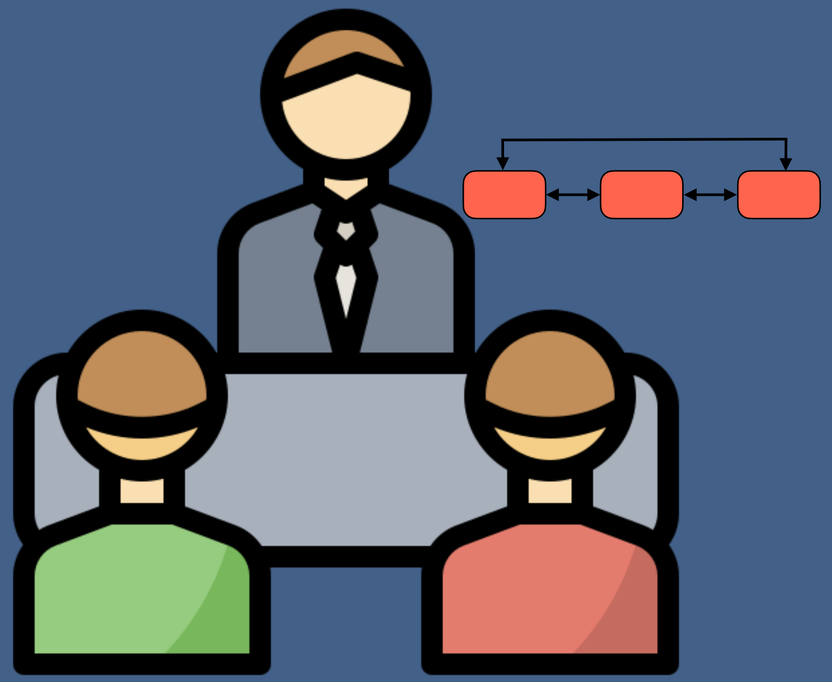


Time Complexity of Array vs Linked List

	Array	Linked List
Creation	O(1)	O(1)
Insertion at first position	O(1)	O(1)
Insertion at last position	O(1)	O(1)
Insertion at n^{th} position	O(1)	O(n)
Searching in Unsorted data	O(n)	O(n)
Searching in Sorted data	O(logn)	O(n)
Traversing	O(n)	O(n)
Deletion at first position	O(1)	O(1)
Deletion at last position	O(1)	O(n)/O(1)
Deletion at n^{th} position	O(1)	O(n)
Deletion of array/linked list	O(1)	O(n)/O(1)
Access n^{th} element	O(1)	O(n)

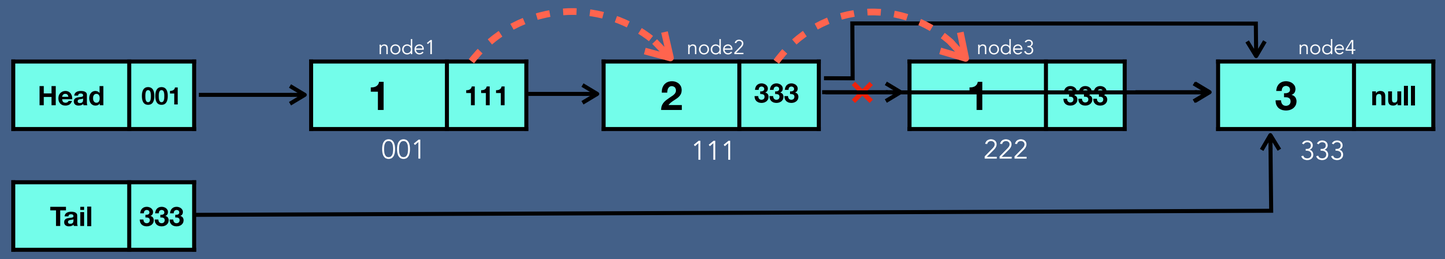


Linked List Interview Questions

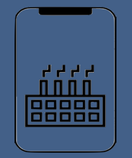


Remove Duplicates

Write a method to remove duplicates from an unsorted linked list.

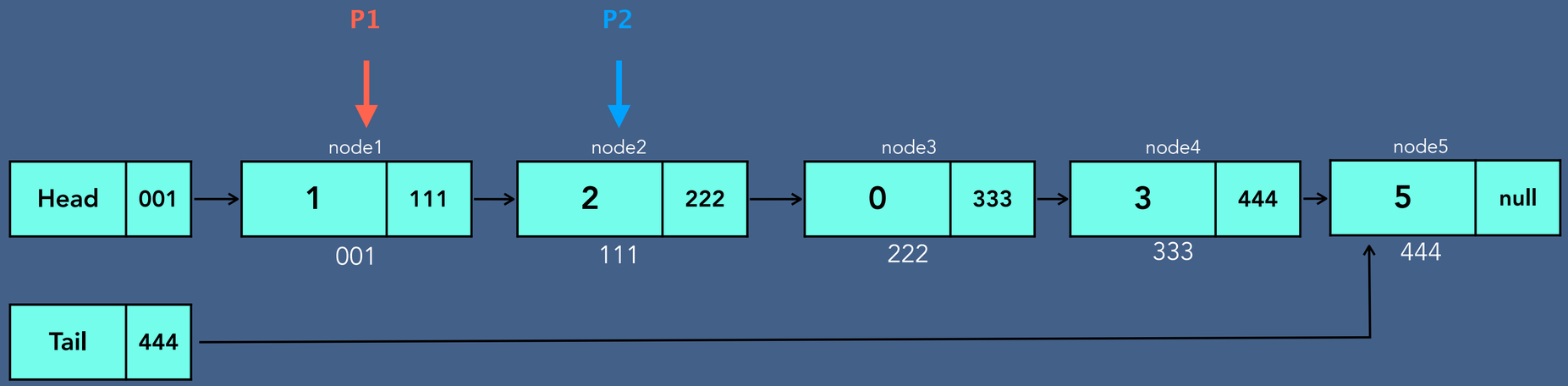


currentNode = node1
hashSet = {1} → {1,2} → {1,2,3}
while currentNode.next is not Null
    If next node's value is in hashSet
        Delete next node
    Otherwise add it to hashSet



Return Nth to Last

Implement and algorithm to find the nth to last element of a singly linked list.



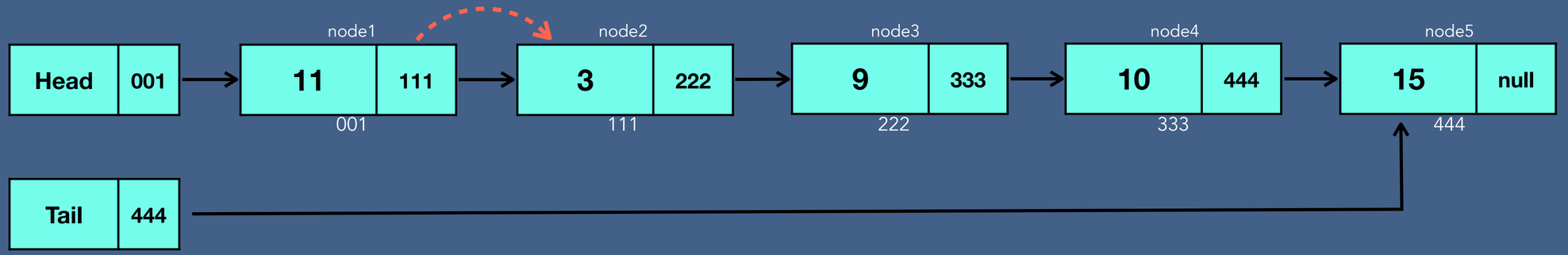
N = 2 ⟶ Node4

pointer1 = node1
= node2
= node3
= node4

pointer2 = node2
= node3
= node4
= node5

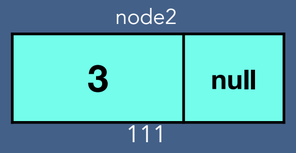
Partition

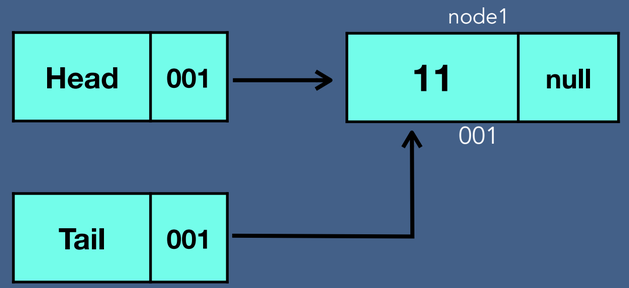
Write code to partition a linked list around a value x, such that all nodes less than x come before all nodes greater than or equal to x.

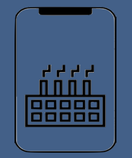


x = 10

currentNode = node1
Tail = node1
currentNode.next = null

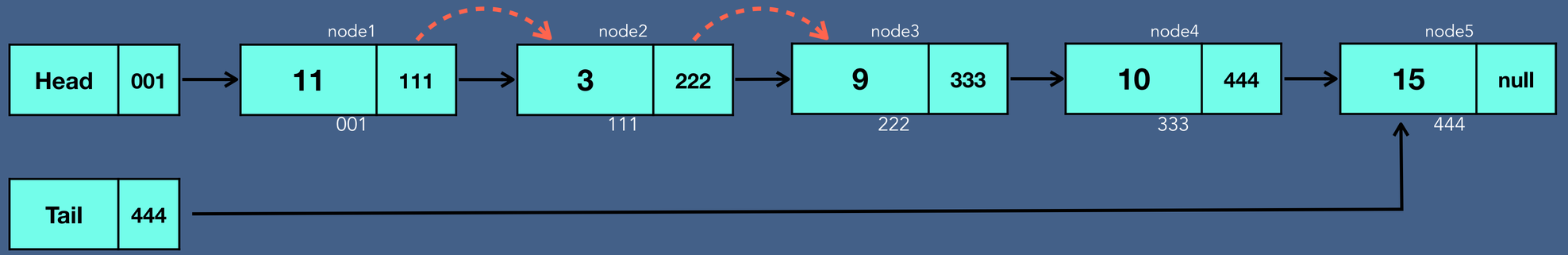






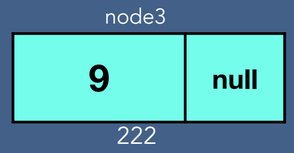
Partition

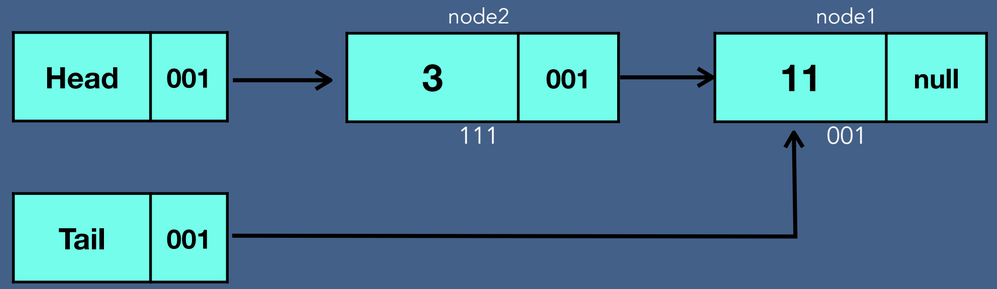
Write code to partition a linked list around a value x, such that all nodes less than x come before all nodes greater than or equal to x.



x = 10

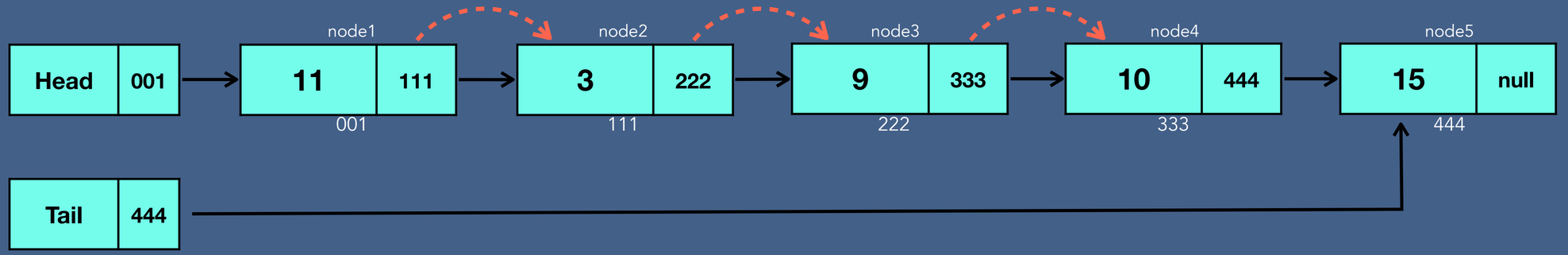
currentNode = node1
Tail = node1
currentNode.next = null





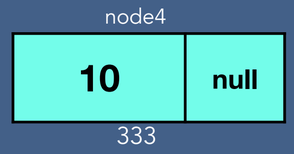
Partition

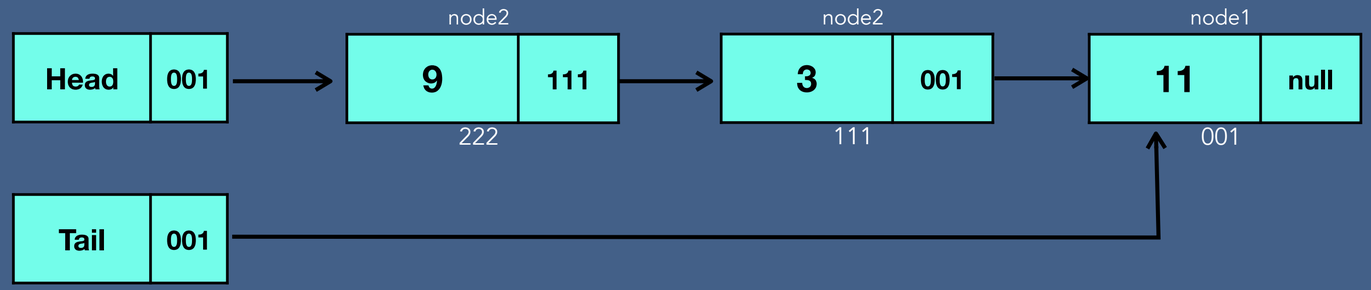
Write code to partition a linked list around a value x, such that all nodes less than x come before all nodes greater than or equal to x.

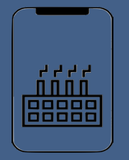


x = 10

currentNode = node1
Tail = node1
currentNode.next = null

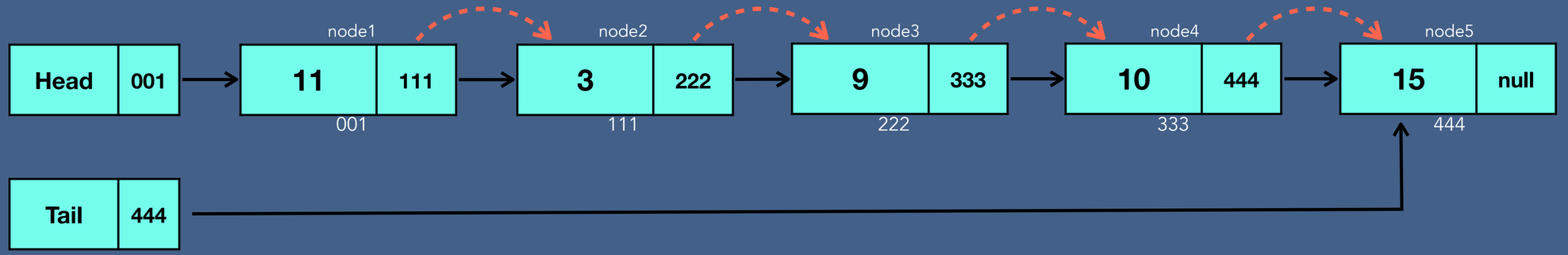






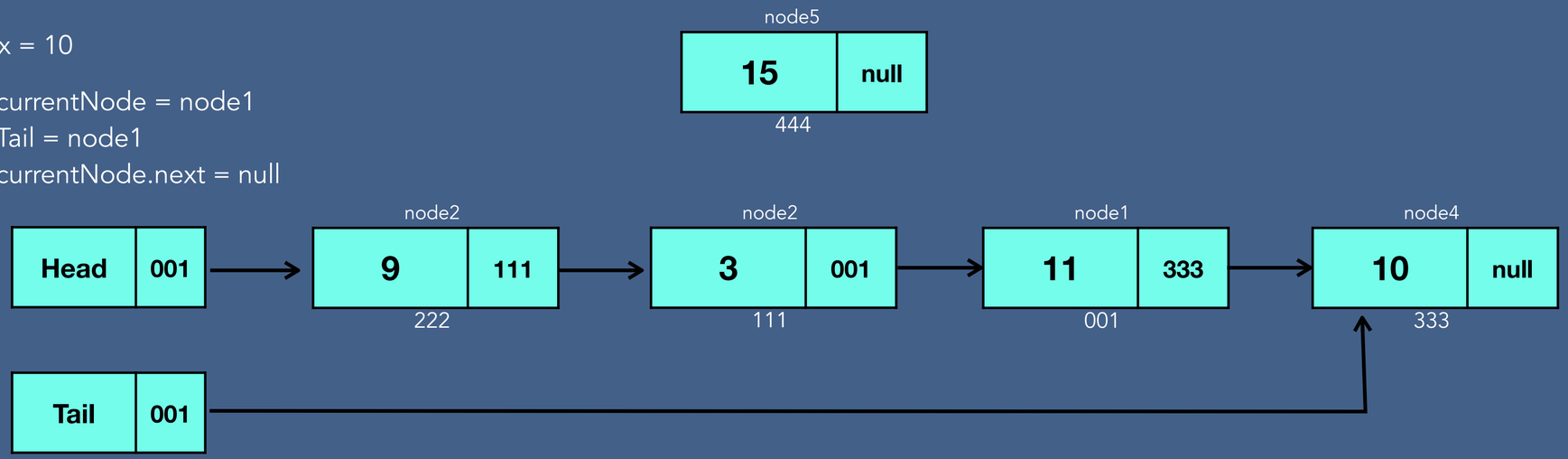
Partition

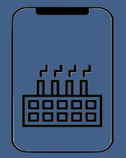
Write code to partition a linked list around a value x, such that all nodes less than x come before all nodes greater than or equal to x.



x = 10

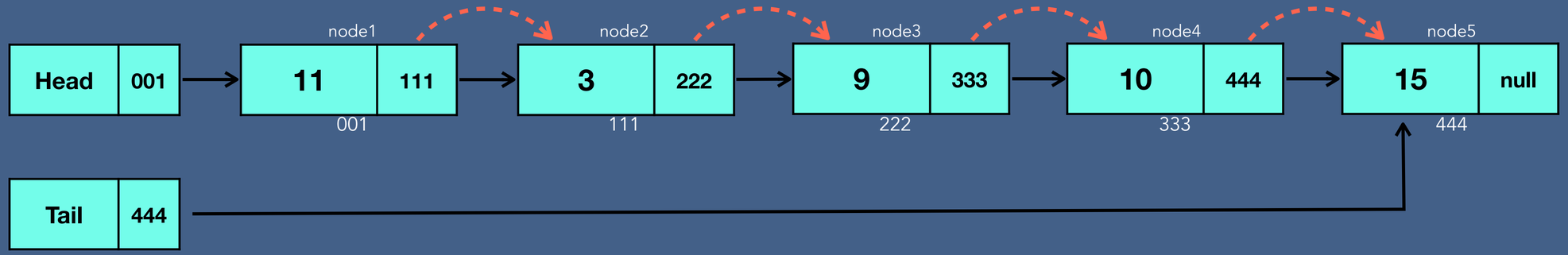
currentNode = node1
Tail = node1
currentNode.next = null





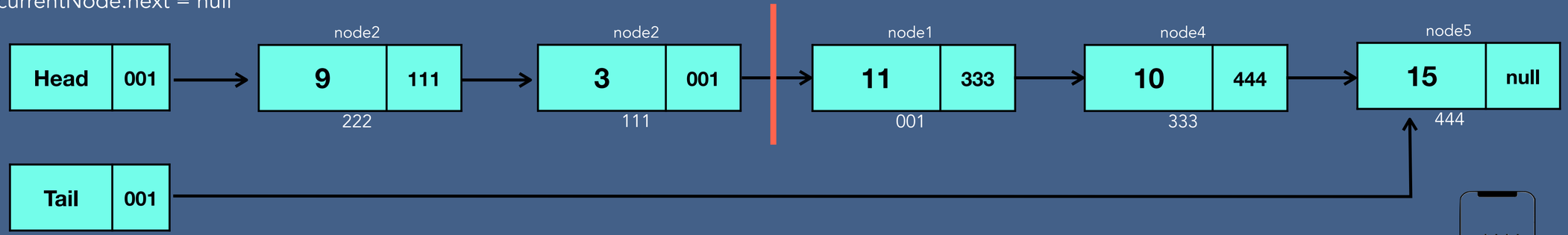
Partition

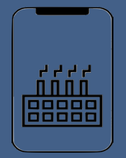
Write code to partition a linked list around a value x, such that all nodes less than x come before all nodes greater than or equal to x.



x = 10

currentNode = node1
Tail = node1
currentNode.next = null





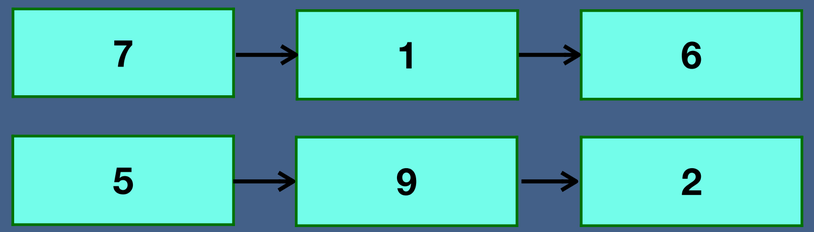
Sum Lists

You have two numbers represented by a linked list, where each node contains a single digit. The digits are stored in reverse order, such that the 1's digit is at the head of the list. Write a function that adds the two numbers and returns the sum as a linked list.

list1 = 7 -> 1 -> 6
list2 = 5 -> 9 -> 2 → 617, 295 → 617 + 295 = 912 → sumList = 2 -> 1 -> 9

```
  617
+ 295
  912
```

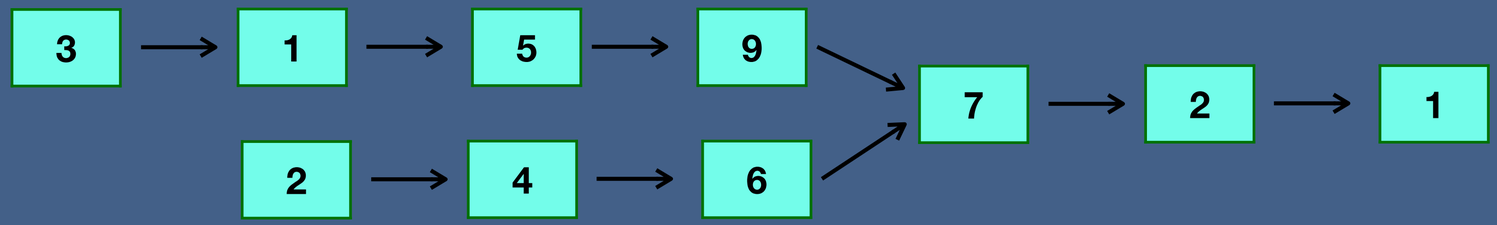
7 + 5 = 12
1+9+1 = 11
6+2+1 = 9



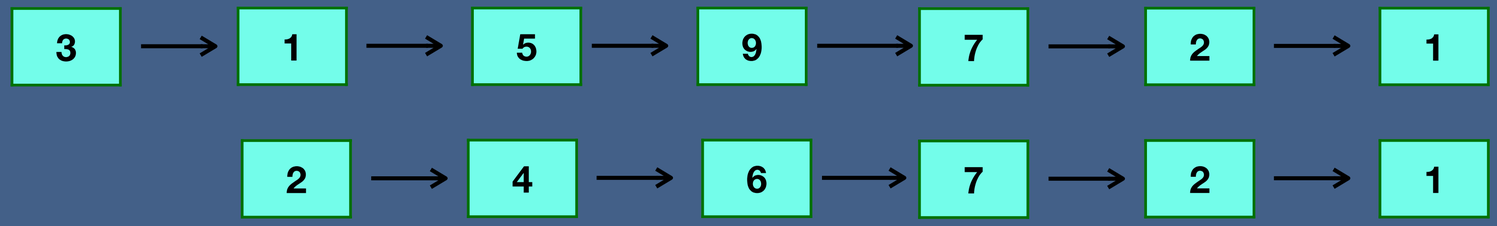
Intersection

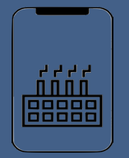
Given two (singly) linked lists, determine if the two lists intersect. Return the intersecting node. Note that the intersection is defined based on reference, not value. That is, if the kth node of the first linked list is the exact same node (by reference) as the jth node of the second linked list, then they are intersecting.

Intersecting linked lists



Non - intersecting linked lists

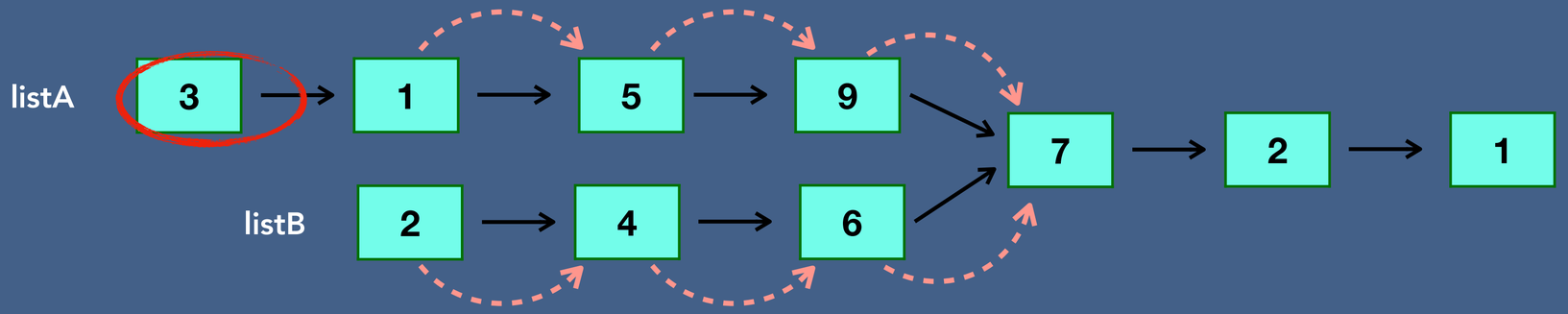




Intersection

Given two (singly) linked lists, determine if the two lists intersect. Return the intersecting node. Note that the intersection is defined based on reference, not value. That is, if the kth node of the first linked list is the exact same node (by reference) as the jth node of the second linked list, then they are intersecting.

Intersecting linked lists



len(listA) = 7
len(listB) = 6 → 7 - 6 = 1